ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Tạ Đình Lương

XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ SÀN GIAO DỊCH THƯƠNG MẠI ĐIỆN TỬ

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin

HÀ NỘI - 2025

LỜI CẢM ƠN

Tôi xin được bày tỏ lòng biết ơn sâu sắc tới tất cả quý thầy cô tại Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, những người đã luôn tận tâm chỉ bảo, chia sẻ kiến thức và kinh nghiệm quý báu, đồng thời định hướng công việc và hỗ trợ tôi trong suốt quá trình học tập. Tôi rất may mắn khi được học tập dưới sự dẫn dắt của các thầy cô, không chỉ có chuyên môn sâu rộng mà còn hết lòng với công tác giảng dạy và phát triển sinh viên. Sự tận tụy của các thầy cô đã tạo ra một môi trường học tập đầy sáng tạo, khuyến khích chúng tôi phát triển không chỉ về kiến thức chuyên môn mà còn về tư duy, kỹ năng nghiên cứu.

Bên cạnh đó, tôi cũng xin cảm ơn gia đình và bạn bè đã luôn đồng hành, ủng hộ và động viên tôi trong suốt quá trình học tập tại Đại học Công nghệ - Đại học Quốc gia Hà Nội. Những người thân yêu trong gia đình luôn là nguồn động viên lớn lao, giúp tôi vượt qua những khó khăn, thử thách, và luôn là chỗ dựa vững chắc khi tôi cảm thấy mệt mỏi hay mất phương hướng. Những lời khuyên, sự động viên và niềm tin của gia đình đã giúp tôi giữ vững mục tiêu học tập và nỗ lực không ngừng. Những người bạn thân thiết cũng là những người luôn sát cánh bên tôi và cùng nhau tạo dựng những kỷ niệm đáng nhớ trong suốt quãng thời gian học đại học.

TÓM TẮT

Tóm tắt: Trong bối cảnh thương mại điện tử ngày càng phát triển mạnh mẽ, việc xây dựng các nền tảng trực tuyến hỗ trợ giao dịch mua bán hiệu quả và thuận tiện trở thành một nhu cầu thiết yếu của xã hội hiện đại. Các giao dịch trực tuyến giúp người tiêu dùng dễ dàng tiếp cận các sản phẩm và dịch vụ mà không bị giới hạn bởi khoảng cách địa lý hay thời gian. Đặc biệt, trong lĩnh vực sản phẩm điện tử, nơi nhu cầu về việc mua bán các thiết bị như điện thoại, laptop, máy tính bảng, phụ kiện điện tử và các thiết bị công nghệ khác luôn rất lớn, việc có một nền tảng hỗ trợ giao dịch chuyên nghiệp là vô cùng quan trọng. Đề tài "Ứng dụng web quản lý sàn giao dịch thương mại điện tử" tập trung vào việc phát triển một hệ thống web dành riêng cho việc quản lý sàn giao dịch các sản phẩm điện tử, với mục đích cung cấp một công cụ tiện lợi, hiệu quả và dễ sử dụng cho cả người mua và người bán.

Hệ thống ứng dụng web này sử dụng các công nghệ tiên tiến như MongoDB, Node.js, Express.js và React.js nhằm đảm bảo tính hiệu quả, mượt mà và khả năng mở rộng. MongoDB là cơ sở dữ liệu NoSQL được chọn để lưu trữ dữ liệu với khả năng mở rộng và xử lý khối lượng lớn thông tin, trong khi Node.js kết hợp với Express.js giúp xây dựng hệ thống backend mạnh mẽ, nhanh chóng và dễ dàng xử lý các yêu cầu của người dùng. React.js, một thư viện JavaScript nổi tiếng, được sử dụng để xây dựng giao diện người dùng, mang lại trải nghiệm mượt mà và tối ưu trên các thiết bị.

Hệ thống sẽ cho phép người dùng đăng ký tài khoản, quản lý sản phẩm, tạo đơn hàng, thực hiện thanh toán trực tuyến và đánh giá sản phẩm. Các tính năng này không chỉ giúp người tiêu dùng dễ dàng tìm kiếm và giao dịch sản phẩm mà còn giúp người bán quản lý hiệu quả cửa hàng trực tuyến của mình, theo dõi doanh thu, tồn kho và thông tin khách hàng. Ứng dụng còn tích hợp các công cụ báo cáo và phân tích để người bán có thể tối ưu hóa chiến lược kinh doanh của mình, từ đó nâng cao hiệu quả bán hàng.

Khóa luận này sẽ trình bày chi tiết quy trình xây dựng, thiết kế hệ thống và phân tích các yếu tố kỹ thuật quan trọng trong việc phát triển ứng dụng web thương mại điện tử sử dụng các công nghệ hiện đại như MongoDB, Node.js, Express.js và React.js. Bên cạnh đó, bài viết cũng sẽ thảo luận về các thử thách và giải pháp trong quá trình phát triển ứng dụng, bao gồm việc đảm bảo tính bảo mật cho các giao dịch trực tuyến và tối ưu hóa hiệu suất hệ thống để phục vụ nhu cầu ngày càng cao của người sử dụng. Đề tài cũng đưa ra những khuyến nghị và hướng phát triển cho hệ thống trong tương lai, nhằm tiếp tục nâng cao chất lượng dịch vụ và đáp ứng được yêu cầu phát triển nhanh chóng của thị trường thương mại điện tử.

***Từ khóa**: Ứng dụng web, thương mại điện tử, MongoDB, Node.js, Express.js, React.js, thiết bị điện tử, quản lý sàn giao dịch, thanh toán trực tuyến.*

LỜI CAM ĐOAN

Tôi xin cam đoan rằng khóa luận tốt nghiệp này là kết quả của quá trình nghiên cứu do chính tôi thực hiện. Toàn bộ nội dung, quá trình nghiên cứu, phân tích và các kết luận được trình bày trong khóa luận đều là kết quả lao động nghiêm túc của cá nhân tôi, được thực hiện theo đúng các yêu cầu và quy định của Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội trong suốt quá trình thực hiện đề tài.

Các thông tin, tài liệu, số liệu, công trình nghiên cứu và ý tưởng của các tác giả khác được sử dụng trong khóa luận đều được trích dẫn đầy đủ, chính xác và đúng quy định, kèm theo nguồn gốc, tên tác giả và các thông tin cần thiết nhằm bảo đảm tính chính xác và độ tin cậy của nội dung nghiên cứu.

Tôi cam kết không sao chép toàn bộ hoặc một phần nội dung của bất kỳ tài liệu, công trình nghiên cứu hay tác phẩm nào mà không ghi rõ nguồn tham khảo. Mọi nội dung trong khóa luận đều do tôi tự nghiên cứu, phân tích, tổng hợp và trình bày trên cơ sở các tài liệu tham khảo hợp pháp, đáng tin cậy và đã được kiểm chứng.

Trong trường hợp phát hiện có sai sót hoặc vi phạm liên quan đến việc sử dụng, trích dẫn tài liệu hoặc quyền sở hữu trí tuệ, tôi xin hoàn toàn chịu trách nhiệm và cam kết thực hiện việc chỉnh sửa, bổ sung theo yêu cầu của nhà trường. Tôi nhận thức rõ rằng hành vi sao chép, sử dụng tài liệu không đúng quy định hoặc gian lận trong nghiên cứu là vi phạm nghiêm trọng đạo đức học thuật và có thể dẫn đến các hình thức xử lý theo quy định hiện hành.

Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực của công trình nghiên cứu này và cam kết chấp hành mọi hình thức xử lý theo quy định của Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội nếu có bất kỳ vi phạm nào liên quan đến việc làm sai lệch thông tin, sao chép tài liệu hoặc gian lận trong quá trình thực hiện khóa luận.

Với những cam kết trên, tôi xác nhận rằng toàn bộ thông tin, số liệu, kết quả và kết luận được trình bày trong khóa luận đều phản ánh đúng quá trình nghiên cứu của bản thân, được thực hiện một cách trung thực, khách quan và phù hợp với các quy định về nghiên cứu khoa học.

Tôi xin trân trọng cảm ơn quý thầy cô và các cán bộ của Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội đã tạo điều kiện, hướng dẫn và hỗ trợ để tôi hoàn thành khóa luận tốt nghiệp này.

Hà Nội, ngày 30 tháng 06 năm 2025

Sinh viên thực hiện

Tạ Đình Lương

Mục lục

LỜI CẢM ƠN **1**

TÓM TẮT **2**

LỜI CAM ĐOAN **3**

DANH SÁCH KÝ HIỆU VÀ CHỮ VIẾT TẮT **7**

Danh sách hình vẽ **8**

Danh sách biểu đồ **9**

Danh sách bảng **10**

Chương 1: Giới thiệu **11**

1.1 Đặt vấn đề 11

1.2 Nội dung khóa luận 11

1.3 Đóng góp của khóa luận 12

1.4 Bố cục khóa luận 13

Chương 2: Kiến thức nền tảng **14**

2.1. Trình bày tổng quan về MERN Stack 14

2.2. NodeJS 16

2.3. ExpressJS 18

2.4. MongoDB 20

2.5. ReactJS 22

2.6. Redux 24

Chương 3: Thu thập yêu cầu và đặc tả yêu cầu **27**

3.1. Thu thập yêu cầu 27

3.2. Yêu cầu chức năng, yêu cầu phi chức năng 27

3.2.1. Yêu cầu chức năng 27

3.2.2. Yêu cầu phi chức năng 29

3.3. Sơ đồ Use Case 31

3.4. Đặc tả Use Case 33

3.4.1. Đăng ký 33
3.4.2. Đăng nhập 34
3.4.3. Chỉnh sửa thông tin cá nhân 35
3.4.4. Tìm kiếm sản phẩm 35
3.4.5. Mua hàng 36
3.4.6. Đặt hàng 37
3.4.7. Quản lý đơn hàng 39
3.4.8. Quản lý sản phẩm 39
3.4.9. Quản lý người dùng 40
3.4.10. Quản lý đơn hàng 41
3.4.11. Đánh giá sản phẩm 42
3.4.11. Nhắn tin 43
3.4.12. Chatbot 43
3.5. Mô tả các biểu đồ hoạt động 45
3.5.1. Đăng ký 45
3.5.2. Đăng nhập 46
3.5.3. Chỉnh sửa thông tin người dùng 47
3.5.4. Tìm kiếm sản phẩm 47
3.5.5. Mua hàng 49
3.5.6. Đặt hàng 49
3.5.7. Quản lý đơn hàng 51
3.5.8. Quản lý sản phẩm 52
3.5.9. Quản lý người dùng 52
3.5.10. Đánh giá sản phẩm 53
3.5.11. Nhắn tin 54
3.5.12. Chatbot 56

Chương 4: Phân tích và thiết kế hệ thống 57
4.1. Phân tích kiến trúc 58
4.2. Biểu đồ tuần tự các Use Case 59
4.2.1. Đăng ký 59
4.2.2. Đăng nhập 59
4.2.3 Tìm kiếm sản phẩm 60

4.2.4 Mua hàng 61
4.2.5 Chỉnh sửa thông tin cá nhân 62
4.2.7 Quản lý sản phẩm 64
4.2.8 Quản lý người dùng 64
4.2.9 Đánh giá sản phẩm 65
4.2.10 Quản lý đơn hàng 66
4.2.11 Nhắn tin 67
4.3 Thiết kế API 68
4.4 Thiết kế cơ sở dữ liệu 68
4.5 Thiết kế giao diện người dùng 70
4.5.1 Giao diện chính của trang web 70
4.5.2 Giao diện đăng nhập 71
4.5.3 Giao diện đăng kí 72
4.5.4 Giao diện sản phẩm 73
4.5.5 Giao diện profile 75
4.5.6 Giao diện mua hàng 75
4.5.7 Giao diện đặt hàng 76
4.5.8 Giao diện đánh giá sản phẩm 76
4.5.9 Giao diện quản lý người dùng 78
4.5.10 Giao diện quản lý sản phẩm 78
4.5.11 Giao diện quản lý đơn hàng 79
4.5.12. Giảm giá 79
4.5.13. Nhắn tin 79
4.5.14. Chatbot 80

Chương 5: Kết luận 81
5.1. Kết quả đạt được 81
5.2. Định hướng phát triển 82

Tài liệu tham khảo 84

DANH SÁCH KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký tự viết tắt	**Tên đầy đủ**	**Giải thích**
API	Application Programming	Giao diện lập trình ứng dụng
BE	Backend	Lớp truy cập dữ liệu
FE	Frontend	Lớp giao diện ứng dụng
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Là một kiểu dữ liệu mở trong JavaScript được định dạng theo cặp "Thuộc tính – Giá trị"
URL	Uniform Resource Locator	Định vị tài nguyên thống nhất hay còn gọi là một địa chỉ web
UI	User interface	Giao diện người dùng
REST	Representational State Transfer	Là một dạng chuyển đổi cấu trúc dữ liệu, một kiểu kiến trúc để viết API
MERN	Mongodb Expressjs Reactjs Nodejs	Các framework và cơ sở dữ liệu để lập trình trang web

Danh sách hình vẽ

Hình 1: Các thành phần của MERN stack....16
Hình 2: Kiến trúc hệ thống thương mại điện tử 58
Hình 3: Biểu đồ tuần tự ca sử dụng đăng ký 59
Hình 4: Biểu đồ tuần tự ca sử dụng đăng nhập 60
Hình 5: Biểu đồ tuần tự ca sử dụng tìm kiếm sản phẩm 61
Hình 6: Biểu đồ tuần tự ca sử dụng mua hàng 61
Hình 7: Biểu đồ tuần tự ca sử dụng chỉnh sửa thông tin cá nhân....62
Hình 8: Biểu đồ tuần tự ca sử dụng đặt hàng 63
Hình 9: Biểu đồ tuần tự ca sử dụng quản lý sản phẩm....64
Hình 10: Biểu đồ tuần tự ca sử dụng quản lý người dùng....65
Hình 11: Biểu đồ tuần tự ca sừ dụng đánh giá sản phẩm 65
Hình 12: Biểu đồ tuần tự ca sử dụng quản lý đơn hàng 66
Hình 13: Biểu đồ tuần tự ca sử dụng nhắn tin....67
Hình 14: Biểu đồ tuần tự ca sử dụng Chatbot 68
Hình 15: Biểu đồ Cơ sở dữ liệu hệ thống....69
Hình 16: Giao diện trang chủ 70
Hình 17: Giao diện đăng nhập....71
Hình 18: Giao diện đăng ký tài khoản....72
Hình 19: Giao diện sản phẩm....73
Hình 20: Giao diện tìm kiếm sản phẩm....74
Hình 21: Giao diện chi tiết sản phẩm....74
Hình 22: Giao diện thông tin cá nhân người dùng 75
Hình 23: Giao diện mua hàng 75
Hình 24: Giao diện đặt hàng 76
Hình 25: Giao diện đánh giá sản phẩm 77
Hình 26: Giao diện quản lý người dùng....78
Hình 27: Giao diện quản lý sản phẩm....78
Hình 28: Giao diện thanh toán thành công....79
Hình 29: Giao diện giảm giá 79
Hình 30: Giao diện nhắn tin 80
Hình 31: Giao diện Chatbot 80

Danh sách biểu đồ

Biểu đồ 1: Biểu đồ ca sử dụng tổng quát cảu website.....32
Biểu đồ 2: Biểu đồ hoạt động ca sử dụng đăng ký.....45
Biểu đồ 3: Biểu đồ hoạt động ca sử dụng đăng nhập.....46
Biểu đồ 4: Biểu đồ hoạt động ca sử dụng chỉnh sửa thông tin cá nhân.....47
Biểu đồ 5: Biểu đồ hoạt động ca sử dụng tìm kiếm sản phẩm.....48
Biểu đồ 6: Biểu đồ hoạt động ca sử dụng mua hàng.....49
Biểu đồ 7: Biểu đồ hoạt động ca sử dụng đặt hàng.....50
Biểu đồ 8: Biểu đồ hoạt động ca sử dụng quản lý đơn hàng.....51
Biểu đồ 9: Biểu đồ hoạt động ca sử dụng quản lý sản phẩm.....52
Biểu đồ 10: Biểu đồ hoạt động ca sử dụng quản lý người dùng.....53
Biểu đồ 11: Biểu đồ hoạt động ca sử dụng đánh giá sản phẩm.....54
Biểu đồ 12: Biểu đồ hoạt động ca sử dụng nhắn tin.....55
Biểu đồ 13: Biểu đồ hoạt động ca sử dụng Chatbot.....57

Danh sách bảng

Bảng 1: Mô tả ca sử dụng đăng ký 33
Bảng 2: Mô tả ca sử dụng đăng nhập 34
Bảng 3: Mô tả ca sử dụng chỉnh sửa thông tin cá nhân 35
Bảng 4: Mô tả ca sử dụng tìm kiếm sản phẩm 36
Bảng 5: Mô tả ca sử dụng mua hàng 36
Bảng 6: Mô tả ca sử dụng thanh toán 37
Bảng 7: Mô tả ca sử dụng quản lý đơn hàng 39
Bảng 8: Mô tả ca sử dụng quản lý sản phẩm 39
Bảng 9: Mô tả ca sử dụng quản lý người dùng 40
Bảng 10: Mô tả ca sử dụng quản lý đơn hàng 41
Bảng 11: Mô tả ca sử dụng đánh giá sản phẩm 42
Bảng 12: Mô tả ca sử dụng nhắn tin 43
Bảng 13: Mô tả ca sử dụng chatbot 43
Bảng 14: Các chức năng được kiểm tra ..... **Error! Bookmark not defined.**

Chương 1: Giới thiệu

1.1 Đặt vấn đề

Trong thời đại công nghệ số phát triển mạnh mẽ, kinh doanh trực tuyến đã trở thành một phần không thể thiếu trong chiến lược của nhiều doanh nghiệp. Nhu cầu mua sắm tiện lợi, nhanh chóng của người tiêu dùng thúc đẩy các doanh nghiệp đầu tư vào website bán hàng như một kênh tiếp cận khách hàng hiệu quả, đồng thời giúp mở rộng thị trường và tăng trưởng doanh thu.

So với mô hình kinh doanh truyền thống, website giúp tiết kiệm chi phí vận hành, đồng thời cho phép doanh nghiệp chủ động trong việc giới thiệu sản phẩm, xây dựng thương hiệu và triển khai các chiến lược marketing. Việc mua sắm chỉ với một thiết bị kết nối Internet cũng mang đến trải nghiệm thuận tiện, phù hợp với xu hướng tiêu dùng hiện nay.

Một website bán hàng hiệu quả không chỉ cần có giao diện thân thiện, dễ sử dụng mà còn phải đảm bảo tính ổn định, tốc độ truy cập nhanh và khả năng quản lý đơn hàng, sản phẩm một cách khoa học. Bên cạnh đó, việc tích hợp các công cụ hỗ trợ khách hàng như trò chuyện trực tuyến, theo dõi đơn hàng hay thông báo khuyến mãi cũng góp phần nâng cao trải nghiệm và giữ chân người dùng.

So với việc phụ thuộc hoàn toàn vào các sàn thương mại điện tử, website riêng mang lại sự linh hoạt và chủ động hơn trong vận hành, đồng thời giúp doanh nghiệp xây dựng hình ảnh thương hiệu rõ nét và lâu dài. Đây cũng là cơ sở để triển khai các hoạt động kinh doanh sáng tạo, cá nhân hóa trải nghiệm người dùng và phát triển mối quan hệ bền vững với khách hàng.

Tiếp theo, bài khoá luận của chúng tôi sẽ trình bày chi tiết hơn về các bước trong quá trình xây dựng hệ thống website bán hàng này, từ việc phân tích yêu cầu, thiết kế hệ thống cho đến triển khai và kiểm thử. Những kết quả đạt được từ việc áp dụng hệ thống này sẽ đóng góp vào sự phát triển không chỉ của doanh nghiệp mà còn của toàn bộ nền kinh tế số trong tương lai.

1.2 Nội dung khóa luận

Trong bối cảnh công nghệ phát triển nhanh chóng, thương mại điện tử ngày càng đóng vai trò quan trọng trong đời sống hiện đại. Việc giao dịch, mua bán các sản phẩm điện tử như điện thoại, laptop, phụ kiện... đã trở nên phổ biến nhờ các nền tảng trực tuyến. Trước nhu cầu đó, đề tài "Ứng dụng web quản lý sàn giao dịch thương mại điện tử" được xây dựng nhằm cung cấp một công cụ hỗ trợ hiệu quả cho cả người mua lẫn người bán trong môi trường số.

Hệ thống được phát triển dựa trên các công nghệ hiện đại như MongoDB, Node.js, Express.js và React.js để đảm bảo tính ổn định, hiệu suất và khả năng mở rộng. Người dùng có thể đăng ký tài khoản, tìm kiếm, đặt hàng, thanh toán và đánh giá sản phẩm, trong khi người bán có thể quản lý hàng hóa, theo dõi doanh số và xử lý đơn hàng một cách thuận tiện.

Quá trình xây dựng bắt đầu từ việc phân tích nhu cầu thực tế để phát triển giao diện dễ dùng và đầy đủ chức năng. Toàn bộ hệ thống được thiết kế dưới dạng mô hình client-server với các API kết nối chặt chẽ giữa frontend, backend và cơ sở dữ liệu, nhằm đảm bảo hiệu quả vận hành.

Sau khi hoàn thiện, hệ thống đã trải qua các bước kiểm thử chức năng, bảo mật và hiệu năng để đánh giá mức độ đáp ứng yêu cầu người dùng trong điều kiện thực tế. Kết quả cho thấy hệ thống vận hành ổn định, xử lý tốt các giao dịch và đảm bảo an toàn thông tin.

Tuy đã đạt được mục tiêu đề ra, hệ thống vẫn còn nhiều tiềm năng phát triển. Trong tương lai, việc mở rộng thêm phương thức thanh toán, cải thiện công cụ tìm kiếm, tích hợp chatbot hỗ trợ khách hàng và phát triển ứng dụng di động sẽ giúp nâng cao trải nghiệm người dùng và đáp ứng tốt hơn nhu cầu thị trường.

1.3 Đóng góp của khóa luận

Khóa luận "Ứng dụng web quản lý sàn giao dịch thương mại điện tử" mang lại nhiều đóng góp thiết thực trong lĩnh vực phát triển hệ thống thương mại số, đặc biệt là đối với các nền tảng chuyên về sản phẩm điện tử. Hệ thống được xây dựng nhằm kết nối hiệu quả giữa người mua và người bán thông qua các tính năng quản lý tài khoản, theo dõi đơn hàng, thanh toán trực tuyến và đánh giá sản phẩm – đáp ứng đúng nhu cầu thực tiễn trong giao dịch điện tử hiện nay.

Một điểm nổi bật khác là việc ứng dụng đồng bộ các công nghệ hiện đại như MongoDB, Node.js, Express.js và React.js, giúp hệ thống vận hành mượt mà, có khả năng xử lý dữ liệu lớn và đảm bảo khả năng mở rộng lâu dài. Việc sử dụng kiến trúc web hiện đại cũng tối ưu hóa trải nghiệm người dùng và cải thiện hiệu suất vận hành cho cả người dùng cuối và nhà quản trị hệ thống.

Khóa luận cũng đóng góp vào việc phát triển các tính năng hỗ trợ người bán như quản lý sản phẩm, tồn kho, doanh thu và thông tin khách hàng. Điều này không chỉ giúp tối ưu hoạt động kinh doanh mà còn hỗ trợ người bán đưa ra các quyết định chiến lược chính xác hơn, từ đó nâng cao hiệu quả kinh doanh trên nền tảng số.

Đặc biệt, hệ thống còn tích hợp chatbot thông minh sử dụng mô hình ngôn ngữ lớn (LLM) để hỗ trợ người dùng mọi lúc. Chatbot giúp tra cứu sản phẩm, theo dõi đơn hàng và giải đáp thắc mắc nhanh chóng, góp phần giảm tải cho bộ phận chăm sóc khách hàng và nâng cao mức độ hài lòng của người sử dụng.

Ngoài ra, tính năng nhắn tin trực tiếp giữa người mua và người bán cũng được phát triển, tạo điều kiện giao tiếp hai chiều hiệu quả và tăng độ tin cậy trong quá trình giao dịch. Đây là cơ sở để mở rộng thêm nhiều tính năng tương tác nâng cao trong tương lai, như chia sẻ hình ảnh, phản hồi tự động hoặc thông báo theo thời gian thực.

Tổng thể, khóa luận không chỉ hoàn thành mục tiêu xây dựng một nền tảng giao dịch hiệu quả mà còn mở ra hướng phát triển cho các mô hình thương mại điện tử thông minh, tích hợp AI, hướng đến trải nghiệm người dùng toàn diện hơn. Đây là đóng góp có giá trị trong bối cảnh thương mại số ngày càng cạnh tranh và đổi mới nhanh chóng.

1.4 Bố cục khóa luận

Khóa luận gồm 6 chương với nội dung tóm tắt như sau:

- **Chương 1 – Giới thiệu:** Trình bày tổng quan đề tài, mục tiêu, lý do lựa chọn và phạm vi nghiên cứu. Chương này định hướng vấn đề cần giải quyết và cấu trúc toàn khóa luận, tạo nền tảng cho phần nội dung tiếp theo.

- **Chương 2 – Cơ sở lý thuyết:** Cung cấp kiến thức nền liên quan đến các công nghệ sử dụng trong hệ thống như MongoDB, Node.js, Express.js, React.js và Redux. Các nội dung này giúp củng cố tính khả thi và lựa chọn công nghệ phù hợp.

- **Chương 3 – Phân tích yêu cầu:** Xác định rõ các yêu cầu chức năng và phi chức năng thông qua biểu đồ và mô hình trực quan. Nội dung tập trung vào việc làm rõ các chức năng chính như quản lý tài khoản, đơn hàng, chatbot hỗ trợ và hệ thống nhắn tin.

- **Chương 4 – Thiết kế hệ thống:** Trình bày kiến trúc tổng thể, cấu trúc cơ sở dữ liệu, giao diện người dùng và luồng xử lý dữ liệu. Thiết kế được xây dựng theo mô hình client-server, đảm bảo dễ mở rộng và bảo trì.

- **Chương 5 – Triển khai và kiểm thử:** Mô tả quá trình xây dựng hệ thống, tích hợp các chức năng chính và kiểm thử toàn diện để đảm bảo hiệu năng, độ ổn định và an toàn thông tin. Kiểm thử bao gồm từ cấp đơn vị đến tích hợp và giao diện người dùng.

- **Chương 6 – Kết luận và định hướng:** Tổng kết kết quả đạt được, đánh giá hiệu quả hệ thống và đưa ra đề xuất phát triển trong tương lai như tích hợp học máy, cải tiến nhắn tin thời gian thực và mở rộng ứng dụng sang các lĩnh vực khác.

Chương 2: Kiến thức nền tảng

Hệ thống website bán hàng được lập trình bằng ngôn ngữ JavaScript, sử dụng bộ công cụ MERN Stack, bao gồm MongoDB, Express.js, React.js và Node.js, nhằm xây dựng một nền tảng giao dịch trực tuyến mạnh mẽ và hiệu quả. MongoDB, một cơ sở dữ liệu NoSQL, đóng vai trò quan trọng trong việc lưu trữ và quản lý thông tin về các đơn hàng, sản phẩm và người dùng. Mọi dữ liệu sẽ được lưu trữ trên Cloud, giúp hệ thống dễ dàng mở rộng và đảm bảo tính sẵn sàng cao, đồng thời giảm thiểu rủi ro mất mát dữ liệu và nâng cao hiệu suất khi truy cập từ xa.

Thông qua việc đăng ký tài khoản, người dùng có thể dễ dàng truy cập và theo dõi thông tin các đơn hàng của mình, từ tình trạng đơn hàng, các sản phẩm đã đặt, đến lịch sử giao dịch và các chi tiết khác. Mỗi tài khoản người dùng đều được gắn liền với một bộ dữ liệu cá nhân, giúp hệ thống bảo mật thông tin và cung cấp trải nghiệm mua sắm cá nhân hóa cho khách hàng. Mỗi khi người dùng thực hiện thao tác như thêm sản phẩm vào giỏ hàng hay tiến hành thanh toán, hệ thống sẽ ngay lập tức cập nhật các thay đổi vào cơ sở dữ liệu MongoDB và phản ánh chúng trên giao diện của người dùng.

Giao diện của hệ thống được phát triển bằng React.js, một thư viện JavaScript mạnh mẽ, giúp xây dựng các ứng dụng web động với khả năng tương tác cao. Một trong những tính năng nổi bật của React.js là khả năng "render lại" giao diện chỉ khi có sự thay đổi trong dữ liệu, thay vì phải tải lại toàn bộ trang web như các phương pháp truyền thống. Điều này giúp tăng tốc độ hiển thị của trang, giảm tải cho máy chủ và giảm thời gian chờ đợi của người dùng. Với React.js, các thao tác của người dùng trở nên mượt mà và nhanh chóng hơn, đồng thời cải thiện hiệu suất tổng thể của hệ thống, đặc biệt khi hệ thống phải xử lý lượng lớn dữ liệu và người dùng cùng lúc.

Ngoài ra, việc sử dụng MERN Stack giúp tối ưu hóa toàn bộ quy trình phát triển, từ phía backend với Node.js và Express.js cho đến frontend với React.js. Node.js, với khả năng xử lý đồng thời nhiều kết nối, giúp hệ thống có thể phục vụ nhiều người dùng cùng lúc mà không bị gián đoạn. Express.js, một framework nhẹ cho Node.js, cung cấp các công cụ và middleware hỗ trợ xây dựng các API RESTful hiệu quả, cho phép frontend và backend giao tiếp mượt mà. Tất cả các công nghệ trong MERN Stack đều hỗ trợ JavaScript, giúp tăng tính linh hoạt và đồng bộ hóa trong quá trình phát triển, từ đó tạo ra một hệ thống bán hàng trực tuyến dễ bảo trì và mở rộng.

2.1. Trình bày tổng quan về MERN Stack

MERN Stack [1] là một bộ công nghệ mạnh mẽ được sử dụng trong việc phát triển các ứng dụng web hiện đại, được xây dựng trên nền tảng JavaScript. MERN Stack bao gồm bốn thành phần chính: MongoDB, Express.js, React.js và Node.js. Mỗi thành phần của MERN Stack đảm nhận một vai trò quan trọng trong việc xây dựng các ứng dụng web có hiệu suất cao và khả năng mở rộng tốt. Các công nghệ này đều sử dụng JavaScript, cho phép phát triển cả frontend và backend bằng một ngôn ngữ duy nhất, điều này mang lại sự đồng nhất và dễ dàng trong quá trình phát triển và bảo trì hệ thống.

MERN Stack được sử dụng rộng rãi trong việc xây dựng các ứng dụng web từ đơn giản đến phức tạp, đặc biệt trong các ứng dụng cần xử lý lượng lớn dữ liệu, yêu cầu tốc độ phản hồi nhanh và khả năng mở rộng cao. Các công nghệ này cũng rất phổ biến trong cộng đồng phát triển phần mềm nhờ vào tính linh hoạt, sự dễ dàng tích hợp với nhau và khả năng hỗ trợ phát triển ứng dụng theo mô hình RESTful.

MongoDB là cơ sở dữ liệu NoSQL được sử dụng trong MERN Stack để lưu trữ và quản lý dữ liệu. MongoDB lưu trữ dữ liệu dưới dạng tài liệu JSON (JavaScript Object Notation) thay vì lưu trong bảng như các hệ quản trị cơ sở dữ liệu quan hệ truyền thống. Điều này giúp MongoDB có thể mở rộng dễ dàng và xử lý các dữ liệu không cấu trúc hoặc bán cấu trúc, rất phù hợp với các ứng dụng web hiện đại nơi dữ liệu có thể thay đổi thường xuyên. MongoDB cung cấp khả năng lưu trữ dữ liệu với một cấu trúc linh hoạt và dễ dàng truy vấn thông qua các công cụ như Mongoose, giúp đơn giản hóa việc tương tác với cơ sở dữ liệu.

Express.js, là một framework cho Node.js, giúp xây dựng các API RESTful và xử lý các yêu cầu HTTP dễ dàng hơn. Express.js giúp việc phát triển backend trở nên nhanh chóng và dễ dàng bằng cách cung cấp các middleware có sẵn để xử lý các tác vụ như xác thực, kiểm tra dữ liệu đầu vào và quản lý lỗi. Với Express, các lập trình viên có thể xây dựng các ứng dụng web với khả năng mở rộng tốt và quản lý các yêu cầu HTTP hiệu quả.

React.js là một thư viện JavaScript do Facebook phát triển để xây dựng giao diện người dùng (UI). React.js sử dụng phương pháp tái sử dụng component, giúp dễ dàng tạo ra các phần tử UI động và có thể thay đổi mà không làm ảnh hưởng đến toàn bộ trang web. Điều này giúp tăng hiệu suất của ứng dụng, đặc biệt trong các ứng dụng có lượng dữ liệu lớn và nhiều thay đổi trong thời gian thực. React giúp các lập trình viên có thể phát triển các ứng dụng frontend một cách hiệu quả, dễ dàng duy trì và mở rộng.

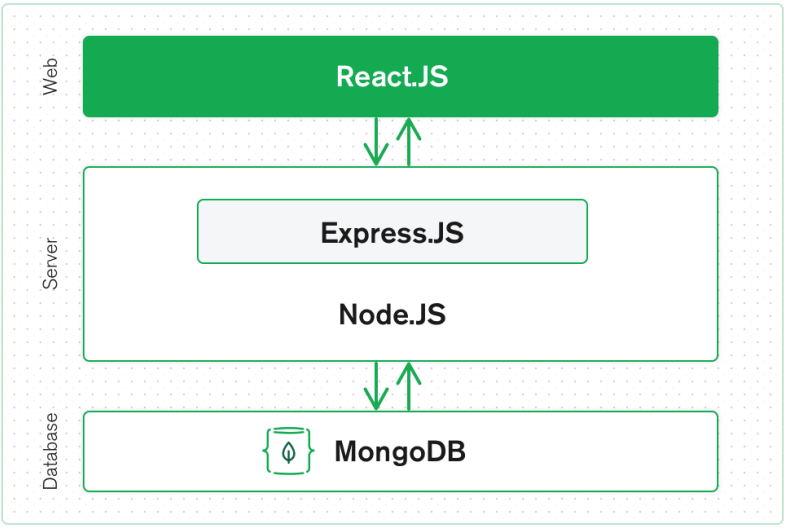
Cuối cùng, **Node.js** là một nền tảng chạy JavaScript trên server. Node.js cho phép xử lý các kết nối đồng thời mà không bị tắc nghẽn, làm cho nó trở thành sự lựa chọn lý tưởng cho các ứng dụng web có lưu lượng truy cập cao hoặc yêu cầu phản hồi nhanh chóng. Node.js sử dụng mô hình non-blocking I/O, giúp tối ưu hóa khả năng xử lý của server và tạo ra các ứng dụng web có tốc độ phản hồi nhanh. Với Node.js, lập trình viên có thể xây dựng các ứng dụng backend mạnh mẽ, dễ dàng kết nối với các cơ sở dữ liệu như MongoDB và triển khai các API hiệu quả.

MERN Stack, với việc sử dụng JavaScript trên cả frontend và backend, giúp giảm thiểu sự phức tạp trong quá trình phát triển phần mềm. Điều này không chỉ giúp tiết kiệm thời gian mà còn tăng cường tính nhất quán trong mã nguồn, đồng thời dễ dàng bảo trì và mở rộng trong tương lai. MERN Stack hỗ trợ xây dựng các ứng dụng web động và phức tạp như các trang mạng xã hội, nền tảng thương mại điện tử, ứng dụng quản lý và phân tích dữ liệu.

MERN Stack [1] mang lại lợi ích lớn khi sử dụng cùng với các công nghệ frontend hiện đại như React, cho phép các lập trình viên xây dựng ứng dụng web động với khả năng tái sử dụng code và tiết kiệm thời gian phát triển.

React [2] có tầm quan trọng trong phát triển ứng dụng web, đặc biệt khi tích hợp với Node.js và MongoDB trong một hệ thống đầy đủ, mang lại khả năng mở rộng, bảo mật và dễ dàng quản lý mã nguồn.

Trong tổng thể, MERN Stack không chỉ là một bộ công nghệ mạnh mẽ, mà còn là một công cụ lý tưởng cho những ai muốn xây dựng các ứng dụng web hiện đại, dễ dàng bảo trì và phát triển trong tương lai.



Hình 1: Các thành phần của MERN stack

2.2. NodeJS

Node.js [3] là một nền tảng chạy mã JavaScript phía server, được xây dựng dựa trên công cụ Chrome V8 JavaScript Engine, vốn được sử dụng để thực thi mã JavaScript trong trình duyệt Chrome. Với Node.js, JavaScript không còn chỉ là ngôn ngữ chạy trên trình duyệt mà đã có thể được sử dụng để phát triển ứng dụng server-side (phía server), mang lại nhiều cơ hội mới trong việc xây dựng các ứng dụng web hiện đại. Node.js giúp các lập trình viên phát triển các ứng dụng mạng, từ các dịch vụ RESTful APIs đến các ứng dụng thời gian thực (real-time applications) như chat, trò chơi trực tuyến, hoặc các hệ thống giám sát trực tiếp.

Một trong những đặc điểm nổi bật của Node.js là khả năng sử dụng JavaScript trên cả frontend và backend. Điều này giúp giảm thiểu sự phân mảnh trong việc phát triển ứng dụng và tạo ra một môi trường đồng nhất, nơi các lập trình viên có thể tập trung vào một ngôn ngữ duy nhất cho cả hai mặt của ứng dụng. Nhờ vào việc sử dụng JavaScript trên server-side, Node.js làm tăng hiệu suất phát triển và dễ dàng trong việc quản lý mã nguồn.

Node.js sử dụng mô hình lập trình bất đồng bộ (asynchronous) và kiến trúc hướng sự kiện (event-driven), điều này giúp nó trở thành một nền tảng lý tưởng cho các ứng dụng cần xử lý đồng thời nhiều kết nối và yêu cầu. Trong mô hình bất đồng bộ, thay vì phải chờ đợi một tác vụ hoàn thành (như trong các mô hình đồng bộ truyền thống), Node.js sẽ tiếp tục xử lý các yêu cầu khác trong khi đợi kết quả từ tác vụ hiện tại. Điều này giúp cải thiện hiệu suất và giảm thiểu độ trễ trong việc xử lý các yêu cầu của người dùng. Kiến trúc hướng sự kiện giúp Node.js xử lý các tác vụ đồng thời mà không làm giảm hiệu suất của hệ thống. Cơ chế này sử dụng một "event loop" để xử lý các sự kiện (events) và callback, giúp ứng dụng có thể phục vụ hàng nghìn kết nối đồng thời mà không bị gián đoạn.

Sự kết hợp của mô hình bất đồng bộ và hướng sự kiện trong Node.js giúp nó xử lý các tác vụ I/O (input/output) như đọc ghi tệp, truy vấn cơ sở dữ liệu, hoặc gọi API từ các hệ thống bên ngoài một cách rất hiệu quả. Node.js không cần phải tạo ra nhiều tiến trình (processes) hoặc thread (chuỗi xử lý) như các hệ thống truyền thống, từ đó tiết kiệm tài nguyên hệ thống và tăng hiệu suất.

Node.js [3] không chỉ giúp giảm thiểu độ trễ khi xử lý nhiều kết nối đồng thời mà còn tạo ra các ứng dụng có khả năng mở rộng dễ dàng. Việc ứng dụng mô hình bất đồng bộ giúp các lập trình viên xây dựng các hệ thống có thể phục vụ hàng nghìn, thậm chí hàng triệu kết nối mà không bị quá tải hoặc làm giảm hiệu suất. Việc xử lý các kết nối theo cách này không chỉ làm giảm chi phí tài nguyên mà còn giúp hệ thống phản hồi nhanh chóng hơn, đặc biệt là trong các môi trường web nơi độ trễ thấp là yếu tố quan trọng.

Một trong những ưu điểm khác của Node.js là việc hỗ trợ rất mạnh mẽ cho việc xây dựng các ứng dụng thời gian thực (real-time applications). Nhờ vào kiến trúc event-driven và mô hình bất đồng bộ, Node.js đặc biệt phù hợp với các ứng dụng yêu cầu phản hồi tức thì từ người dùng, chẳng hạn như các ứng dụng chat, live-streaming, hay các dịch vụ theo dõi trực tiếp. Node.js sử dụng các thư viện như **Socket.io** để hỗ trợ giao tiếp thời gian thực giữa server và client, giúp các lập trình viên dễ dàng xây dựng các ứng dụng tương tác mà không gặp phải các vấn đề về độ trễ hay tải.

Một ví dụ điển hình về ứng dụng Node.js là các dịch vụ của **Netflix**, nơi họ sử dụng Node.js để tối ưu hóa hiệu suất và giảm thiểu thời gian tải dữ liệu từ server. Trong bài viết **"Node.js at Netflix"** trên blog chính thức của Netflix, họ đã chỉ ra rằng việc sử dụng Node.js giúp hệ thống của họ xử lý hàng triệu yêu cầu mỗi ngày mà vẫn duy trì được hiệu suất cao, đặc biệt là trong các tình huống có tải cao, nhờ vào tính bất đồng bộ và khả năng xử lý các yêu cầu đồng thời của Node.js.

Bên cạnh đó, Node.js [4] đã trở thành công cụ phát triển ưu tiên cho các ứng dụng web hiện đại nhờ vào khả năng xử lý nhanh chóng và hiệu quả trong việc quản lý các kết nối mạng. Cùng với sự hỗ trợ mạnh mẽ từ cộng đồng, Node.js đã trở thành nền tảng không thể thiếu trong việc xây dựng các ứng dụng phức tạp và có khả năng mở rộng cao.

Tóm lại, Node.js là một nền tảng rất mạnh mẽ cho việc xây dựng các ứng dụng web hiệu suất cao, đặc biệt trong các ứng dụng cần xử lý đồng thời nhiều kết nối. Với mô hình lập trình bất đồng bộ và kiến trúc hướng sự kiện, Node.js giúp tối ưu hóa việc xử lý các tác vụ I/O, tiết kiệm tài nguyên và tăng tốc độ phản hồi của ứng dụng. Node.js là sự lựa chọn lý tưởng cho các ứng dụng thời gian thực, có khả năng mở rộng và hoạt động với tải trọng lớn, đồng thời hỗ trợ phát triển nhanh chóng và bảo trì dễ dàng.

2.3. ExpressJS

ExpressJS [5] là một framework ứng dụng web nhẹ, linh hoạt và phổ biến cho Node.js, được thiết kế để giúp xây dựng các ứng dụng web và API nhanh chóng và dễ dàng. Được phát triển bởi TJ Holowaychuk vào năm 2010, ExpressJS đã trở thành một trong những framework được ưa chuộng nhất trong cộng đồng lập trình Node.js nhờ vào tính đơn giản, mạnh mẽ và khả năng mở rộng linh hoạt. Với Express, các lập trình viên có thể xây dựng các ứng dụng web, dịch vụ RESTful APIs và các ứng dụng thời gian thực với ít mã nguồn hơn, tiết kiệm thời gian phát triển mà vẫn giữ được tính linh hoạt và khả năng mở rộng cao.

ExpressJS cung cấp nhiều tính năng mạnh mẽ và tiện ích để xây dựng ứng dụng web và API, từ xử lý yêu cầu HTTP, định tuyến (routing), middleware, cho đến hỗ trợ các tính năng bảo mật cơ bản và quản lý lỗi. Những tính năng này giúp lập trình viên dễ dàng xây dựng và duy trì các ứng dụng mà không cần phải viết quá nhiều mã nguồn.

- **Xử lý yêu cầu HTTP**: Một trong những tính năng cơ bản nhất của Express là khả năng xử lý các yêu cầu HTTP (GET, POST, PUT, DELETE, v.v.) từ client. Express cho phép lập trình viên định nghĩa các tuyến đường (routes) cho từng loại yêu cầu, giúp việc xử lý dữ liệu từ client trở nên dễ dàng và hiệu quả hơn. Việc định nghĩa các handler cho các loại yêu cầu HTTP giúp Express trở thành công cụ lý tưởng cho việc xây dựng các API RESTful, nơi các phương thức HTTP tương ứng với các thao tác CRUD (Create, Read, Update, Delete).
- **Định tuyến (Routing)**: Express cung cấp một hệ thống định tuyến rất linh hoạt. Lập trình viên có thể định nghĩa các tuyến đường (routes) cho từng URL cụ thể và gán chúng với các phương thức HTTP tương ứng. Điều này giúp việc xử lý yêu cầu từ client trở nên rõ ràng và dễ dàng hơn. Ngoài ra, Express hỗ trợ định tuyến động, cho phép xử lý các tham số trong URL (route parameters), tạo ra các tuyến đường linh hoạt phù hợp với nhu cầu thực tế của ứng dụng.
- **Middleware**: Middleware là một trong những đặc điểm quan trọng của Express. Middleware trong Express là các hàm có thể được chèn vào chuỗi xử lý yêu cầu và được thực thi theo thứ tự từ trên xuống. Middleware giúp xử lý các yêu cầu

như xác thực người dùng, kiểm tra quyền truy cập, xử lý dữ liệu (parse body request), log các hoạt động hoặc quản lý lỗi. Cơ chế này giúp việc phát triển ứng dụng trở nên dễ dàng hơn khi các chức năng có thể được tái sử dụng và sắp xếp một cách rõ ràng.

- **Tiện ích khác**: Express còn cung cấp một số tiện ích hỗ trợ việc phát triển ứng dụng như khả năng dễ dàng sử dụng template engine (ví dụ: EJS, Pug) để render các view động, hỗ trợ việc gửi các phản hồi HTTP dễ dàng, và khả năng mở rộng với các module và thư viện bên ngoài.

Một trong những ưu điểm lớn nhất của ExpressJS là sự đơn giản và tính linh hoạt. Express cung cấp một cấu trúc rất dễ hiểu và dễ sử dụng cho các lập trình viên, giúp giảm thiểu việc phải viết mã nguồn phức tạp và tiết kiệm thời gian phát triển. Hệ thống middleware của Express giúp dễ dàng thêm vào các chức năng như xác thực, log, và quản lý lỗi mà không làm rối cấu trúc ứng dụng. Điều này giúp Express trở thành lựa chọn lý tưởng cho việc phát triển các ứng dụng nhỏ, vừa và các API đơn giản.

Ngoài ra, ExpressJS còn hỗ trợ rất tốt việc xây dựng các ứng dụng RESTful API. Việc xử lý các yêu cầu HTTP, định nghĩa các tuyến đường và gán chúng với các phương thức HTTP đã giúp Express trở thành một framework tuyệt vời cho việc xây dựng các dịch vụ API. Điều này đã làm cho Express trở thành một framework cực kỳ phổ biến trong việc phát triển các ứng dụng web động và các dịch vụ API trong môi trường Node.js.

Một yếu tố quan trọng khiến Express trở nên mạnh mẽ là khả năng mở rộng của nó. Express cho phép lập trình viên có thể dễ dàng tích hợp các thư viện bên ngoài và các module khác, từ đó xây dựng các ứng dụng có tính năng phức tạp. Với một cộng đồng phát triển lớn mạnh và các tài liệu phong phú, lập trình viên dễ dàng tìm kiếm sự hỗ trợ và các giải pháp cho các vấn đề gặp phải trong quá trình phát triển.

ExpressJS còn có một số tiện ích khác như tích hợp với các cơ sở dữ liệu NoSQL (MongoDB) và SQL (MySQL, PostgreSQL) giúp việc phát triển ứng dụng trở nên dễ dàng hơn. Các thư viện như **Mongoose** (để tương tác với MongoDB) hay **Sequelize** (ORM cho SQL databases) thường xuyên được kết hợp với Express để tạo ra các ứng dụng web có khả năng mở rộng tốt, xử lý dữ liệu hiệu quả.

ExpressJS đã được sử dụng rộng rãi trong việc xây dựng các ứng dụng web và dịch vụ API cho nhiều công ty và dự án. Ví dụ, các công ty như **Uber**, **IBM**, và **LinkedIn** đều sử dụng Express trong việc xây dựng các ứng dụng và dịch vụ của mình. **Uber** sử dụng Express để xây dựng các API phục vụ các dịch vụ của mình, giúp xử lý các yêu cầu và cung cấp dịch vụ thông qua các kết nối HTTP nhanh chóng và hiệu quả. Trong khi đó, **IBM** và **LinkedIn** sử dụng Express để xây dựng các ứng dụng web doanh nghiệp phức tạp, tận dụng các tính năng linh hoạt và hiệu suất cao của framework này.

ExpressJS [6] là một trong những lựa chọn phổ biến khi phát triển các API cho các ứng dụng hiện đại, nhờ vào tính linh hoạt và dễ sử dụng. Bằng cách sử dụng Express, lập

trình viên có thể nhanh chóng xây dựng các API RESTful có hiệu suất cao, dễ dàng quản lý và mở rộng.

Express [5] có thể hỗ trợ các ứng dụng web, từ các dịch vụ nhỏ đến các ứng dụng lớn, đồng thời cũng chia sẻ các chiến lược phát triển và tối ưu hóa ứng dụng Express cho các dự án quy mô lớn.

ExpressJS là một framework mạnh mẽ và linh hoạt cho việc phát triển ứng dụng web và API trong môi trường Node.js. Với tính năng định tuyến linh hoạt, hỗ trợ middleware mạnh mẽ và sự đơn giản trong cấu trúc, Express đã giúp các lập trình viên xây dựng các ứng dụng web hiệu quả, dễ duy trì và mở rộng. Việc sử dụng ExpressJS kết hợp với Node.js tạo ra một môi trường phát triển ứng dụng web đồng nhất và mạnh mẽ, phù hợp với các yêu cầu ngày càng cao của các ứng dụng web hiện đại.

2.4. MongoDB

MongoDB [7] là một hệ quản trị cơ sở dữ liệu không quan hệ (NoSQL) được thiết kế để lưu trữ và truy vấn dữ liệu dưới dạng tài liệu (document) với cấu trúc JSON (JavaScript Object Notation). Khác với các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) truyền thống, MongoDB không yêu cầu dữ liệu phải được lưu trữ trong các bảng với các mối quan hệ cố định giữa các bảng. Thay vào đó, MongoDB lưu trữ dữ liệu dưới dạng các tài liệu JSON hoặc BSON (Binary JSON), cho phép dễ dàng mở rộng và xử lý dữ liệu không cấu trúc hoặc bán cấu trúc. Đây là một trong những lý do MongoDB trở thành lựa chọn phổ biến cho các ứng dụng cần khả năng mở rộng linh hoạt và hiệu suất cao, đặc biệt là trong các hệ thống web hiện đại và ứng dụng đám mây.

MongoDB được phát triển bởi 10gen (nay là MongoDB, Inc.) và lần đầu tiên ra mắt vào năm 2009. Kể từ đó, MongoDB đã trở thành một trong những cơ sở dữ liệu NoSQL phổ biến nhất nhờ vào tính linh hoạt trong lưu trữ dữ liệu, khả năng mở rộng dễ dàng và hiệu suất vượt trội trong việc xử lý lượng dữ liệu lớn và phức tạp. Các ứng dụng web, dịch vụ đám mây, các hệ thống thời gian thực, và các hệ thống phân tán là những lĩnh vực mà MongoDB có thể phát huy tối đa sức mạnh của mình.

MongoDB lưu trữ dữ liệu dưới dạng tài liệu (document) với định dạng JSON hoặc BSON. Một tài liệu trong MongoDB có thể chứa các cặp khóa-giá trị (key-value pairs), tương tự như các đối tượng trong JavaScript, và có thể bao gồm các mảng (arrays), các đối tượng lồng nhau (nested objects), hoặc thậm chí các tài liệu con. Điều này cho phép MongoDB lưu trữ dữ liệu với cấu trúc linh hoạt và phù hợp với các ứng dụng cần lưu trữ dữ liệu không có cấu trúc hoặc thay đổi thường xuyên.

Cấu trúc cơ bản của MongoDB bao gồm ba thành phần chính:

1. **Database**: Một cơ sở dữ liệu trong MongoDB là tập hợp các bộ sưu tập (collections) và mỗi cơ sở dữ liệu có thể chứa nhiều bộ sưu tập.

2. **Collection**: Bộ sưu tập là một nhóm các tài liệu. Mỗi bộ sưu tập có thể chứa nhiều tài liệu với cấu trúc khác nhau, điều này giúp MongoDB dễ dàng lưu trữ dữ liệu có tính linh hoạt cao.
3. **Document**: Tài liệu là đơn vị dữ liệu cơ bản trong MongoDB. Mỗi tài liệu là một đối tượng JSON hoặc BSON với các cặp khóa-giá trị.

Khác với các hệ quản trị cơ sở dữ liệu quan hệ, MongoDB không yêu cầu phải xác định trước cấu trúc bảng hay các mối quan hệ giữa các bảng. Điều này làm cho MongoDB rất linh hoạt và dễ dàng thay đổi khi ứng dụng phát triển, điều này đặc biệt hữu ích trong các dự án phát triển phần mềm theo phương pháp Agile hoặc khi yêu cầu thay đổi nhanh chóng.

Một trong những ưu điểm nổi bật của MongoDB là khả năng mở rộng ngang (horizontal scaling), giúp nó có thể dễ dàng xử lý khối lượng dữ liệu ngày càng lớn mà không làm giảm hiệu suất. Điều này có được nhờ vào việc hỗ trợ sharding trong MongoDB, tức là dữ liệu có thể được chia nhỏ và phân tán trên nhiều máy chủ, giúp hệ thống có thể phục vụ hàng triệu người dùng và truy vấn một cách nhanh chóng. Các ứng dụng cần xử lý một lượng lớn dữ liệu, như các mạng xã hội, các dịch vụ trực tuyến, hoặc các ứng dụng thương mại điện tử, thường lựa chọn MongoDB để tận dụng khả năng mở rộng này.

Ngoài khả năng mở rộng, MongoDB còn nổi bật với tính linh hoạt trong việc lưu trữ và truy vấn dữ liệu. Khi làm việc với các dữ liệu không cấu trúc hoặc bán cấu trúc, MongoDB cho phép các lập trình viên có thể thay đổi cấu trúc dữ liệu dễ dàng mà không cần phải thực hiện các thay đổi phức tạp đối với cơ sở dữ liệu. Điều này làm cho MongoDB trở thành lựa chọn lý tưởng cho các ứng dụng web, nơi dữ liệu có thể thay đổi thường xuyên và yêu cầu tính linh hoạt cao.

Thêm vào đó, MongoDB cũng hỗ trợ việc thực hiện các truy vấn phức tạp và chỉ mục (indexing) hiệu quả. Dù không phải là cơ sở dữ liệu quan hệ, MongoDB cung cấp nhiều phương pháp truy vấn mạnh mẽ như tìm kiếm theo trường, lọc dữ liệu, và thậm chí tìm kiếm văn bản toàn văn, giúp người dùng dễ dàng lấy dữ liệu nhanh chóng và chính xác.

MongoDB rất phổ biến trong các ứng dụng cần khả năng mở rộng cao và tính linh hoạt trong việc lưu trữ và xử lý dữ liệu. Các công ty lớn như **eBay**, **ShutterStock**, **The New York Times**, và **Adobe** đều sử dụng MongoDB để lưu trữ và xử lý dữ liệu của họ. Ví dụ, **eBay** đã sử dụng MongoDB để xây dựng các ứng dụng theo thời gian thực, nơi dữ liệu thay đổi liên tục và có tính chất phân tán, đồng thời đòi hỏi khả năng mở rộng linh hoạt để đáp ứng yêu cầu khối lượng người dùng lớn.

MongoDB [8] là lựa chọn lý tưởng cho các ứng dụng cần khả năng mở rộng linh hoạt và xử lý dữ liệu không cấu trúc. Bằng cách sử dụng các tính năng như sharding, replica sets, và hỗ trợ các chỉ mục, MongoDB có thể đáp ứng yêu cầu của các ứng dụng phân tán và quy mô lớn mà các cơ sở dữ liệu quan hệ không thể làm được.

Ngoài ra, MongoDB [7] có thể xây dựng các hệ thống quản lý nội dung, các ứng dụng mạng xã hội, và các dịch vụ đám mây, nơi dữ liệu có thể thay đổi nhanh chóng và đòi hỏi khả năng xử lý đồng thời nhiều yêu cầu.

MongoDB không chỉ mạnh mẽ về khả năng lưu trữ dữ liệu mà còn cung cấp nhiều công cụ hỗ trợ mạnh mẽ cho việc quản lý và truy vấn dữ liệu. Một trong những công cụ phổ biến là **MongoDB Atlas**, là dịch vụ cơ sở dữ liệu MongoDB được quản lý hoàn toàn trên đám mây, giúp người dùng dễ dàng triển khai và quản lý cơ sở dữ liệu MongoDB mà không cần phải tự xây dựng hạ tầng.

Ngoài ra, MongoDB còn cung cấp **MongoDB Compass**, một công cụ GUI (giao diện người dùng đồ họa) cho phép người dùng quản lý cơ sở dữ liệu MongoDB, trực quan hóa các dữ liệu, và tối ưu hóa các truy vấn. Các công cụ này giúp giảm thiểu các công việc quản lý cơ sở dữ liệu phức tạp và giúp lập trình viên dễ dàng làm việc với MongoDB.

MongoDB là một cơ sở dữ liệu NoSQL mạnh mẽ, linh hoạt và có khả năng mở rộng tuyệt vời. Với khả năng lưu trữ dữ liệu dạng tài liệu JSON/BSON và hỗ trợ các tính năng như sharding, replica sets, và chỉ mục, MongoDB trở thành lựa chọn phổ biến cho các ứng dụng web, hệ thống phân tán, và các dịch vụ đám mây. Tính linh hoạt trong lưu trữ và truy vấn dữ liệu giúp MongoDB phù hợp với các dự án có yêu cầu thay đổi dữ liệu thường xuyên và không có cấu trúc cố định.

2.5. ReactJS

ReactJS [9] là một thư viện JavaScript mã nguồn mở được phát triển và duy trì bởi Facebook, giúp xây dựng giao diện người dùng (UI) cho các ứng dụng web, đặc biệt là các ứng dụng đơn trang (Single Page Applications - SPA). Được ra mắt lần đầu tiên vào năm 2013, React đã nhanh chóng trở thành một trong những thư viện phổ biến nhất trong cộng đồng phát triển web, nhờ vào tính năng độc đáo là “component-based architecture” (kiến trúc dựa trên thành phần), khả năng tái sử dụng mã nguồn và hiệu suất cao trong việc cập nhật giao diện người dùng.

React cho phép các lập trình viên xây dựng giao diện người dùng bằng cách chia nhỏ giao diện thành các thành phần (components) độc lập. Mỗi thành phần có thể quản lý trạng thái (state) của riêng mình và có thể tái sử dụng trong các phần khác của ứng dụng. Điều này giúp giảm thiểu việc viết lại mã, tăng tính bảo trì của ứng dụng và cải thiện hiệu suất tổng thể. React không chỉ phù hợp với các ứng dụng web, mà còn có thể được sử dụng để phát triển ứng dụng di động với React Native, giúp tạo ra các ứng dụng di động cho cả iOS và Android.

Một trong những đặc điểm nổi bật của React là kiến trúc component-based. Trong React, một giao diện người dùng được chia thành nhiều thành phần (components) độc lập và tái sử dụng được. Mỗi thành phần có thể bao gồm cả logic và giao diện, giúp lập trình viên quản lý mã nguồn dễ dàng và hiệu quả. Thành phần trong React có thể là một phần

giao diện người dùng nhỏ như một nút bấm (button) hay một trường nhập liệu (input field), hoặc một phần lớn hơn như một toàn bộ trang (page).

Mỗi component trong React có thể có trạng thái riêng (state) và có thể tương tác với các thành phần khác thông qua props (properties). Props là các giá trị được truyền từ component cha xuống các component con, trong khi state quản lý dữ liệu nội bộ của component. Cách thức quản lý dữ liệu này giúp tách biệt rõ ràng logic và giao diện, đồng thời giữ cho ứng dụng dễ bảo trì và mở rộng.

Một trong những lợi ích lớn nhất của việc xây dựng ứng dụng theo kiểu component-based là khả năng tái sử dụng mã [9]. Mỗi component có thể được phát triển, kiểm thử và tái sử dụng độc lập mà không ảnh hưởng đến phần còn lại của ứng dụng. Điều này không chỉ giúp tiết kiệm thời gian phát triển mà còn đảm bảo rằng ứng dụng luôn dễ dàng duy trì và mở rộng.

React sử dụng một kỹ thuật rất hiệu quả để quản lý việc cập nhật giao diện người dùng, đó là **Virtual DOM** (DOM ảo). Thông thường, khi có sự thay đổi trong dữ liệu, các thư viện JavaScript khác sẽ cập nhật trực tiếp DOM của trình duyệt, điều này có thể gây ra hiệu suất kém khi có nhiều thay đổi hoặc khi làm việc với các ứng dụng phức tạp. React giải quyết vấn đề này bằng cách sử dụng Virtual DOM.

Khi có sự thay đổi trong state của một component, React sẽ tạo ra một phiên bản Virtual DOM mới để phản ánh sự thay đổi đó. Sau đó, React so sánh Virtual DOM mới với Virtual DOM cũ để xác định những phần nào của giao diện thực sự cần phải được cập nhật. Chỉ những phần thay đổi mới được render lại vào DOM thực tế, điều này giúp giảm thiểu số lần phải tương tác với DOM thực tế và cải thiện hiệu suất tổng thể.

Virtual DOM [10] không chỉ giúp cải thiện hiệu suất mà còn làm cho React trở thành một thư viện dễ sử dụng. Các thao tác DOM phức tạp, vốn có thể gây khó khăn cho các lập trình viên, giờ đây được React trừu tượng hóa và xử lý một cách tự động và hiệu quả.

Mặc dù React cung cấp cơ chế quản lý trạng thái cho từng component thông qua state, nhưng đối với các ứng dụng lớn, khi trạng thái cần được chia sẻ giữa nhiều component, việc quản lý trạng thái trở nên khó khăn. Để giải quyết vấn đề này, React thường được sử dụng kết hợp với **Redux**, một thư viện quản lý trạng thái cho JavaScript.

Redux giúp quản lý trạng thái toàn cục của ứng dụng trong một store duy nhất. Thay vì mỗi component tự quản lý trạng thái của mình, Redux cho phép lưu trữ tất cả dữ liệu trong một đối tượng duy nhất, giúp các component dễ dàng truy cập và cập nhật dữ liệu từ bất kỳ đâu trong ứng dụng. Redux sử dụng mô hình hành động (actions) và bộ giảm (reducers) để quản lý các thay đổi trong trạng thái.

Redux [9] là công cụ rất mạnh mẽ khi làm việc với các ứng dụng React phức tạp, nơi trạng thái của ứng dụng cần được đồng bộ hóa giữa nhiều phần của ứng dụng. Redux

giúp việc quản lý trạng thái trở nên đơn giản và dễ dàng kiểm soát hơn, đặc biệt trong các ứng dụng có nhiều màn hình hoặc giao diện người dùng phức tạp.

Một trong những lý do khiến React trở nên phổ biến là khả năng tích hợp và phối hợp với nhiều công nghệ khác. React có thể được sử dụng kết hợp với các thư viện như **React Router** để quản lý điều hướng trong ứng dụng đơn trang (SPA), hoặc với các thư viện UI như **Material-UI** để tạo ra các giao diện đẹp và dễ sử dụng. React cũng có thể được tích hợp với các công nghệ phía server như Node.js, cho phép phát triển các ứng dụng full-stack hiệu quả.

Ngoài ra, React Native, một dự án con của React, cho phép lập trình viên sử dụng React để phát triển ứng dụng di động cho cả iOS và Android. Điều này giúp các lập trình viên có thể tận dụng kỹ năng JavaScript và React của mình để phát triển ứng dụng di động mà không cần học các ngôn ngữ lập trình riêng biệt cho từng nền tảng.

React mang lại rất nhiều lợi ích cho các lập trình viên, bao gồm khả năng tái sử dụng mã, khả năng quản lý giao diện phức tạp một cách dễ dàng, và hiệu suất cao trong việc cập nhật giao diện người dùng. Với React, việc phát triển các ứng dụng đơn trang (SPA) trở nên dễ dàng hơn bao giờ hết, và các lập trình viên có thể tập trung vào việc xây dựng các tính năng ứng dụng mà không phải lo lắng về các vấn đề hiệu suất liên quan đến DOM.

React [11] còn giúp việc phát triển các ứng dụng dễ dàng mở rộng và bảo trì, nhờ vào các mô hình thiết kế và best practices được áp dụng trong các thành phần. Điều này đặc biệt quan trọng khi ứng dụng của bạn ngày càng phát triển và cần được duy trì qua nhiều năm.

ReactJS đã trở thành một trong những thư viện phát triển giao diện người dùng phổ biến nhất nhờ vào sự linh hoạt, hiệu suất và khả năng tái sử dụng mã của nó. Việc áp dụng kiến trúc component-based, sử dụng Virtual DOM để tối ưu hóa hiệu suất và khả năng kết hợp với các thư viện quản lý trạng thái như Redux, đã giúp React trở thành lựa chọn hàng đầu cho việc phát triển các ứng dụng web và di động hiện đại.

2.6. Redux

Redux [12] là một thư viện quản lý trạng thái (state management) phổ biến trong các ứng dụng JavaScript, đặc biệt là các ứng dụng được phát triển bằng React. Được phát triển bởi Dan Abramov và Andrew Clark vào năm 2015, Redux cung cấp một phương pháp có cấu trúc và mạnh mẽ để quản lý trạng thái toàn cục của ứng dụng. Thay vì cho phép mỗi component trong ứng dụng tự quản lý trạng thái của mình, Redux centralizes (tập trung) tất cả trạng thái vào trong một đối tượng duy nhất gọi là "store". Điều này giúp các developer dễ dàng quản lý dữ liệu trong ứng dụng, làm cho việc truy xuất và

thay đổi dữ liệu trở nên hiệu quả và dễ kiểm soát hơn, đặc biệt đối với các ứng dụng có quy mô lớn với nhiều thành phần phụ thuộc vào dữ liệu chung.

Trong Redux, việc quản lý trạng thái dựa trên ba khái niệm chính: **Actions**, **Store**, và **Reducers**. Cả ba khái niệm này làm việc cùng nhau để đảm bảo rằng các thay đổi trạng thái trong ứng dụng luôn được theo dõi một cách nhất quán và có thể dễ dàng kiểm soát. Theo cuốn sách **"Learning Redux"** của Ben M. (2019), một trong những điểm mạnh lớn nhất của Redux là việc tách biệt rõ ràng giữa các thành phần chịu trách nhiệm thay đổi trạng thái và các thành phần chỉ hiển thị trạng thái, giúp cho mã nguồn trở nên dễ bảo trì và mở rộng hơn.

Actions [13] trong Redux là các đối tượng JavaScript đơn giản mô tả một hành động hoặc sự kiện xảy ra trong ứng dụng. Những hành động này được gửi từ các thành phần trong ứng dụng để thay đổi trạng thái của store. Mỗi action ít nhất phải có một thuộc tính bắt buộc là `type`, giúp xác định loại hành động. Một action có thể có thêm các dữ liệu đi kèm (payload), tùy thuộc vào yêu cầu của ứng dụng. Dữ liệu này có thể là thông tin từ người dùng, kết quả từ một API, hoặc các thông tin cần thiết khác.

Trong dự án của nhóm, có bốn action chính là `cartActions`, `orderActions`, `productActions`, và `userActions`. Các action này đảm bảo rằng khi người dùng thực hiện một thao tác nào đó, chẳng hạn như thêm sản phẩm vào giỏ hàng, đặt hàng, hay đăng nhập, trạng thái của ứng dụng sẽ được cập nhật kịp thời và đồng bộ.

Actions là cách thức chính để truyền tải dữ liệu vào Redux store. Việc thiết kế các action hợp lý và rõ ràng giúp quá trình thay đổi trạng thái trong ứng dụng trở nên trực quan và dễ kiểm soát. Các action trong Redux giúp tách biệt logic xử lý trạng thái khỏi giao diện người dùng, từ đó giữ cho mã nguồn của ứng dụng trở nên rõ ràng và dễ bảo trì hơn.

Store là nơi lưu trữ toàn bộ trạng thái của ứng dụng. Mỗi ứng dụng Redux chỉ có một store duy nhất, và store này sẽ lưu trữ toàn bộ trạng thái của ứng dụng ở dạng đối tượng JavaScript. Store không chỉ giữ trạng thái, mà còn cung cấp các phương thức để truy xuất trạng thái hiện tại và đăng ký các thay đổi trạng thái khi có action được gửi đến.

Store [9] trong Redux đóng vai trò trung tâm trong việc quản lý dữ liệu của ứng dụng. Store giúp các thành phần trong ứng dụng có thể truy cập trực tiếp vào bất kỳ trạng thái nào mà không cần phải truyền dữ liệu giữa các thành phần con qua các props. Điều này giúp giảm bớt sự phụ thuộc giữa các thành phần, khiến ứng dụng dễ dàng mở rộng và bảo trì hơn.

Khi một action được gửi đến store, store sẽ không thay đổi trạng thái của mình ngay lập tức. Thay vào đó, store sẽ gửi action đó đến reducers để xử lý.

Reducers là các hàm thuần túy (pure functions) trong Redux, có vai trò quan trọng trong việc cập nhật trạng thái của ứng dụng. Một reducer nhận vào hai tham số: trạng thái hiện tại và một action. Dựa trên action được nhận, reducer sẽ trả về một trạng thái mới mà không làm thay đổi trạng thái cũ (tính bất biến của trạng thái). Điều này là nguyên tắc

quan trọng trong Redux, giúp đảm bảo rằng các thay đổi trong trạng thái luôn có thể được theo dõi và kiểm tra.

Mỗi reducer trong Redux sẽ xử lý một phần trạng thái của ứng dụng và chỉ thay đổi phần này dựa trên action đã nhận được. Trong dự án của nhóm, có bốn reducer chính là `cartReducers`, `orderReducers`, `productReducers`, và `userReducers`. Mỗi reducer này sẽ xử lý các thay đổi liên quan đến giỏ hàng, đơn hàng, sản phẩm và người dùng tương ứng.

Một trong những đặc điểm nổi bật của Redux là cách nó xử lý các thay đổi trạng thái thông qua các reducers [14]. Việc sử dụng hàm thuần túy giúp cho Redux có thể dễ dàng kiểm thử (unit testing) và giúp các developer hiểu rõ cách thức trạng thái của ứng dụng thay đổi qua thời gian.

Redux giúp giải quyết các vấn đề mà các ứng dụng JavaScript lớn thường gặp phải khi quản lý trạng thái, đặc biệt là trong các ứng dụng với nhiều thành phần phụ thuộc vào trạng thái toàn cục. Bằng cách tập trung trạng thái vào một store duy nhất, Redux giúp quản lý dữ liệu trong ứng dụng trở nên dễ dàng hơn và tránh được tình trạng "prop drilling", tức là phải truyền dữ liệu qua nhiều lớp thành phần con.

Ngoài ra, việc sử dụng Redux giúp các hành động và thay đổi trạng thái trở nên dễ kiểm soát và dễ dàng gỡ lỗi (debugging), vì tất cả các thay đổi trạng thái đều diễn ra qua các action và reducer có thể được theo dõi rõ ràng. Các công cụ như Redux DevTools cung cấp một giao diện đồ họa để theo dõi và kiểm tra lịch sử các action và trạng thái, giúp lập trình viên dễ dàng phát hiện và sửa lỗi trong ứng dụng.

Redux [12] không chỉ đơn giản là một công cụ quản lý trạng thái, mà còn là một cách tiếp cận có hệ thống giúp xây dựng các ứng dụng web với một cấu trúc rõ ràng và dễ duy trì. Việc quản lý trạng thái theo cách này giúp ứng dụng trở nên mở rộng được và giảm thiểu sự phức tạp khi ứng dụng ngày càng lớn.

Redux là một công cụ mạnh mẽ và hiệu quả trong việc quản lý trạng thái của các ứng dụng JavaScript. Bằng cách sử dụng store, action và reducer, Redux giúp các developer xây dựng các ứng dụng dễ bảo trì, dễ kiểm soát và dễ mở rộng. Việc áp dụng Redux trong các ứng dụng phức tạp giúp đảm bảo rằng mọi thay đổi trạng thái đều có thể theo dõi được, giúp tăng cường khả năng gỡ lỗi và tối ưu hóa hiệu suất của ứng dụng.

Chương 3: Thu thập yêu cầu và đặc tả yêu cầu

3.1. Thu thập yêu cầu

Website thương mại điện tử cần được xây dựng nhằm đáp ứng nhu cầu mua sắm trực tuyến của khách hàng, cung cấp một nền tảng dễ dàng truy cập, tìm kiếm và mua sắm các sản phẩm một cách nhanh chóng và tiện lợi. Mục tiêu của dự án là tạo ra một website với các tính năng hiện đại, dễ sử dụng và có thể tích hợp các công nghệ mới nhất để tăng cường trải nghiệm người dùng và nâng cao hiệu quả kinh doanh.

Phạm vi dự án bao gồm:

- Xây dựng giao diện người dùng (UI) thân thiện, dễ sử dụng.
- Tích hợp hệ thống quản lý sản phẩm, đơn hàng và khách hàng.
- Phát triển các chức năng tìm kiếm và phân loại sản phẩm.
- Cung cấp các phương thức thanh toán trực tuyến và giao hàng linh hoạt.
- Đảm bảo tính bảo mật và bảo vệ thông tin người dùng.

Hơn nữa, hệ thống website bán đồ dùng online còn cung cấp nhiều chương trình khuyến mãi, ưu đãi, dịch vụ hỗ trợ khách hàng tốt và tích hợp tính năng nhắn tin, chatbot.

3.2. Yêu cầu chức năng, yêu cầu phi chức năng

3.2.1. Yêu cầu chức năng

- Đăng nhập: Chức năng này nhằm mục đích xác thực người dùng khi tương tác với hệ thống nhằm cung cấp quyền cũng như phạm vi truy cập hệ thống.
- Đăng xuất: Thoát khỏi tài khoản đang sử dụng.
- Đăng ký: Để truy cập sử dụng hệ thống thì Người dùng trước hết cần đăng ký
- Hiển thị sản phẩm:

  - Website cần hiển thị danh sách sản phẩm đồ dùng cá nhân với hình ảnh, tên sản phẩm, giá cả và mô tả ngắn gọn.
  - Người dùng có thể sắp xếp và lọc sản phẩm theo giá, nhãn hiệu, danh mục hoặc đánh giá của người dùng.

- Tìm kiếm sản phẩm: Website cung cấp chức năng tìm kiếm để người dùng có thể dễ dàng tìm kiếm sản phẩm theo tên, từ khóa hoặc mô tả.
- Chi tiết sản phẩm: Khi người dùng nhấp vào một sản phẩm, website cần hiển thị trang chi tiết sản phẩm với hình ảnh chi tiết, mô tả đầy đủ, thông tin nhà sản xuất, đánh giá của người dùng và các thông tin liên quan khác.
- Giỏ hàng và thanh toán: Người dùng có thể thêm sản phẩm vào giỏ hàng, xem giỏ hàng, cập nhật số lượng sản phẩm và xóa sản phẩm khỏi giỏ hàng.
- Hỗ trợ quá trình thanh toán an toàn và nhanh chóng thông qua nhiều phương thức thanh toán như thẻ tín dụng, chuyển khoản ngân hàng, ví điện tử, tiền mặt khi giao hàng, v.v.
- Quản lý tài khoản người dùng:

– Người dùng có thể đăng ký, đăng nhập, đăng xuất, đổi mật khẩu và cập nhật thông tin cá nhân.
– Người dùng có thể xem đơn mua, người bán có thể xem đơn bán.

- Đánh giá và nhận xét sản phẩm: Người dùng có thể viết đánh giá và nhận xét về sản phẩm sau khi mua hàng, cũng như xem đánh giá của người dùng khác.
- Hỗ trợ khách hàng: Website cung cấp thông tin liên hệ, hỗ trợ trực tuyến (chat, email) và cơ chế giải quyết khiếu nại để hỗ trợ người dùng khi có vấn đề liên quan đến sản phẩm, đơn hàng hoặc dịch vụ.
- Quản lý nội dung và tin tức: Website cần cung cấp tính năng quản lý nội dung và tin tức liên quan đến sản phẩm, chương trình khuyến mãi, hướng dẫn sử dụng và các thông tin hữu ích khác.
- Mã giảm giá và khuyến mãi: Website cung cấp hiển thị danh sách các mã giảm giá và sử dụng để giảm giá sản phẩm khi thanh toán.
- Quản lý sản phẩm: Admin có thể quản lý toàn bộ sản phẩm, người bán có thể thêm, xóa hay chỉnh sửa sản phẩm của mình.
- Quản lý đơn hàng: Admin có thể đánh dấu là đã chuyển hàng, xem danh sách toàn bộ đơn hàng, người bán có thể xem danh sách toàn bộ đơn bán và người mua có thể xem đơn mua.
- Nhắn tin giữa người mua và người bán:

– Hệ thống cần cung cấp chức năng nhắn tin trực tiếp giữa người mua và người bán trong quá trình trao đổi sản phẩm.
– Người dùng có thể gửi và nhận tin nhắn văn bản, hỗ trợ hiển thị theo dạng hội thoại trực quan, giúp việc trao đổi trở nên nhanh chóng và dễ dàng.
– Tin nhắn được lưu trữ trong cơ sở dữ liệu để đảm bảo tính liên tục của cuộc trò

chuyện, có thể hỗ trợ tính năng thông báo khi có tin nhắn mới.
– Tính năng này giúp tăng cường tính tương tác và tin cậy giữa các bên trong giao dịch, đồng thời hỗ trợ xử lý các vấn đề phát sinh như thông tin sản phẩm, thời gian giao hàng, hoặc yêu cầu đổi trả.

• Chatbot hỗ trợ thông minh:

– Website tích hợp một chatbot sử dụng công nghệ trí tuệ nhân tạo (AI) nhằm cung cấp hỗ trợ tự động 24/7 cho người dùng trong các tình huống phổ biến như tư vấn chọn sản phẩm, hướng dẫn đặt hàng, kiểm tra tình trạng đơn hàng và giải đáp thắc mắc.
– Chatbot có khả năng hiểu ngôn ngữ tự nhiên (Natural Language Processing) và phản hồi linh hoạt, thân thiện với người dùng, tạo cảm giác tương tác như với nhân viên tư vấn thực sự.
– Chatbot có thể hoạt động song song với hệ thống hỗ trợ khách hàng thủ công (qua email, điện thoại) và sẽ chuyển tiếp cho nhân viên khi không thể giải quyết vấn đề.
– Việc ứng dụng chatbot giúp tiết kiệm chi phí vận hành, tăng trải nghiệm người dùng và nâng cao mức độ chuyên nghiệp của nền tảng thương mại điện tử.

3.2.2. Yêu cầu phi chức năng

1. Hiệu suất và tốc độ

- **Thời gian tải trang**: Website cần phải tải nhanh chóng, với thời gian tải trang không quá 3 giây cho bất kỳ trang nào. Điều này sẽ cải thiện trải nghiệm người dùng và cũng có tác dụng tích cực trong việc tối ưu hóa SEO (Search Engine Optimization).
- **Khả năng xử lý lưu lượng truy cập cao**: Website cần phải có khả năng xử lý lưu lượng truy cập đột ngột hoặc lượng người dùng lớn mà không bị giảm hiệu suất. Cần có cơ chế tối ưu hóa và cân bằng tải (load balancing) để phân phối lưu lượng truy cập đều giữa các máy chủ.
- **Quản lý tài nguyên**: Đảm bảo tài nguyên của hệ thống như bộ nhớ, CPU, và dung lượng ổ cứng được sử dụng hợp lý và hiệu quả.
- **Cải tiến hiệu suất**: Hệ thống cần có khả năng cải thiện hiệu suất khi có sự gia tăng lưu lượng người dùng, chẳng hạn như sử dụng các dịch vụ CDN (Content Delivery Network) để giảm thời gian tải nội dung tĩnh.

2. Bảo mật

- **Bảo vệ dữ liệu người dùng**: Website cần đảm bảo việc mã hóa dữ liệu người dùng (đặc biệt là thông tin thanh toán và dữ liệu cá nhân) thông qua các giao thức bảo mật như HTTPS/SSL.

- **Xác thực và phân quyền**: Hệ thống cần có cơ chế xác thực mạnh mẽ như 2FA (Xác thực 2 yếu tố) và phân quyền người dùng (admin, người bán, khách hàng) rõ ràng, đảm bảo chỉ những người có quyền mới có thể truy cập các thông tin nhạy cảm.
- **Chống tấn công và phần mềm độc hại**: Website cần được bảo vệ khỏi các cuộc tấn công phổ biến như SQL injection, XSS (Cross-site scripting), CSRF (Cross-site Request Forgery), và các lỗ hổng bảo mật khác.
- **Quản lý thông tin thanh toán**: Các giao dịch thanh toán trực tuyến phải tuân thủ các tiêu chuẩn bảo mật như PCI-DSS (Payment Card Industry Data Security Standard).

3. Khả năng mở rộng và khả năng duy trì

- **Khả năng mở rộng (Scalability)**: Website cần có khả năng mở rộng linh hoạt để đáp ứng nhu cầu tăng trưởng trong tương lai. Điều này có thể bao gồm mở rộng về mặt phần cứng (server), phần mềm (cơ sở dữ liệu), hoặc kiến trúc microservices để xử lý lượng người dùng và giao dịch lớn hơn.
- **Khả năng duy trì và cập nhật**: Hệ thống cần có khả năng duy trì dễ dàng, cho phép cập nhật phần mềm và cơ sở dữ liệu mà không gây gián đoạn dịch vụ. Các bản cập nhật cần được triển khai tự động hoặc có quy trình kiểm tra và cập nhật một cách an toàn.
- **Quản lý lỗi**: Website phải có cơ chế giám sát và ghi nhận lỗi (logging), thông báo tự động khi có sự cố để đội ngũ phát triển và hỗ trợ có thể xử lý kịp thời. Các lỗi nghiêm trọng cần được khắc phục ngay lập tức để đảm bảo hệ thống không bị gián đoạn.

4. Khả năng sử dụng (Usability)

- **Giao diện người dùng (UI)**: Website phải có giao diện dễ sử dụng, trực quan và thân thiện với người dùng. Các chức năng như tìm kiếm sản phẩm, lọc sản phẩm, thanh toán và quản lý giỏ hàng cần phải được thiết kế sao cho người dùng có thể thao tác dễ dàng mà không gặp khó khăn.
- **Trải nghiệm người dùng (UX)**: Website cần đảm bảo cung cấp trải nghiệm người dùng tuyệt vời, bao gồm tính dễ dàng trong việc tìm kiếm, tham khảo và mua sắm sản phẩm. Các quy trình cần phải được tối ưu hóa để người dùng có thể hoàn thành giao dịch nhanh chóng và thuận tiện.
- **Hỗ trợ đa thiết bị (Responsive Design)**: Website phải hoạt động tốt trên mọi thiết bị, bao gồm máy tính để bàn, laptop, điện thoại di động và máy tính bảng. Giao diện người dùng cần tự động điều chỉnh để phù hợp với màn hình của từng loại thiết bị.

5. Khả năng tương thích (Compatibility)

- **Trình duyệt và hệ điều hành**: Website phải tương thích với các trình duyệt phổ biến như Google Chrome, Firefox, Safari, và Microsoft Edge, cũng như các hệ điều hành phổ biến như Windows, macOS, iOS, và Android.
- **Tích hợp với các dịch vụ bên ngoài**: Hệ thống cần có khả năng tích hợp với các dịch vụ thanh toán trực tuyến (PayPal, Stripe), các công cụ phân tích (Google Analytics), mạng xã hội (Facebook, Instagram), và các phần mềm bên ngoài phục vụ cho việc quản lý sản phẩm, kho hàng, hoặc giao hàng.

6. Khả năng phân tích và báo cáo

- **Phân tích hành vi người dùng**: Website cần có các công cụ phân tích người dùng để theo dõi hành vi của khách hàng trên website, chẳng hạn như Google Analytics, để hiểu rõ hơn về hành vi mua sắm và tối ưu hóa trải nghiệm người dùng.
- **Báo cáo doanh thu và tồn kho**: Các báo cáo về doanh thu, đơn hàng, tồn kho và các chỉ số quan trọng khác cần được cung cấp một cách dễ dàng và tự động để hỗ trợ việc ra quyết định của quản lý.

7. Tuân thủ pháp lý và quy định

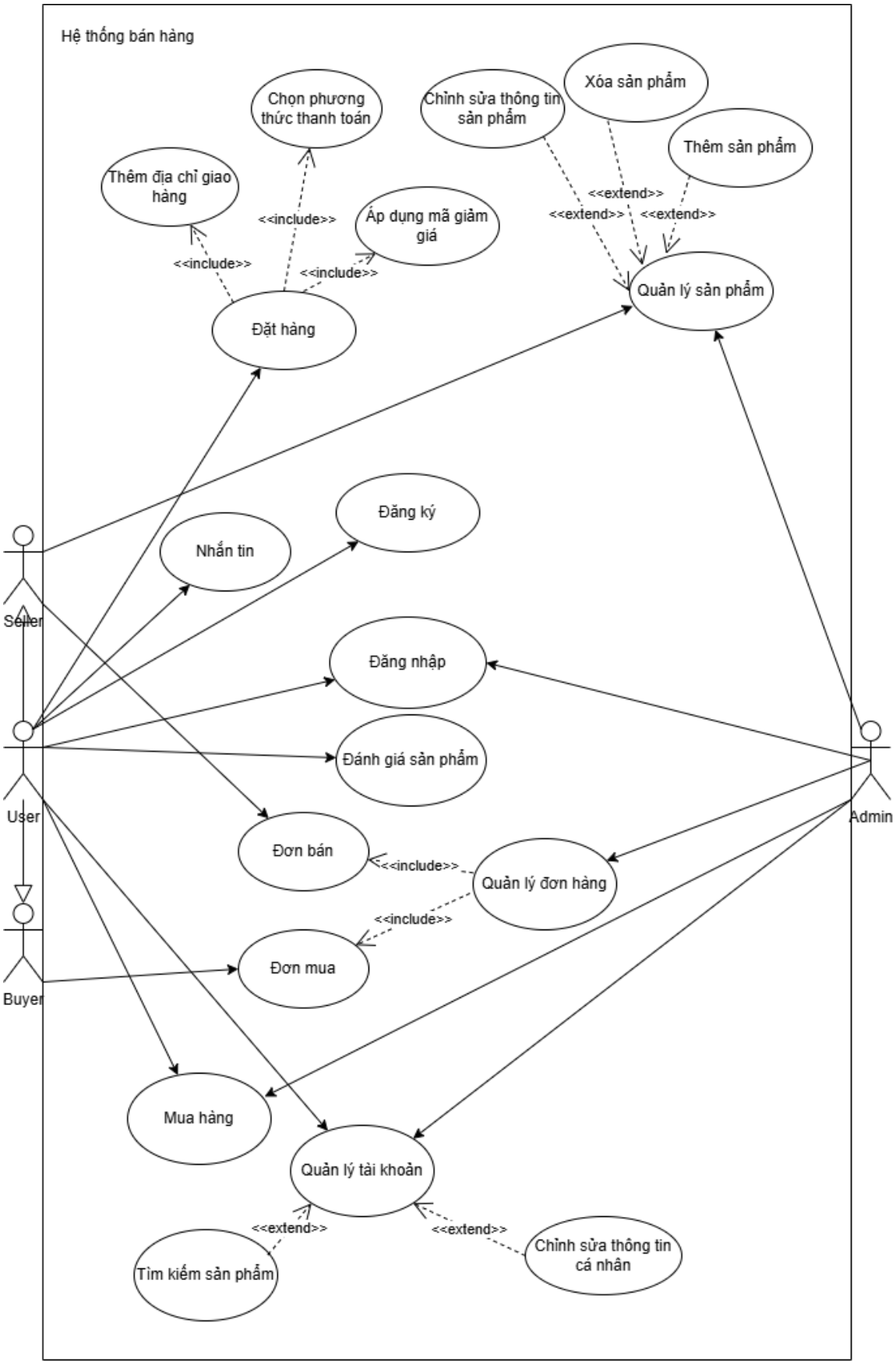
- **Chính sách bảo mật và quyền riêng tư**: Website cần tuân thủ các quy định về bảo mật thông tin và quyền riêng tư của khách hàng, chẳng hạn như các quy định của GDPR (General Data Protection Regulation) đối với khách hàng ở Liên minh châu Âu.
- **Quy định về thuế và giao dịch**: Website cần phải đảm bảo việc tính toán thuế và xử lý các giao dịch thanh toán theo các quy định pháp luật hiện hành, bao gồm các chính sách về thuế suất và phí vận chuyển.

8. Hỗ trợ khách hàng

- **Hệ thống hỗ trợ khách hàng trực tuyến**: Website cần có các kênh hỗ trợ khách hàng như chat trực tuyến, email, hoặc số điện thoại. Các nhân viên hỗ trợ cần được đào tạo để giải quyết vấn đề của khách hàng một cách nhanh chóng và hiệu quả.

3.3. Sơ đồ Use Case

Hệ thống bán hàng gồm có các tác nhân là Người dùng(User), Người mua(Buyer), Người bán(Seller) và quản trị hệ thống(Admin)



Biểu đồ 1: Biểu đồ ca sử dụng tổng quát cảu website

3.4. Đặc tả Use Case

3.4.1. Đăng ký

Bảng 1: Mô tả ca sử dụng đăng ký

Tên Use Case	Đăng ký
Mô tả	Use Case này cho phép người dùng đăng ký tài khoản mới
Các tác nhân	Người dùng
Tiền điều kiện	1. Người dùng chưa đăng nhập vào hệ thống. 2. Thiết bị kết nối Internet.
Hậu điều kiện	Chuyển sang giao diện người dùng hoặc admin
Các sự kiện chính	1. Người dùng truy cập vào website. 2. Người dùng chọn đăng ký. 3. Người dùng nhập thông tin tài khoản muốn đăng ký. 4. Hệ thống kiểm tra tài khoản có hợp lệ hay không. 5. Đăng ký thành công.
Các sự kiện phụ	1. Hệ thống hiển thị thông báo cho trường hợp tài khoản hoặc mật khẩu không đúng định dạng 2. Người dùng nhập lại thông tin. 3. Hệ thống xác nhận tài khoản đủ yêu cầu.

3.4.2. Đăng nhập

Bảng 2: Mô tả ca sử dụng đăng nhập

Tên Use Case	Đăng nhập
Mô tả	Use Case này cho phép người dùng hoặc admin đăng nhập vào hệ thống
Các tác nhân	Người dùng, admin
Tiền điều kiện	Người dùng, admin chưa đăng nhập vào hệ thống
Hậu điều kiện	Chuyển sang giao diện người dùng hoặc admin
Các sự kiện chính	1. Người dùng/admin truy cập vào web. 2. Người dùng/admin chọn đăng nhập 3. Nhập thông tin người dùng 4. Hệ thống check user có hợp lệ hay không 5. Chuyển sang giao diện người dùng
Các sự kiện phụ	Đăng nhập thất bại 1. Hệ thống chuyển người dùng về lại trang đăng nhập 2. Hệ thống hiển thị thông báo không hợp lệ

3.4.3. Chỉnh sửa thông tin cá nhân

Bảng 3: Mô tả ca sử dụng chỉnh sửa thông tin cá nhân

Tên Use Case	Chỉnh sửa thông tin cá nhân
Mô tả	Use Case giúp người dùng chỉnh sửa thông tin tài khoản
Các tác nhân	Người dùng, admin
Điều kiện kích hoạt	Người dùng, admin chọn chức năng chỉnh sửa thông tin
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.
Hậu điều kiện	Người dùng,admin chỉnh sửa tài khoản thành công
Các sự kiện chính	1. Người dùng chọn chức năng chỉnh sửa thông tin 2. Người dùng điền form thông tin tài khoản cần chỉnh sửa 3. Người dùng xác nhận
Các sự kiện phụ	1. Hệ thống hiển thị thông báo lỗi cho các trường dữ liệu không hợp lệ và không thay đổi trạng thái hiện tại. 2. Người dùng nhập lại thông tin cho các trường không hợp lệ. 3 Hệ thống xác nhận lại tất cả các trường đã đạt yêu cầu.

3.4.4. Tìm kiếm sản phẩm

Bảng 4: Mô tả ca sử dụng tìm kiếm sản phẩm

Tên Use Case	Tìm kiếm sản phẩm
Mô tả	Use Case này cho phép người dùng tìm kiếm sản phẩm có trong trang web
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đăng nhập vào hệ thống
Hậu điều kiện	Không có
Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn tìm kiếm sản phẩm 3. Người dùng nhập tên sản phẩm 4. Hệ thống kiểm tra sản phẩm 5. Hiển thị các sản phẩm
Các sự kiện phụ	1. Hệ thống không tìm thấy sản phẩm 2. Thông báo không tìm thấy sản phẩm 3. Chuyển về giao diện tìm kiếm sản phẩm 4. Trở về luồng 3 của sự kiện chính

3.4.5. Mua hàng

Bảng 5: Mô tả ca sử dụng mua hàng

Tên Use Case	Mua hàng

Mô tả	Use Case này cho phép người dùng có thể thêm sản phẩm vào giỏ mua.
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.
Hậu điều kiện	Đơn hàng được lưu vào cơ sở dữ liệu.
Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn vào sản phẩm muốn mua. 3. Người dùng chọn các thông tin thêm. 4. Hệ thống cung cấp thông tin kích cỡ số lượng. 5. Người dùng chọn thông tin đầy đủ vào các trường dữ liệu. 6. Hệ thống xác nhận tính hợp lệ của thông tin mà người dùng chọn. 7. Người dùng chọn checkout.
Các sự kiện phụ	1. Hệ thống hiển thị thông báo lỗi cho các trường dữ liệu không hợp lệ và không thay đổi trạng thái hiện tại. 2. Người dùng nhập lại thông tin cho các trường không hợp lệ. 3 Hệ thống xác nhận lại tất cả các trường đã đạt yêu cầu.

3.4.6. Đặt hàng

Bảng 6: Mô tả ca sử dụng thanh toán

Tên Use Case	Thanh toán

Mô tả	Use Case này cho phép người dùng thanh toán đơn hàng.
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.
Hậu điều kiện	Người dùng thanh toán thành công.
Các sự kiện chính	1. Người dùng điền thông tin nhận hàng và chọn thanh toán. 2. Người dùng chọn phương thức thanh toán và dùng mã giảm giá nếu có. 3.1. Người dùng chọn thanh toán khi nhận hàng, hệ thống sẽ gửi thông báo xác nhận tới người dùng. 3.2. Người dùng chọn các hình thức còn lại, hệ thống sẽ yêu cầu người dùng nhập thông tin thẻ thanh toán. 4. Người dùng xác nhận thanh toán
Các sự kiện phụ	3c1. Hệ thống thông báo lỗi với các trường hợp thẻ không hợp lệ. 3c2. Người dùng nhập lại thẻ thanh toán.

3.4.7. Quản lý đơn hàng

Bảng 7: Mô tả ca sử dụng quản lý đơn hàng

Tên Use Case	Quản lý đơn hàng
Mô tả	Use case này cho admin có thể thêm,xoá sản phẩm trong các đơn hàng
Các tác nhân	Admin.
Tiền điều kiện	Người dùng thêm sản phẩm vào giỏ hàng
Hậu điều kiện	- Sản phẩm được thêm/xóa vào/khỏi database. - Sản phẩm hiển thị lên danh mục/ sản phẩm được gỡ khỏi tất cả danh mục.
Các sự kiện chính	1. Người dùng/Admin đăng nhập hệ thống và chọn giỏ hàng 2. Người dùng/Admin nhập thông tin sản phẩm muốn thêm/xóa. 3. Người dùng/Admin xác nhận thêm/xoá sản phẩm. 4. Hệ thống cập nhật lại sản phẩm trong database
Các sự kiện phụ	Không có.

3.4.8. Quản lý sản phẩm

Bảng 8: Mô tả ca sử dụng quản lý sản phẩm

Tên Use Case	Quản lý sản phẩm
Mô tả	Use case này cho phép admin có thể thêm xoá,sửa sản phẩm trong hệ thống.
Các tác nhân	Admin, Seller

Tiền điều kiện	Admin đăng nhập vào hệ thống
Hậu điều kiện	Không có
Các sự kiện chính	1. Tại giao diện chính, admin chọn chức năng chỉnh sửa sản phẩm 2. Admin nhập thông tin sản phẩm muốn thêm/xóa,chỉnh sửa. 3. Admin xác nhận thêm/xoá,chỉnh sửa sản phẩm. 4. Hệ thống cập nhật lại sản phẩm trong database
Các sự kiện phụ	Không có.

3.4.9. Quản lý người dùng

Bảng 9: Mô tả ca sử dụng quản lý người dùng

Tên Use Case	Quản lý người dùng
Mô tả	Use Case giúp admin chỉnh sửa thông tin/xoá người dùng
Các tác nhân	Admin
Tiền điều kiện	Admin đã đăng nhập vào ứng dụng.
Hậu điều kiện	Admin chỉnh sửa tài khoản thành công
Các sự kiện chính	1. Admin chọn danh sách các người dùng 2. Chọn chỉnh sửa thông tin/xoá người dùng 3. Xác nhận
	1. Hệ thống hiển thị thông báo lỗi cho các trường dữ liệu không hợp lệ, không thay đổi trạng thái hiện tại. 2. Admin nhập lại thông tin cho các trường không hợp lệ.

Các sự kiện phụ	3 Hệ thống xác nhận lại tất cả các trường đã đạt yêu cầu.

3.4.10. Quản lý đơn hàng

Bảng 10: Mô tả ca sử dụng quản lý đơn hàng

Tên Use Case	Quản lý đơn hàng.
Mô tả	Use Case này cho phép người dùng có thể quản lý đơn bán, đơn mua.
Các tác nhân	Người dùng
Điều kiện kích hoạt	Người dùng chọn chức năng quản lý đơn hàng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.
Hậu điều kiện	Không có

Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn chức năng quản lý đơn mua, đơn bán. 3. Người dùng truy cập vào đơn hàng muốn xem để biết thêm thông tin. 4. Người dùng có thể xác nhận đơn hàng.
Các sự kiện phụ	Không có

3.4.11. Đánh giá sản phẩm

Bảng 11: Mô tả ca sử dụng đánh giá sản phẩm

Tên Use Case	Đánh giá sản phẩm
Mô tả	Use Case này cho phép người dùng có thể đánh giá các sản phẩm trên website.
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.
Hậu điều kiện	Không có
Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn chức năng "create new review". 4. Người dùng chọn mức đánh giá và điền thêm nhận xét. 5. Sau khi xong người dùng chọn submit.
Các sự kiện phụ	Không có

3.4.11. Nhắn tin

Bảng 12: Mô tả ca sử dụng nhắn tin

Tên Use Case	Nhắn tin
Mô tả	Use Case này cho phép người dùng có thể nhắn tin tức thời.
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng
Hậu điều kiện	Không có
Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn chức năng nhắn tin. 3. Người dùng chọn tên email người dùng 4. Người dùng nhập tin nhắn và gửi
Các sự kiện phụ	Không có

3.4.12. Chatbot

Bảng 13: Mô tả ca sử dụng chatbot

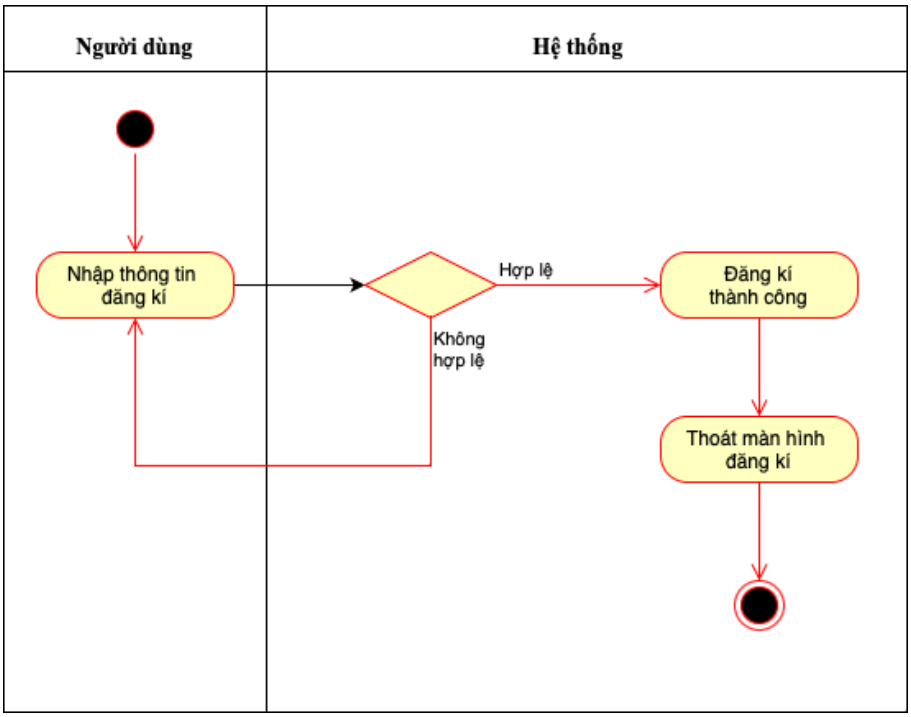
Tên Use Case	Chatbot
Mô tả	Use Case này cho phép người dùng có thể nhắn tin với Chatbot cung cấp thông tin về Shop.
Các tác nhân	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập vào ứng dụng.

Hậu điều kiện	Không có
Các sự kiện chính	1. Người dùng truy cập vào ứng dụng. 2. Người dùng chọn chức năng Chatbot. 3. Người dùng nhập tin nhắn và gửi
Các sự kiện phụ	Không có

3.5. Mô tả các biểu đồ hoạt động

3.5.1. Đăng ký

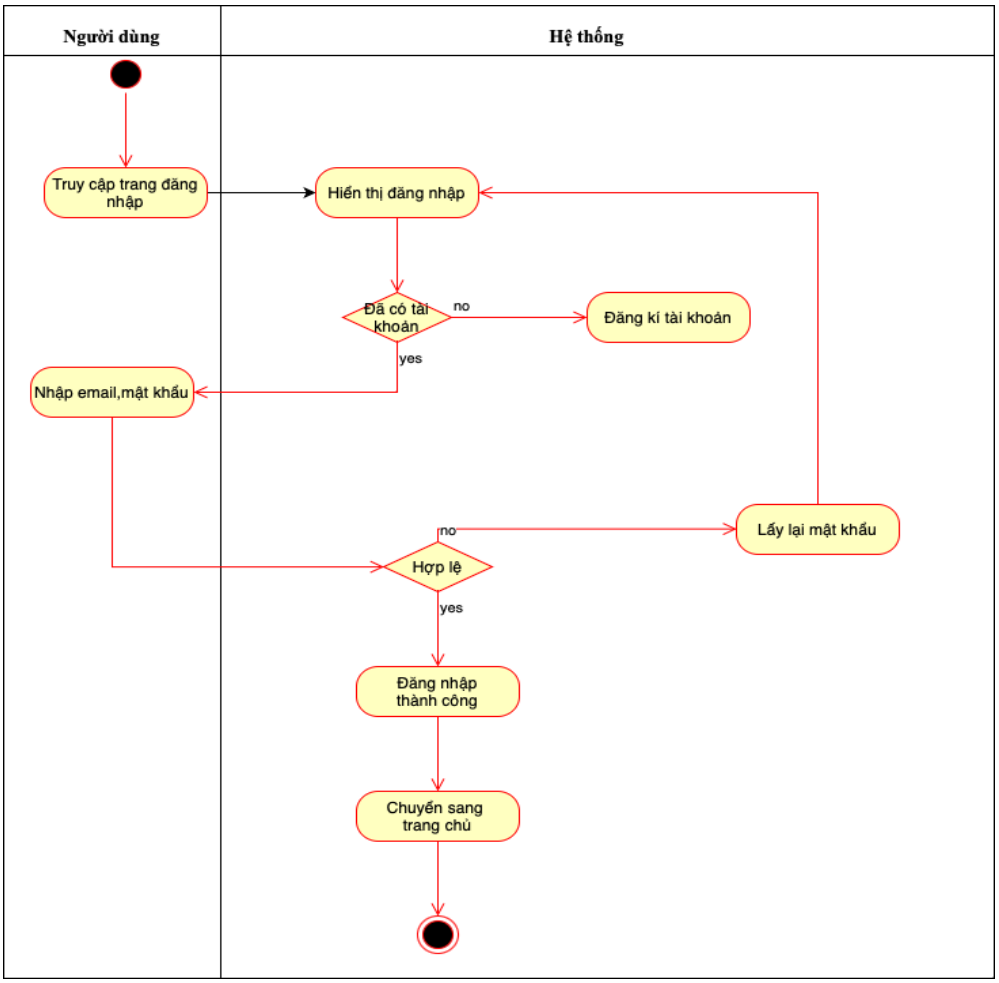
Mô tả hoạt động đăng ký: Khi người dùng truy cập màn hình đăng ký, hệ thống sẽ hiển thị một mẫu đăng ký yêu cầu họ nhập các thông tin cần thiết, bao gồm tên đăng nhập, mật khẩu, địa chỉ email và các thông tin khác theo yêu cầu của hệ thống. Người dùng sẽ điền thông tin vào các trường tương ứng và nhấn nút "Đăng ký" để gửi thông tin cho hệ thống. Sau khi nhận thông tin từ người dùng, hệ thống sẽ tiến hành kiểm tra tính hợp lệ của các thông tin được cung cấp. Hệ thống sẽ kiểm tra xem tên đăng nhập hoặc địa chỉ email đã được sử dụng trước đó hay chưa, đảm bảo mật khẩu đạt yêu cầu về độ dài và độ phức tạp, cũng như xác thực các thông tin khác theo quy định. Nếu hệ thống xác định rằng thông tin đăng ký hợp lệ, người dùng sẽ nhận được phản hồi từ hệ thống thông báo thành công và tạo tài khoản mới cho người dùng. Trong trường hợp thông tin không hợp lệ hoặc thiếu sót, hệ thống sẽ thông báo lỗi và yêu cầu người dùng sửa đổi thông tin trước khi tiếp tục gửi yêu cầu đăng ký. Sau khi đăng ký thành công, người dùng có thể đăng nhập vào hệ thống.



Biểu đồ 2: Biểu đồ hoạt động ca sử dụng đăng ký

3.5.2. Đăng nhập

Mô tả hoạt động đăng nhập: Khi người dùng muốn đăng nhập vào hệ thống, họ sẽ truy cập màn hình đăng nhập, nơi hệ thống hiển thị giao diện yêu cầu họ nhập thông tin đăng nhập, bao gồm tên đăng nhập và mật khẩu. Người dùng sau đó điền vào các trường tương ứng và nhấn nút đăng nhập để gửi thông tin cho hệ thống. Hệ thống tiến hành kiểm tra và xác minh thông tin người dùng, đảm bảo danh tính và quyền truy cập của họ. Trong trường hợp thông tin đăng nhập không chính xác hoặc tài khoản không hợp lệ, hệ thống sẽ thông báo lỗi cho người dùng và đưa ra gợi ý về việc lấy lại mật khẩu nếu cần thiết.

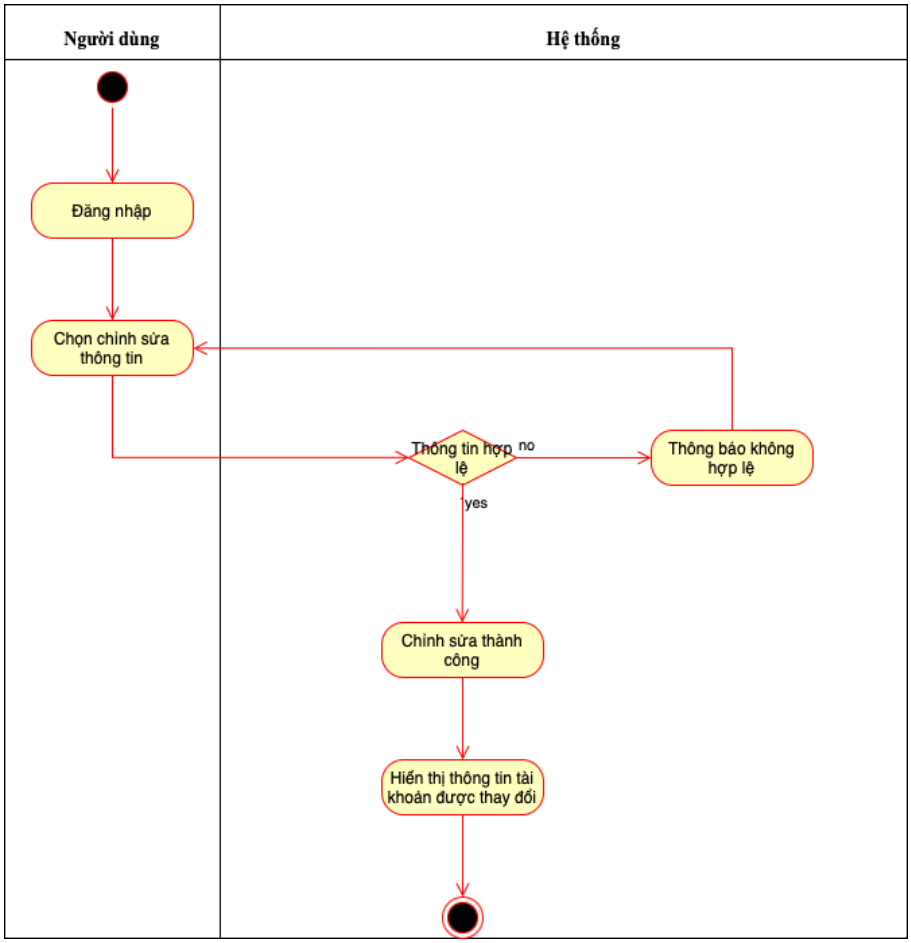


Biểu đồ 3: Biểu đồ hoạt động ca sử dụng đăng nhập

Khi thông tin đăng nhập được xác minh thành công, người dùng sẽ được chuyển hướng đến trang chủ của website, nơi họ có thể tiếp tục trải nghiệm và tương tác với các tính năng, dịch vụ hoặc sản phẩm được cung cấp bởi hệ thống.

3.5.3. Chỉnh sửa thông tin người dùng

Mô tả hoạt động: người dùng muốn xem và chỉnh sửa thông tin cá nhân của mình trên hệ thống. Đầu tiên, họ truy cập vào trang hồ sơ cá nhân, nơi hiển thị đầy đủ thông tin liên quan, bao gồm tên, địa chỉ email, số điện thoại và các thông tin khác. Để chỉnh sửa thông tin, người dùng chọn chức năng chỉnh sửa thông tin hoặc biểu tượng chỉnh sửa tương ứng, chọn chức năng chỉnh sửa thông tin và nhập thông tin mới. Sau khi kiểm tra tính hợp lệ của thông tin mới,nếu không hợp lệ hệ thống sẽ hiện thị thông báo cho người dùng, nếu hợp lệ hệ thống sẽ thay đổi thông tin cho người dùng và hiển thị thông tin cá nhân mới.



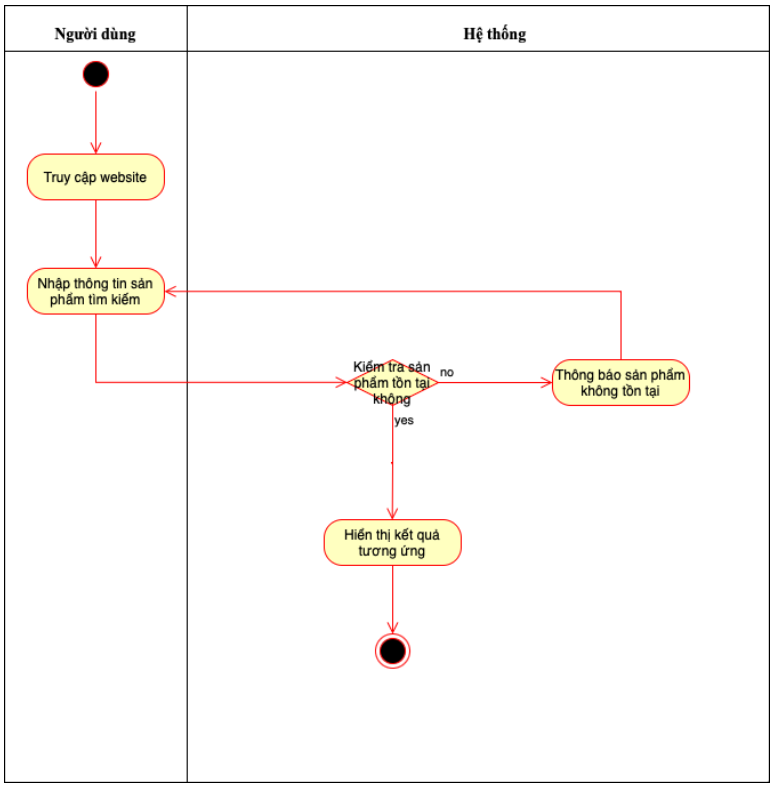
Biểu đồ 4: Biểu đồ hoạt động ca sử dụng chỉnh sửa thông tin cá nhân

3.5.4. Tìm kiếm sản phẩm

Mô tả hoạt động tìm kiếm sản phẩm: Trong hoạt động tìm kiếm sản phẩm, khi người dùng truy cập trang chủ của website, họ có thể sử dụng công cụ tìm kiếm sản phẩm được cung cấp trên trang để tìm kiếm các sản phẩm mà họ quan tâm. Để thực hiện tìm kiếm, người dùng nhập từ khóa liên quan đến sản phẩm cần tìm vào thanh tìm kiếm và nhấn nút "Tìm kiếm".

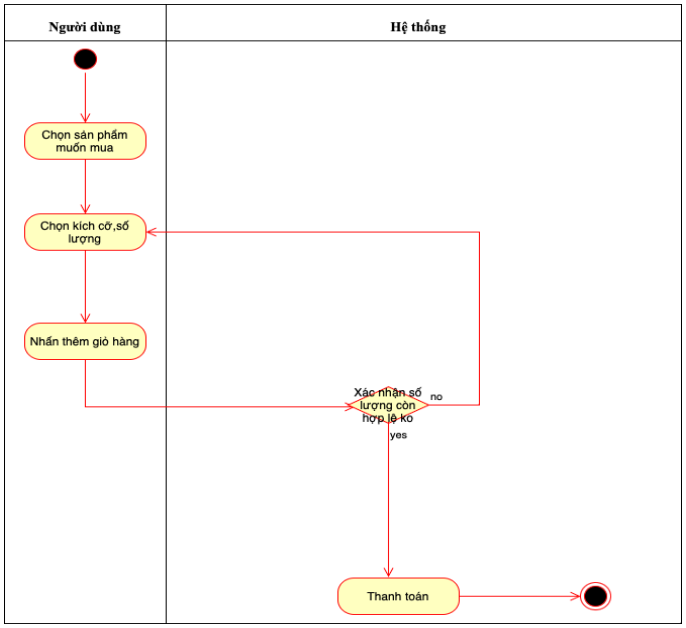
Sau khi nhận yêu cầu từ người dùng, hệ thống sẽ kiểm tra trong cơ sở dữ liệu của mình để tìm sản phẩm phù hợp với từ khóa được cung cấp. Nếu không tìm thấy sản phẩm nào phù hợp, hệ thống sẽ thông báo cho người dùng rằng không có sản phẩm nào tồn tại với từ khóa đã nhập. Người dùng có thể thử lại bằng cách nhập từ khóa khác hoặc sử dụng các bộ lọc để thu hẹp kết quả tìm kiếm.

Trong trường hợp tìm thấy sản phẩm phù hợp, hệ thống sẽ hiển thị danh sách các kết quả tìm kiếm tương ứng với từ khóa, bao gồm tên sản phẩm, hình ảnh, giá cả và thông tin khác. Người dùng có thể xem chi tiết sản phẩm, so sánh giữa các sản phẩm và thực hiện mua hàng nếu họ quyết định chọn một sản phẩm nào đó.



Biểu đồ 5: Biểu đồ hoạt động ca sử dụng tìm kiếm sản phẩm

3.5.5. Mua hàng



Biểu đồ 6: Biểu đồ hoạt động ca sử dụng mua hàng

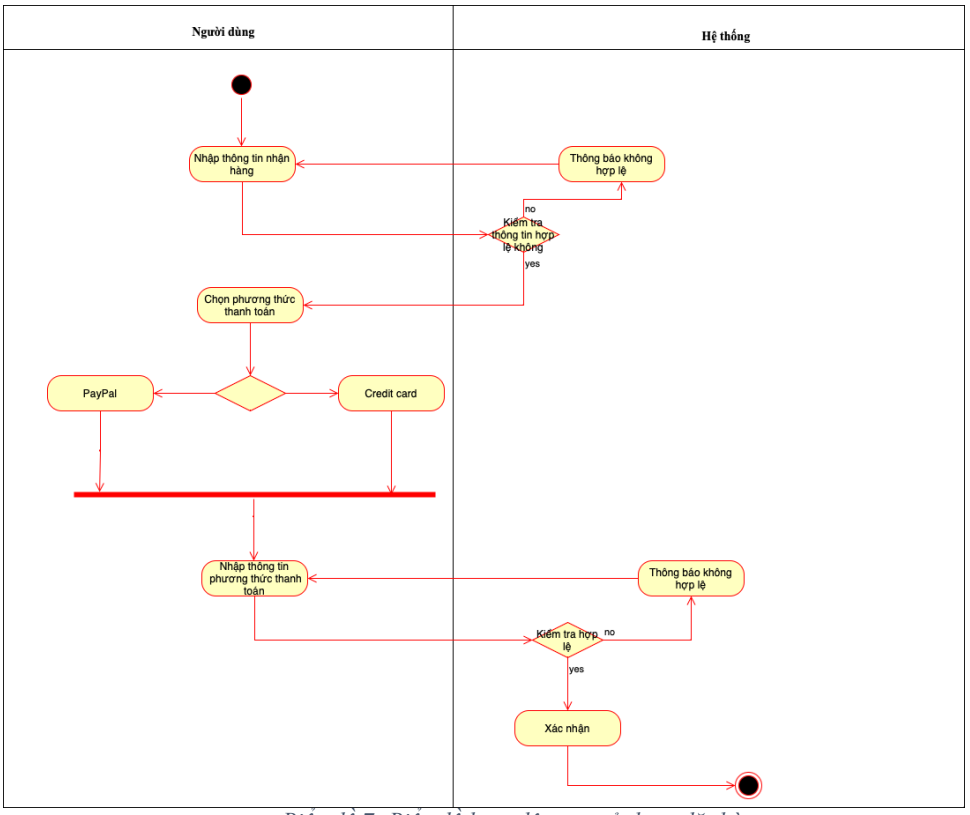
Mô tả hoạt động mua hàng: Khi người dùng đã chọn được sản phẩm mong muốn, họ sẽ tiến hành các bước lựa chọn thông tin chi tiết của sản phẩm.

Bắt đầu bằng việc chọn số lượng sản phẩm cần mua. Người dùng có thể tùy chỉnh số lượng tùy theo nhu cầu. Tiếp theo, họ có thể lựa chọn màu sắc và kích cỡ của sản phẩm nếu sản phẩm này có các lựa chọn đó.

Hệ thống sẽ kiểm tra tính hợp lệ của lựa chọn - chẳng hạn như sản phẩm với màu sắc và kích cỡ đã chọn có sẵn trong kho hay không. Nếu lựa chọn không hợp lệ, ví dụ như sản phẩm đã hết hàng, hệ thống sẽ thông báo cho người dùng và yêu cầu họ thực hiện lựa chọn khác.

Khi tất cả các lựa chọn đều hợp lệ và sản phẩm cần mua có sẵn trong kho, người dùng sẽ nhấn vào mục "Thêm vào giỏ hàng".

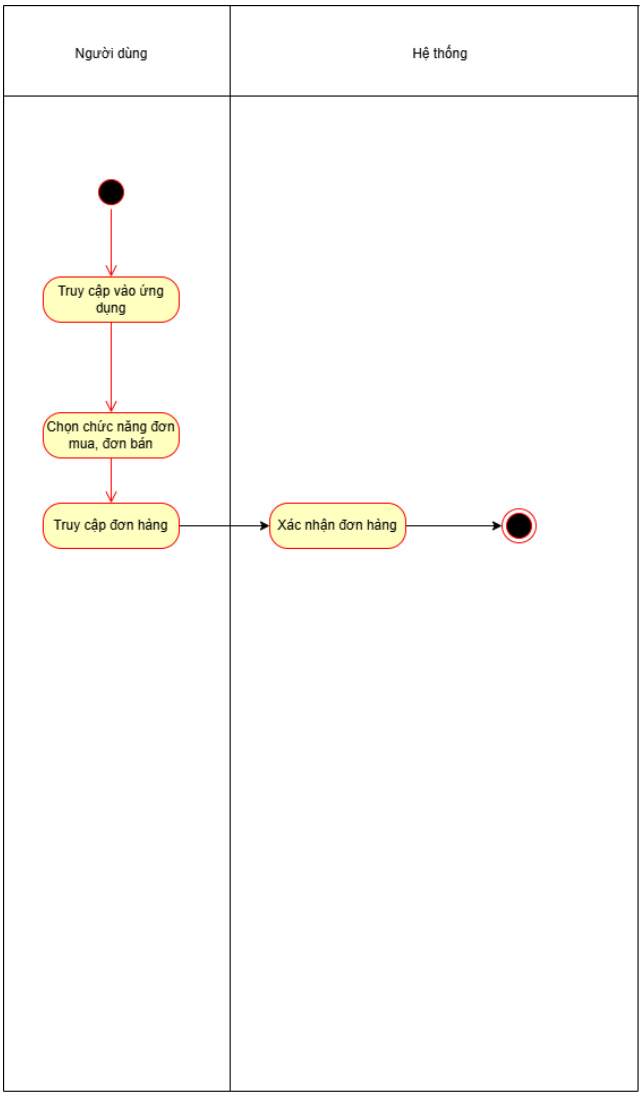
3.5.6. Đặt hàng



Biểu đồ 7: Biểu đồ hoạt động ca sử dụng đặt hàng

Mô tả hoạt động đặt hàng: Trong hoạt động thanh toán khi người dùng đã nhấp vào mục "Thêm vào giỏ hàng" để đưa sản phẩm vào giỏ hàng của mình. Sau đó, người dùng chọn "Checkout" để bắt đầu quá trình thanh toán. Nếu người dùng chưa đăng nhập, hệ thống sẽ chuyển đến giao diện đăng nhập yêu cầu người dùng cung cấp thông tin đăng nhập để tiếp tục quá trình mua hàng. Ngược lại, nếu người dùng đã đăng nhập, hệ thống sẽ tự động điền thông tin người nhận dựa trên thông tin trong hồ sơ cá nhân của họ. Người dùng sẽ kiểm tra và điền các thông tin cần thiết cho đơn hàng, bao gồm tên người nhận, địa chỉ giao hàng, số điện thoại liên hệ và phương thức thanh toán. Sau khi hoàn tất việc nhập thông tin, người dùng nhấn nút "Xác nhận đơn hàng" để gửi yêu cầu đặt hàng đến hệ thống. Hệ thống sẽ kiểm tra tính hợp lệ của thông tin người nhận, đảm bảo không có dữ liệu sai hoặc không phù hợp. Nếu thông tin không hợp lệ, hệ thống sẽ thông báo lỗi và yêu cầu người dùng điều chỉnh thông tin trước khi tiếp tục gửi yêu cầu đặt hàng. Nếu thông tin hợp lệ, hệ thống sẽ ghi nhận đơn hàng và xác nhận với người dùng rằng đơn hàng đã được đặt thành công. Khi người dùng xác nhận thông tin nhận hàng thành công, hệ thống sẽ yêu cầu người dùng chọn phương thức thanh toán.Có 2 phương thức thanh toán ở đây là PayPal và Credit Card. Người dùng cần chọn một trong hai phương thức và điền thông tin. Nếu thông tin không hợp lệ hệ thống sẽ yêu cầu người dùng nhập lại.Nếu thông tin hợp lệ hệ thống sẽ xác nhận và yêu cầu thanh toán cho đơn hàng.

3.5.7. Quản lý đơn hàng

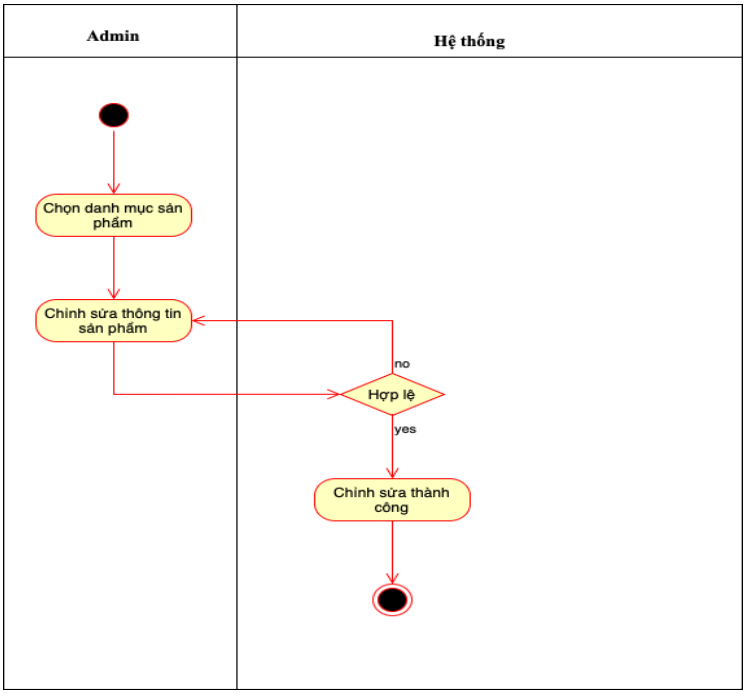


Biểu đồ 8: Biểu đồ hoạt động ca sử dụng quản lý đơn hàng

Biểu đồ hoạt động trong quản lý đơn hàng mô tả quy trình từ khi người dùng đăng nhập vào ứng dụng cho đến khi hoàn tất việc xử lý các đơn mua và bán. Quá trình bắt đầu khi người dùng mở ứng dụng và đăng nhập vào hệ thống, sau đó hệ thống sẽ kiểm tra kết nối internet. Nếu thiết bị không có kết nối, người dùng sẽ được yêu cầu kết nối lại. Khi đảm bảo kết nối, người dùng tiếp tục chọn chức năng "Quản lý đơn hàng" để truy cập vào các đơn mua hoặc bán. Sau đó, người dùng có thể chọn một đơn hàng cụ thể để xem chi tiết, bao gồm thông tin trạng thái, sản phẩm, giá trị đơn hàng và thời gian giao hàng. Cuối cùng, người dùng có thể xác nhận đơn hàng, sửa đổi thông tin hoặc hủy đơn nếu cần thiết. Quá trình quản lý đơn hàng kết thúc khi người dùng đã thực hiện xong các thao tác cần thiết, giúp đảm bảo đơn hàng được xử lý hiệu quả và chính xác.

3.5.8. Quản lý sản phẩm

Mô tả hoạt động quản lý sản phẩm: Khi muốn cập nhật hoặc chỉnh sửa thông tin của một sản phẩm, admin sẽ tiếp cận với hệ thống thông qua giao diện quản lý sản phẩm. Tại đây, họ sẽ tìm kiếm và chọn sản phẩm cần cập nhật hoặc chỉnh sửa. Quá trình chỉnh sửa thông tin sản phẩm có thể bao gồm việc thay đổi giá, mô tả, hình ảnh, số lượng trong kho, và các thông tin khác. Khi thực hiện các thay đổi này, hệ thống sẽ kiểm tra tính hợp lệ của thông tin mới. Nếu chỉnh sửa không hợp lệ hệ thống sẽ yêu cầu chỉnh sửa lại. Nếu chỉnh sửa hợp lệ hệ thống sẽ ghi nhận chỉnh sửa thành công.

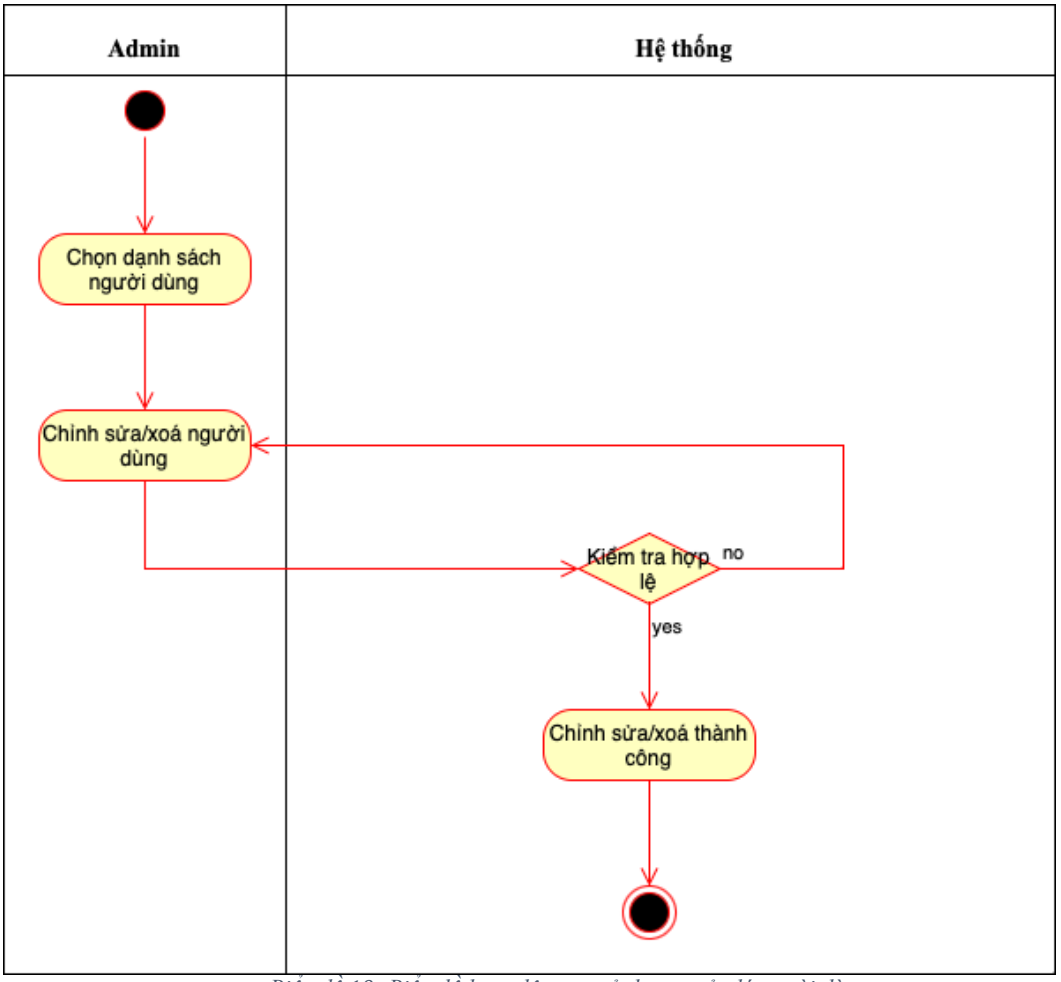


Biểu đồ 9: Biểu đồ hoạt động ca sử dụng quản lý sản phẩm

3.5.9. Quản lý người dùng

Mô tả hoạt động quản lý người dùng: Khi admin của hệ thống muốn quản lý người dùng, admin sẽ truy cập vào phần quản lý người dùng trên trang web hoặc ứng dụng. Tại đây, admin có thể xem danh sách các người dùng và chọn một người dùng để xem thông tin chi tiết hoặc thực hiện các thao tác quản lý như sửa, xoá hoặc tạo người dùng mới.

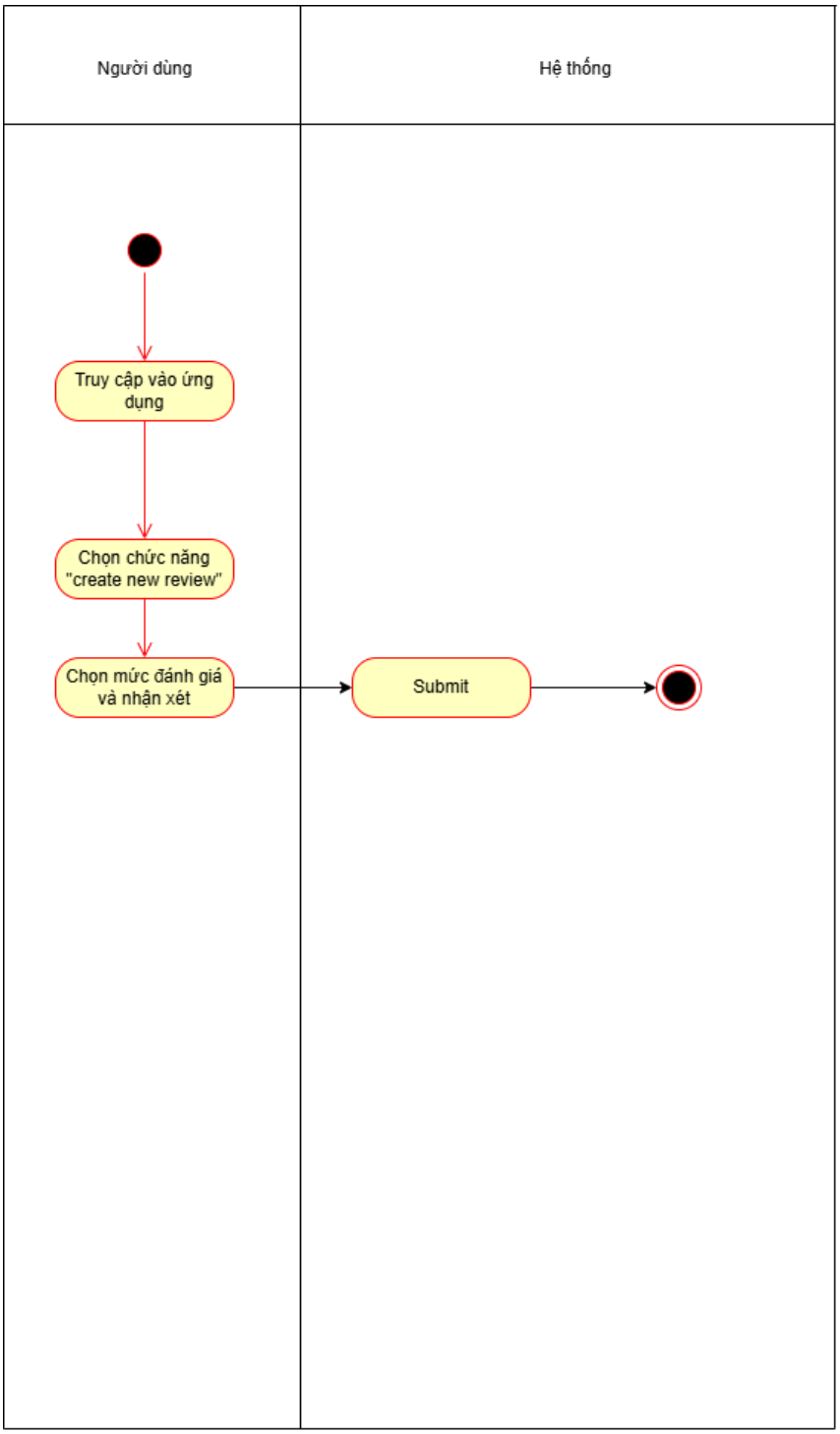
Nếu admin muốn sửa thông tin người dùng, họ sẽ thực hiện các thay đổi cần thiết và lưu lại để hệ thống cập nhật thông tin mới cho người dùng. Nếu thông tin chỉnh sửa không hợp lệ, hệ thống sẽ yêu cầu admin chỉnh sửa lại.



Biểu đồ 10: Biểu đồ hoạt động ca sử dụng quản lý người dùng

3.5.10. Đánh giá sản phẩm

Đánh giá sản phẩm từ người dùng trong các website bán hàng đóng vai trò quan trọng trong việc cung cấp thông tin chính xác và hữu ích cho các khách hàng tiềm năng. Sau khi mua và sử dụng sản phẩm, người dùng có thể để lại đánh giá, bao gồm những nhận xét chi tiết về chất lượng, tính năng, giá trị sử dụng và trải nghiệm chung. Những đánh giá này thường được thể hiện dưới dạng số sao và bình luận, giúp người mua khác dễ dàng đưa ra quyết định. Các phản hồi tích cực giúp tăng độ tin cậy của sản phẩm, trong khi những nhận xét mang tính xây dựng giúp nhà cung cấp cải thiện chất lượng sản phẩm hoặc dịch vụ. Ngoài ra, hệ thống đánh giá cũng cho phép người dùng lọc và tìm kiếm các ý kiến từ những người đã sử dụng sản phẩm trước đó, tạo nên một cộng đồng chia sẻ thông tin. Vì vậy, đánh giá sản phẩm không chỉ mang lại lợi ích cho người mua mà còn giúp nhà cung cấp cải thiện chất lượng và nâng cao trải nghiệm khách hàng.



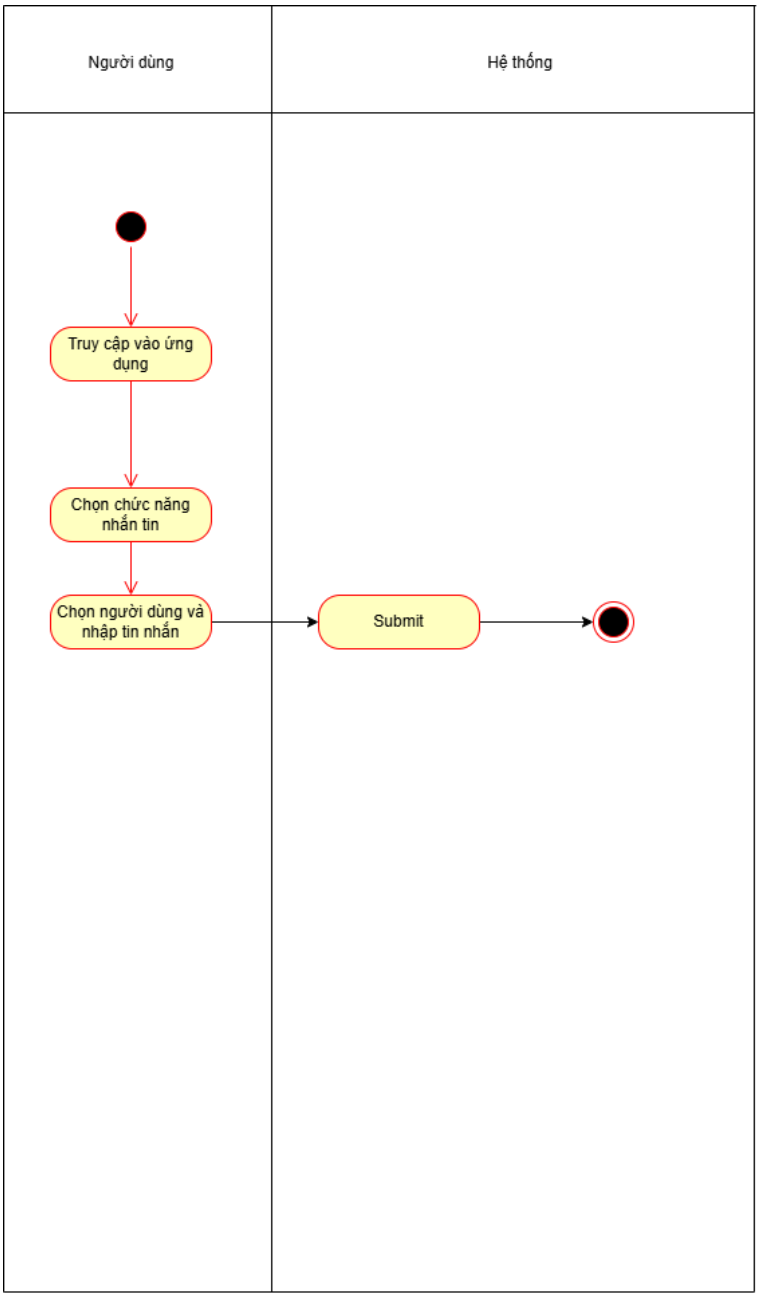
Biểu đồ 11: Biểu đồ hoạt động ca sử dụng đánh giá sản phẩm

3.5.11. Nhắn tin

Trong môi trường thương mại điện tử, việc tạo ra một hệ thống nhắn tin trực tiếp giữa người mua và người bán là rất quan trọng. Điều này không chỉ giúp giải đáp các thắc

mắc mà còn tăng cường trải nghiệm khách hàng, thúc đẩy quá trình mua hàng, và tạo ra sự kết nối trực tiếp, kịp thời. Một trong những cách tiếp cận hiệu quả là sử dụng Socket.io để nhắn tin tức thời.

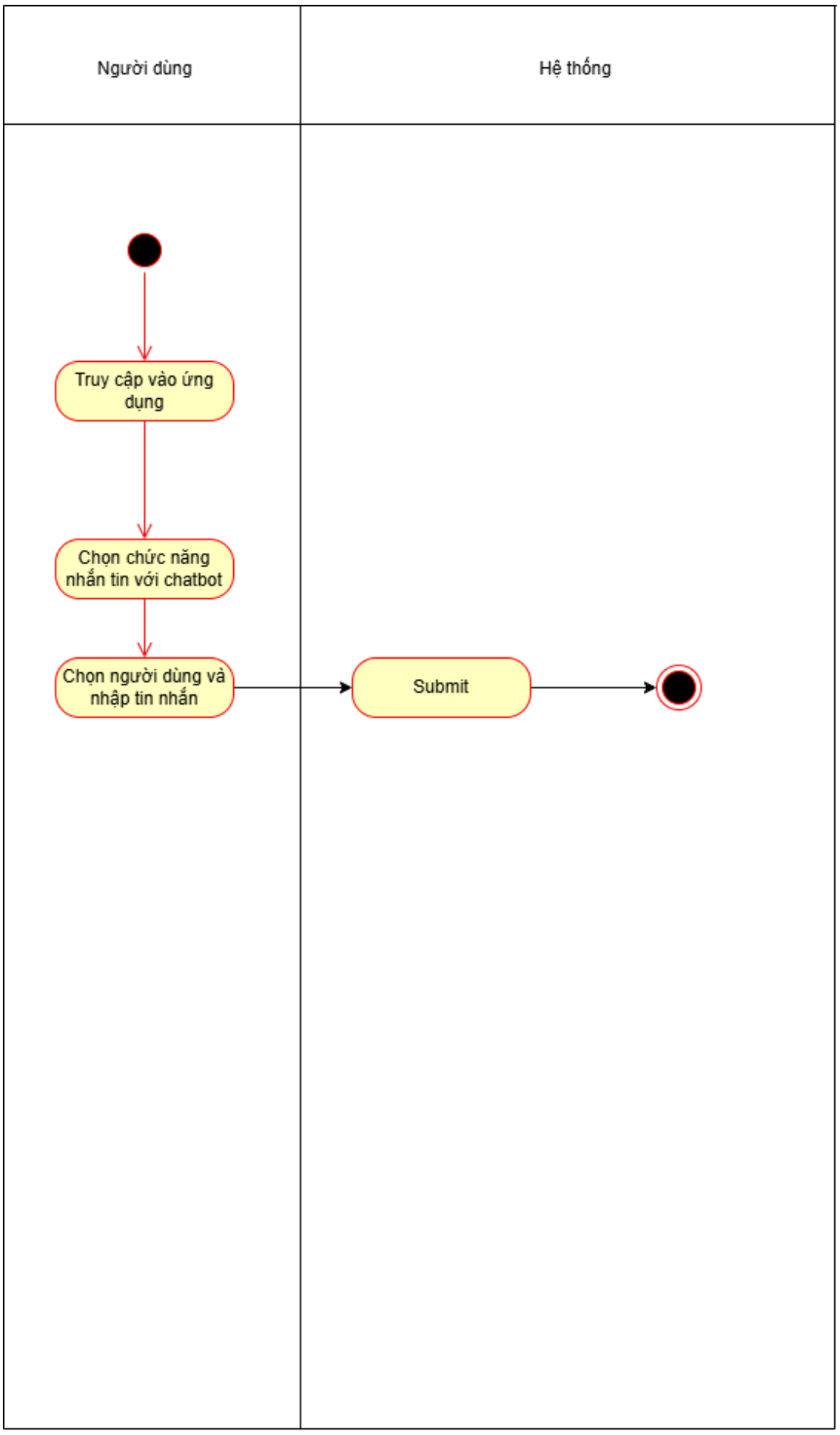
Hệ thống sẽ cho phép người dùng nhắn tin trực tiếp với người bán mà không cần phải rời khỏi nền tảng mua sắm, đồng thời giúp người bán dễ dàng quản lý và trả lời các câu hỏi của khách hàng trong thời gian thực. Việc hiển thị thông báo khi có tin nhắn mới, tô đậm email người nhận, cùng với tính năng tìm kiếm theo tên hoặc email của người dùng sẽ mang lại sự tiện lợi và tối ưu hóa trải nghiệm người dùng.



Biểu đồ 12: Biểu đồ hoạt động ca sử dụng nhắn tin

3.5.12. Chatbot

Hiện nay, các ứng dụng trí tuệ nhân tạo sử dụng mô hình ngôn ngữ lớn đã phát triển nhanh chóng và có nhiều thành tựu vượt bậc. Trong thương mại điện tử, chúng ta có thể ứng dụng trí tuệ nhân tạo để tạo ra Chatbot làm trợ lý trả lời câu hỏi về cửa hàng. Người dùng có thể đưa ra bất kỳ câu hỏi nào về mã giảm giá, sản phẩm, người dùng và Chatbot sẽ trả lời ngay tức thì.



Biểu đồ 13: Biểu đồ hoạt động ca sử dụng Chatbot

Chương 4: Phân tích và thiết kế hệ thống

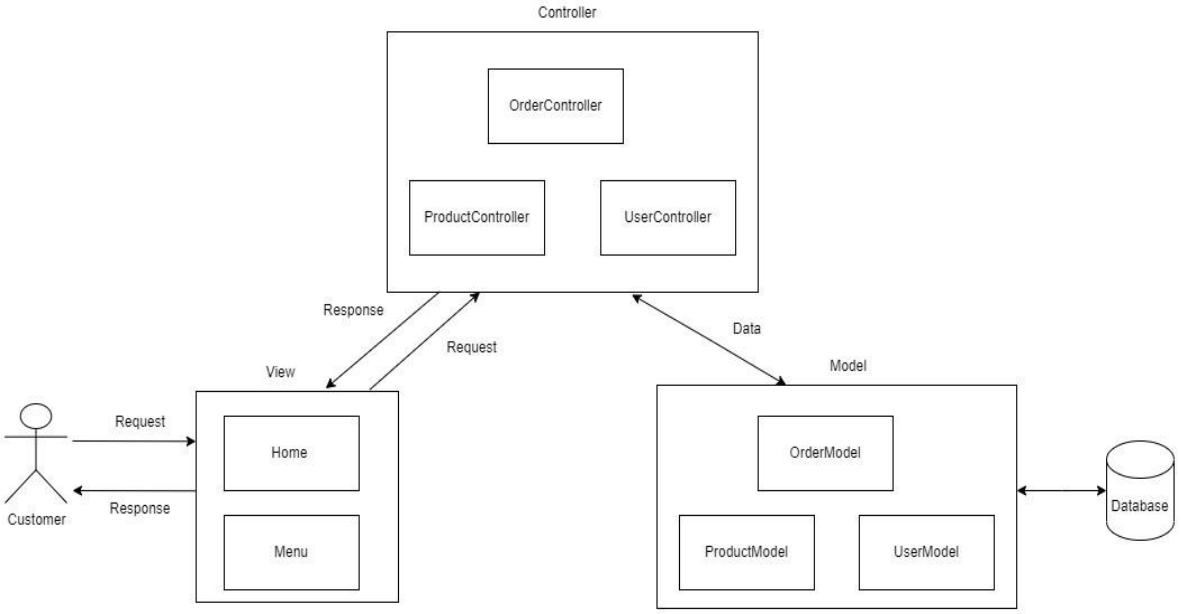
4.1. Phân tích kiến trúc

Hệ thống dựa trên kiến trúc **Client-Server**, mẫu kiến trúc phần mềm **MVC**.

Kiến trúc **Client-Server** là một mô hình phân tán trong đó các nhiệm vụ, nguồn lực và khả năng xử lý được phân chia giữa các máy chủ (được gọi là server) và các máy khách (được gọi là client). Trong mô hình này, máy chủ cung cấp dịch vụ và quản lý tài nguyên, trong khi máy khách yêu cầu và sử dụng dịch vụ này. Máy khách có thể là máy tính cá nhân, điện thoại di động hoặc bất kỳ thiết bị nào có khả năng kết nối mạng để truy cập dịch vụ.

Trong mô hình **MVC**:

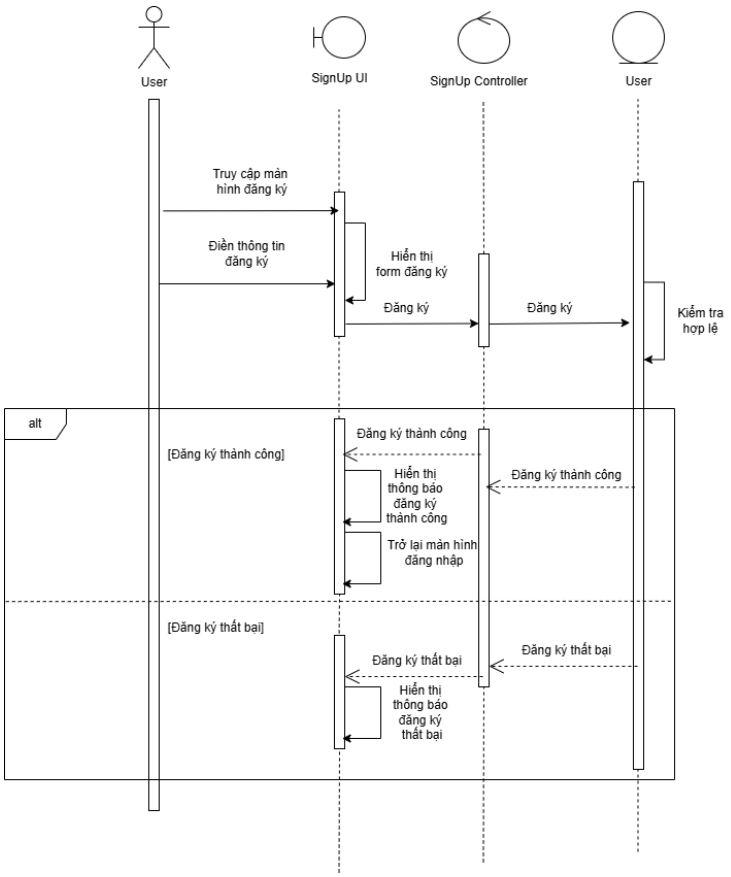
- **Model**: quản lý dữ liệu, quy tắc, và logic nghiệp vụ của ứng dụng. Nó xử lý các yêu cầu từ Controller, thao tác với cơ sở dữ liệu hoặc các nguồn dữ liệu khác, và sau đó trả kết quả về cho Controller.
- **View**: hiển thị thông tin, tương tác trực tiếp với người dùng, chuyển tiếp các yêu cầu tới hệ thống và hiển thị kết quả đầu ra cho người dùng.
- **Controller**: Thành phần này nhận các yêu cầu từ người dùng, sau đó dùng Model để lấy dữ liệu cần thiết và dùng View để tạo ra trang phù hợp để trả về cho người dùng. Nó hoạt động như một trung gian giữa Model và View.



Hình 2: Kiến trúc hệ thống thương mại điện tử

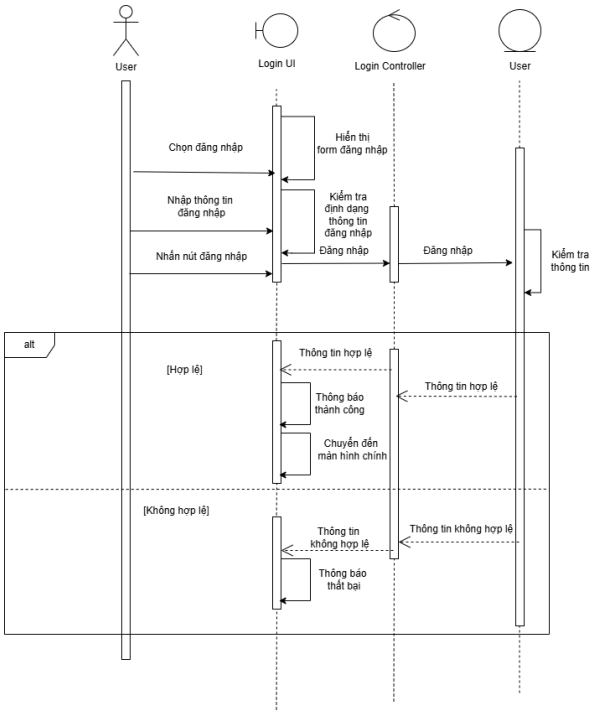
4.2. Biểu đồ tuần tự các Use Case

4.2.1. Đăng ký



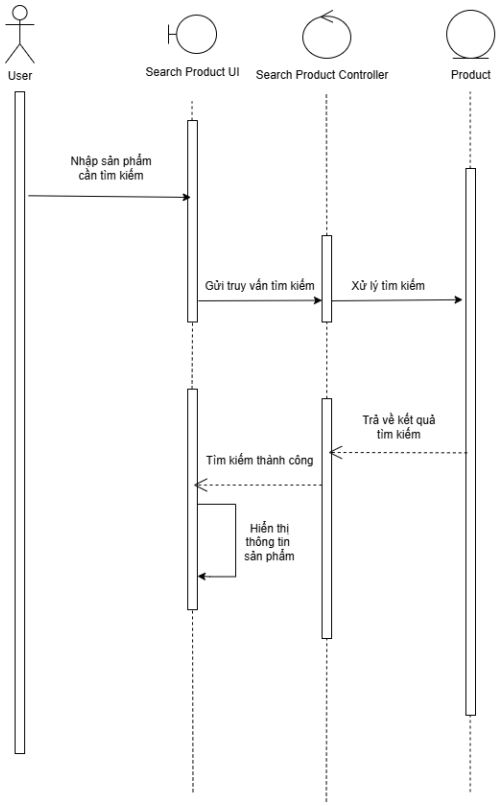
Hình 3: Biểu đồ tuần tự ca sử dụng đăng ký

4.2.2. Đăng nhập



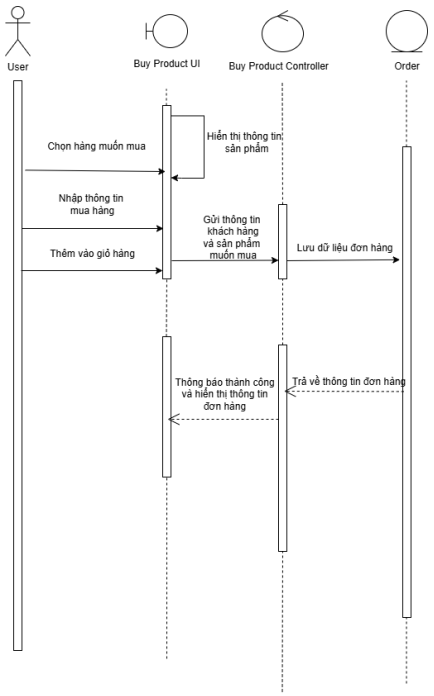
Hình 4: Biểu đồ tuần tự ca sử dụng đăng nhập

4.2.3 Tìm kiếm sản phẩm



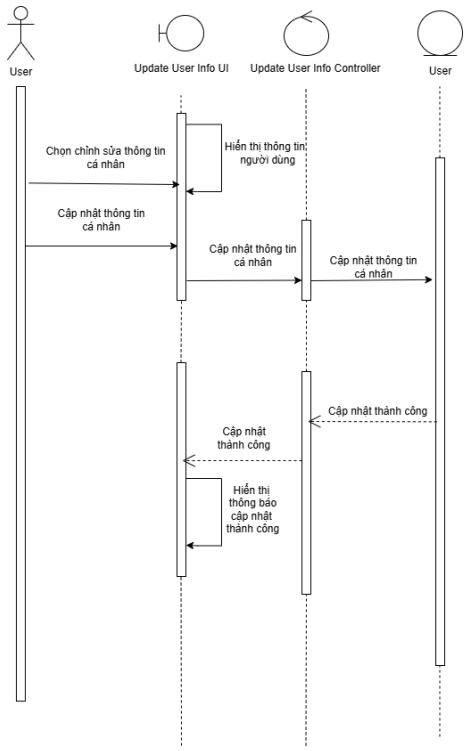
Hình 5: Biểu đồ tuần tự ca sử dụng tìm kiếm sản phẩm

4.2.4 Mua hàng



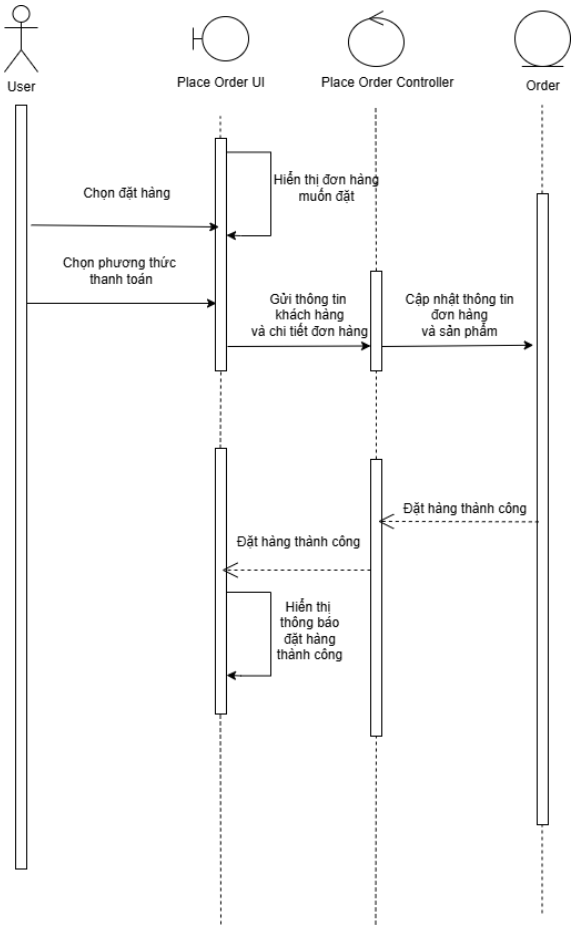
Hình 6: Biểu đồ tuần tự ca sử dụng mua hàng

4.2.5 Chỉnh sửa thông tin cá nhân



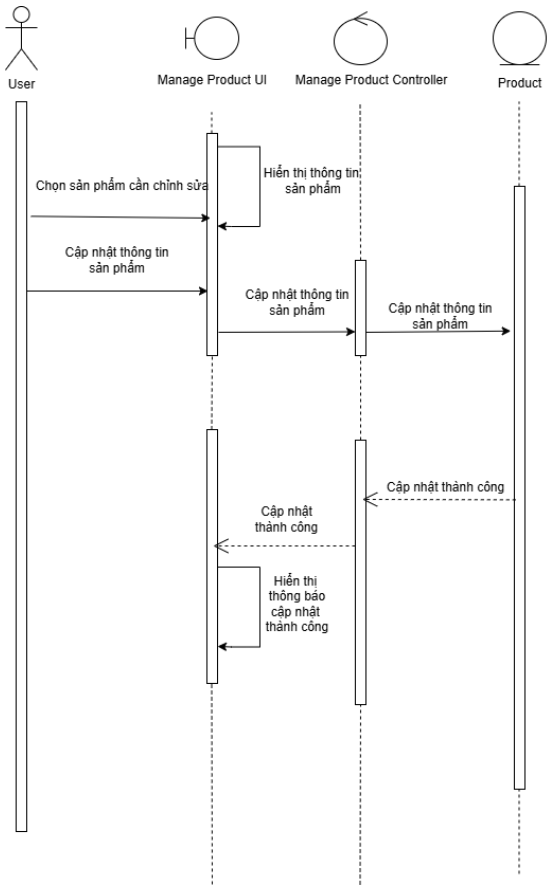
Hình 7: Biểu đồ tuần tự ca sử dụng chỉnh sửa thông tin cá nhân

4.2.6 Đặt hàng



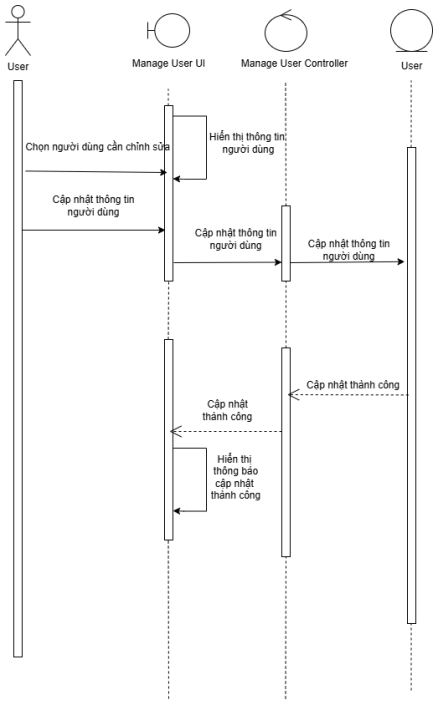
Hình 8: Biểu đồ tuần tự ca sử dụng đặt hàng

4.2.7 Quản lý sản phẩm



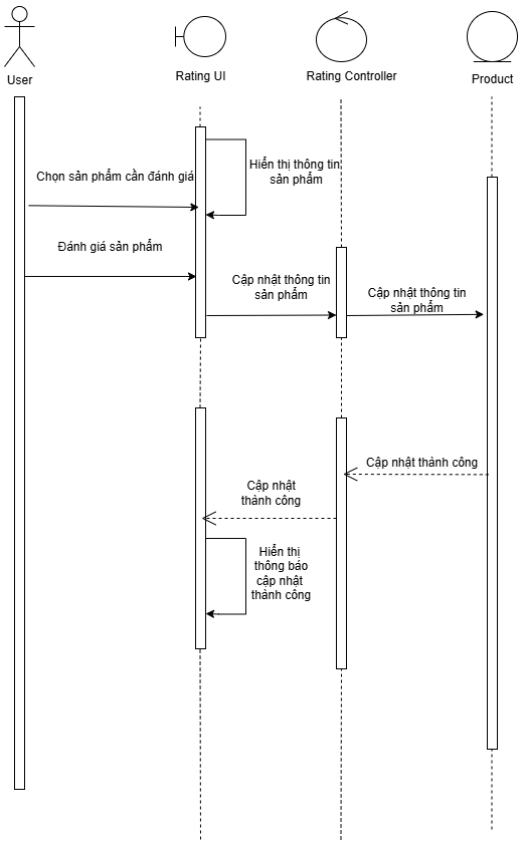
Hình 9: Biểu đồ tuần tự ca sử dụng quản lý sản phẩm

4.2.8 Quản lý người dùng



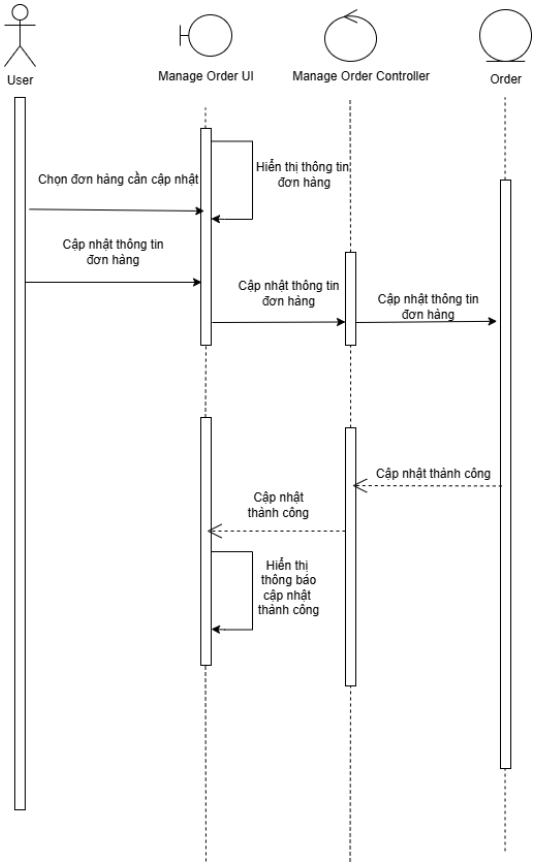
Hình 10: Biểu đồ tuần tự ca sử dụng quản lý người dùng

4.2.9 Đánh giá sản phẩm



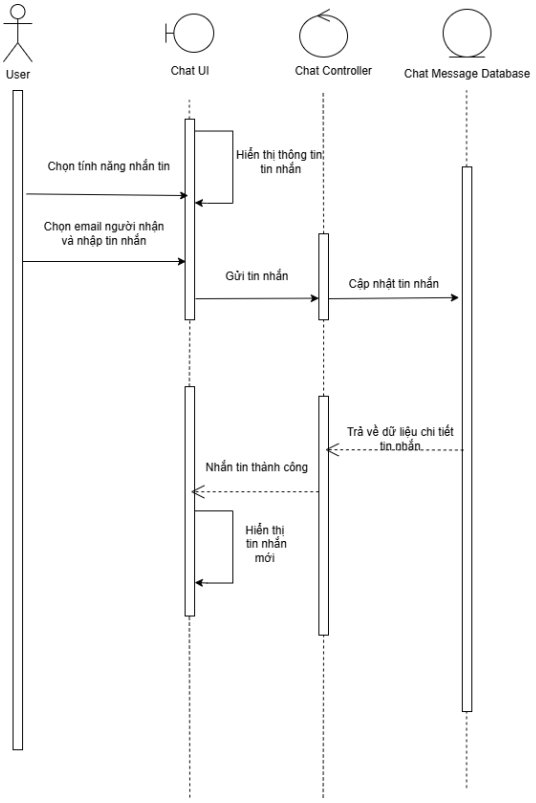
Hình 11: Biểu đồ tuần tự ca sử dụng đánh giá sản phẩm

4.2.10 Quản lý đơn hàng



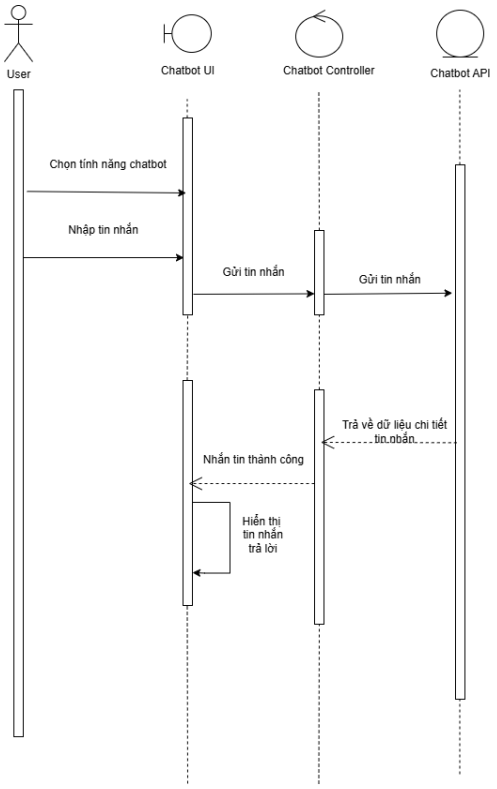
Hình 12: Biểu đồ tuần tự ca sử dụng quản lý đơn hàng

4.2.11 Nhắn tin



Hình 13: Biểu đồ tuần tự ca sử dụng nhắn tin

4.2.12 Chatbot



Hình 14: Biểu đồ tuần tự ca sử dụng Chatbot

4.3 Thiết kế API

Hệ thống bao gồm các chức năng chính nhằm hỗ trợ người dùng trong quá trình mua sắm và quản lý thông tin cá nhân một cách hiệu quả. Trước tiên, người dùng cần thực hiện xác thực tài khoản để đảm bảo quyền truy cập hợp lệ, hoặc có thể đăng ký tài khoản mới nếu chưa có. Sau khi đăng nhập, người dùng có thể truy xuất và quản lý thông tin cá nhân cũng như cập nhật các chi tiết liên quan đến tài khoản của mình. Ngoài ra, hệ thống cho phép người dùng tìm kiếm và xem thông tin chi tiết về các sản phẩm hiện có như mô tả, giá cả và tình trạng hàng. Khi có nhu cầu mua sắm, người dùng có thể đặt hàng trực tiếp thông qua hệ thống và theo dõi thông tin đơn hàng của mình, bao gồm trạng thái và nội dung chi tiết từng đơn. Cuối cùng, để hoàn tất giao dịch, hệ thống cung cấp các phương thức thanh toán tiện lợi, giúp người dùng dễ dàng thực hiện việc mua hàng một cách nhanh chóng và an toàn. Định dạng trả về dữ liệu sẽ là JSON.

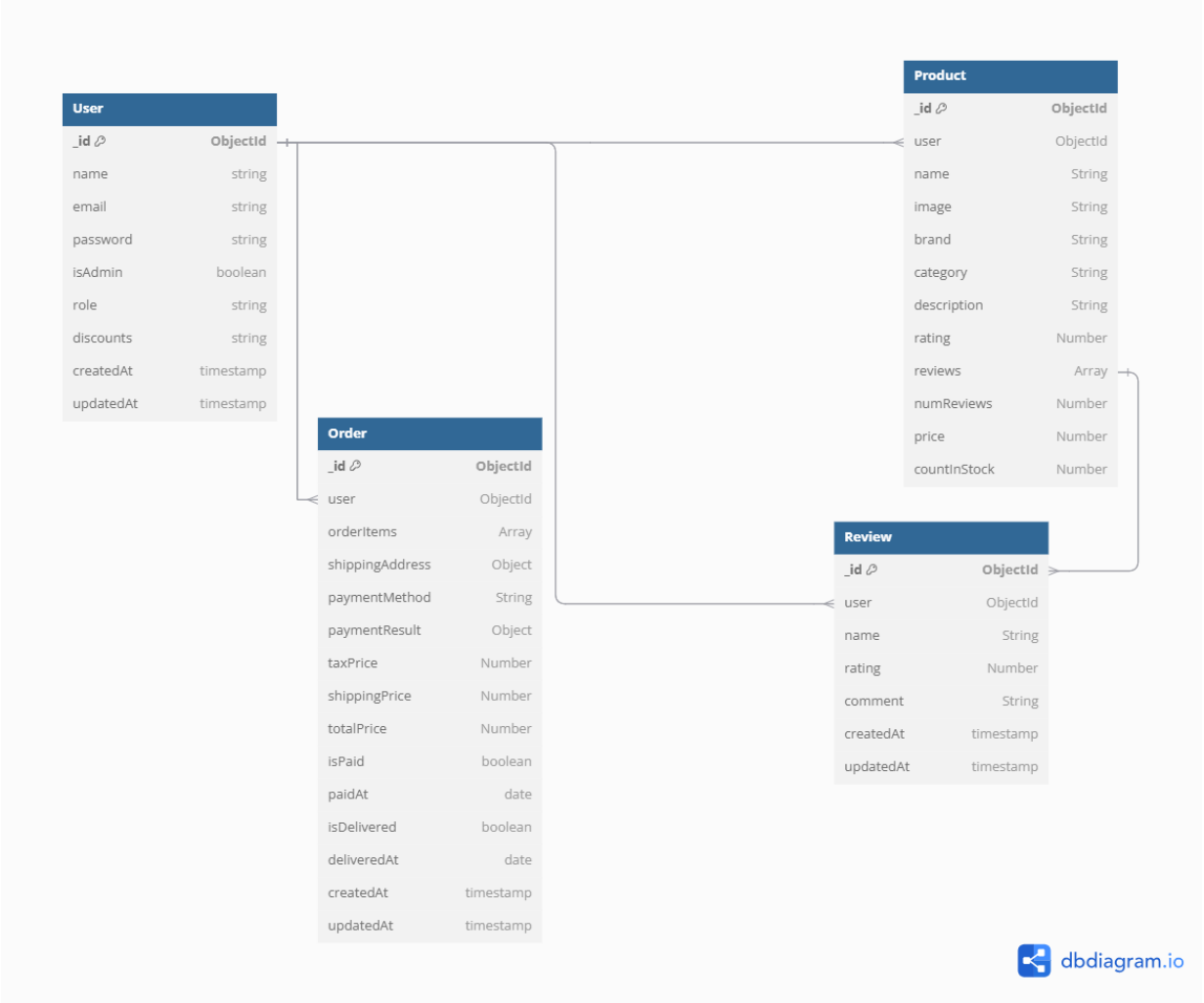
Hệ thống cung cấp nhiều endpoint phục vụ các chức năng khác nhau trong quá trình tương tác giữa người dùng và máy chủ. Để xác thực người dùng và lấy mã token truy cập, hệ thống sử dụng endpoint `POST /api/users/login`. Việc đăng ký tài khoản mới được thực hiện thông qua `POST /api/users`, trong khi đó thông tin tài khoản của người dùng đã xác thực có thể được truy xuất bằng endpoint `GET /api/users/profile`. Đối với tài khoản có quyền admin, có thể lấy thông tin người dùng theo ID bằng cách gọi `GET /api/users/:id`. Về phía sản phẩm, hệ thống cho phép lấy danh sách tất cả sản phẩm qua `GET /api/products`, hoặc xem chi tiết sản phẩm theo ID với `GET /api/products/:id`. Người dùng đã xác thực có thể tạo đánh giá sản phẩm thông qua `PUT /api/products/:id/reviews`.

Đối với đặt hàng, endpoint `POST /api/orders` cho phép tạo đơn hàng với người dùng đã đăng nhập, còn chi tiết đơn hàng có thể được truy vấn qua `GET /api/orders/:id`. Để cập nhật thông tin tài khoản cá nhân, người dùng sử dụng `PUT /api/users/profile`. Ngoài ra, hệ thống còn hỗ trợ chức năng nhắn tin và quản lý phòng chat. Việc lấy thông tin phòng chat dựa trên ID người dùng được thực hiện qua `GET /api/room/userID`, hoặc thông qua ID của hai người dùng bằng endpoint `GET /api/room/:firstUserID/:secondUserID`. Người dùng cũng có thể gửi tin nhắn thông qua `POST /api/message` và truy xuất lịch sử tin nhắn theo ID phòng chat bằng `POST /api/message/:chatRoomId`. Tất cả các endpoint này đều yêu cầu hoặc hỗ trợ xác thực để đảm bảo an toàn và quyền riêng tư cho người dùng.

4.4 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng hệ quản trị cơ sở dữ liệu MongoDB. MongoDB cho phép lưu các dữ liệu dưới dạng các cặp Key-Value, có thể dễ dàng lưu và truy vấn dữ liệu

người dùng trên Cloud thông qua công cụ MongoDB Atlas. Để có thể đưa dữ liệu của người dùng vào cơ sở dữ liệu, cần đưa chuyển đổi chúng thành các document.



Hình 15: Biểu đồ Cơ sở dữ liệu hệ thống

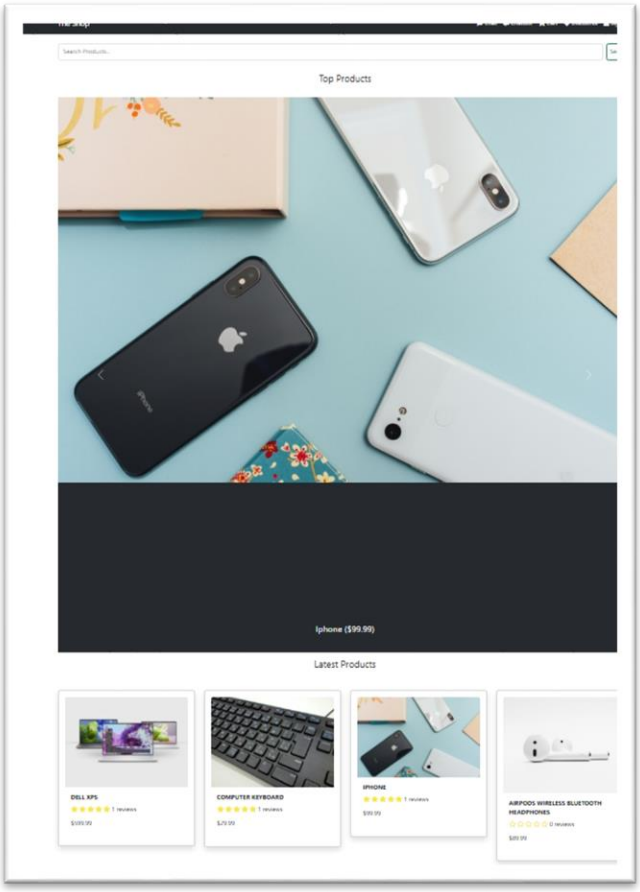
Cơ sở dữ liệu hệ thống gồm 3 collection:

- **User**: lưu trữ thông tin của người dùng trong hệ thống, có thể là người mua (buyer) hoặc người bán (seller). Người dùng có thể tạo tài khoản, đăng nhập, và thực hiện các hành động như đặt hàng, viết đánh giá sản phẩm, và quản lý sản phẩm của họ (nếu là seller).
- **Order**: lưu trữ thông tin về các đơn hàng của người dùng. Một đơn hàng bao gồm các sản phẩm mà người dùng đã mua, thông tin thanh toán, địa chỉ giao hàng, và trạng thái của đơn hàng (đã thanh toán hay đã giao).

- **Review**: lưu trữ các đánh giá sản phẩm từ người dùng. Đánh giá này có thể bao gồm tên của người viết, xếp hạng, bình luận và liên kết tới người dùng đã viết đánh giá đó.
- **Product**: lưu trữ các sản phẩm được bán trên hệ thống. Mỗi sản phẩm sẽ có thông tin chi tiết về tên, mô tả, giá cả, số lượng tồn kho, và các đánh giá từ người dùng. Sản phẩm được tạo ra bởi một người bán (seller), và mỗi sản phẩm có thể nhận được nhiều đánh giá từ người mua.

4.5 Thiết kế giao diện người dùng

4.5.1 Giao diện chính của trang web



Hình 16: Giao diện trang chủ

Hình 4.5.1 mô tả giao diện chính khi khách hàng truy cập vào trang web.

4.5.2 Giao diện đăng nhập



Hình 17: Giao diện đăng nhập

Hình 4.5.2 mô tả giao diện đăng nhập cho phép người dùng (đã có tài khoản) có thể đăng nhập vào hệ thống. Người dùng có thể là khách hàng (user) hoặc quản trị viên (admin).

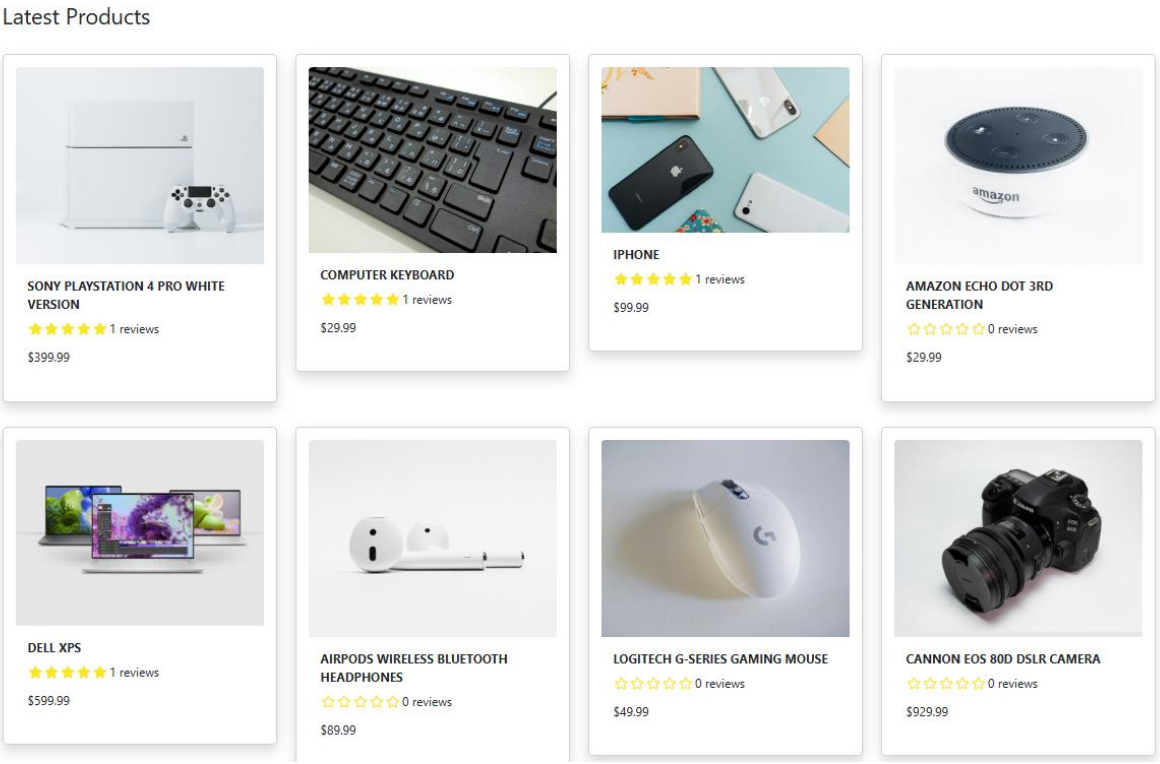
4.5.3 Giao diện đăng kí



Hình 18: Giao diện đăng ký tài khoản

Hình 4.5.3 mô tả giao diện đăng ký cho phép khách hàng (guest) có thể đăng kí tài khoản để đăng nhập vào hệ thống.

4.5.4 Giao diện sản phẩm



Hình 19: Giao diện sản phẩm

Hình 4.5.4 mô tả giao diện sản phẩm của trang web, người dùng có thể tìm kiếm, sắp xếp sản phẩm, xem thông tin chi tiết của sản phẩm, thêm vào giỏ hàng cũng như đánh giá sản phẩm.

Giao diện thành phần

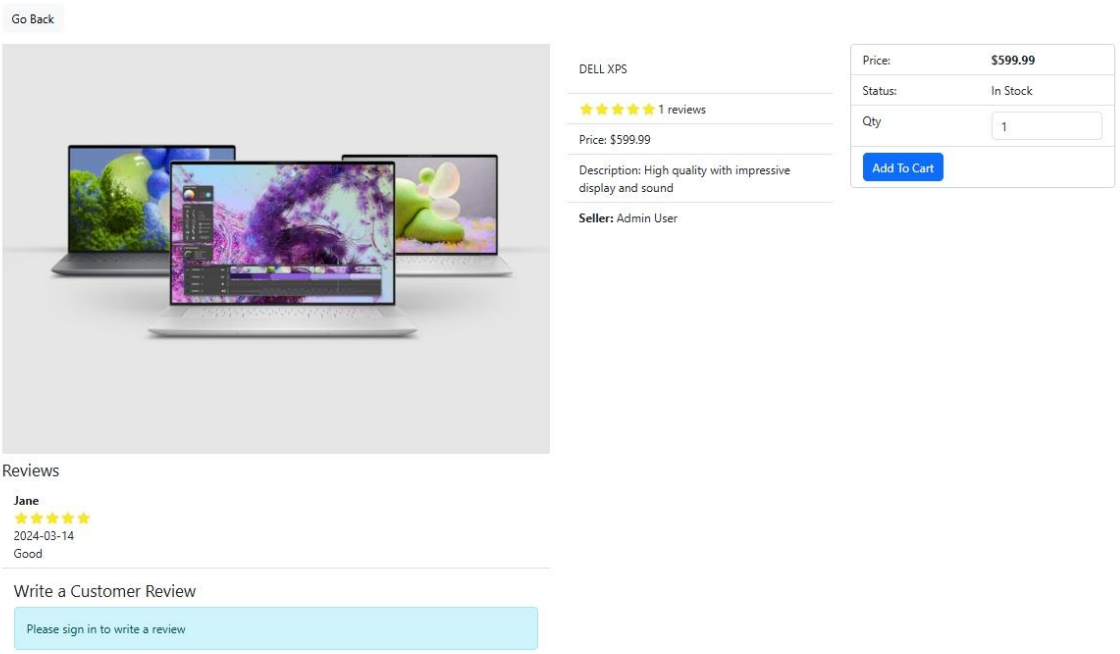
- Tìm kiếm sản phẩm



Hình 20: Giao diện tìm kiếm sản phẩm

Hình 4.5.5 mô tả chức năng tìm kiếm sản phẩm. Người dùng có thể nhập vào từ khóa và nhấn Enter để tìm kiếm.

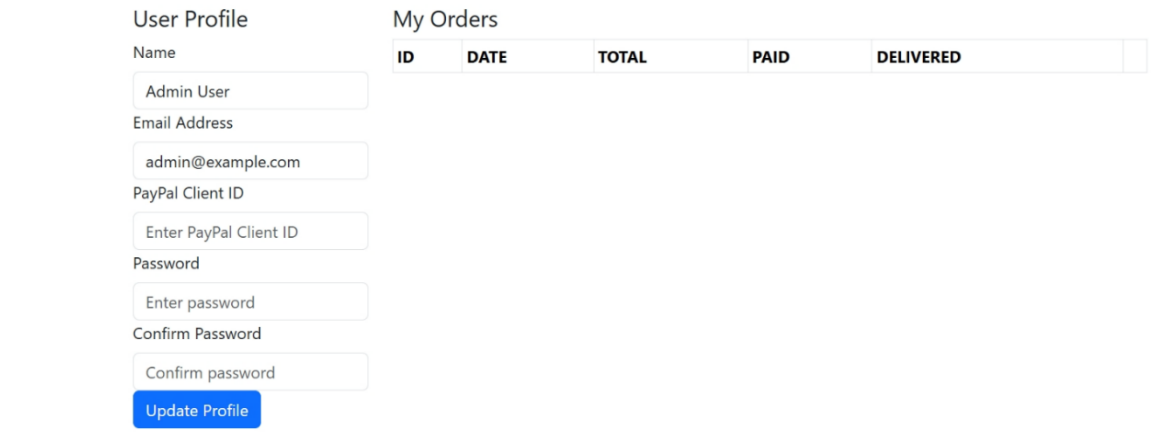
- Chi tiết sản phẩm



Hình 21: Giao diện chi tiết sản phẩm

Hình 4.5.6 mô tả thông tin chi tiết của sản phẩm. Người dùng có thể xem thông tin của sản phẩm, chọn kích cỡ cũng như số lượng của sản phẩm (nếu người dùng có nhu cầu mua sản phẩm) sau đó thêm vào giỏ hàng. Ngoài ra người dùng cũng có thể đánh giá sản phẩm.

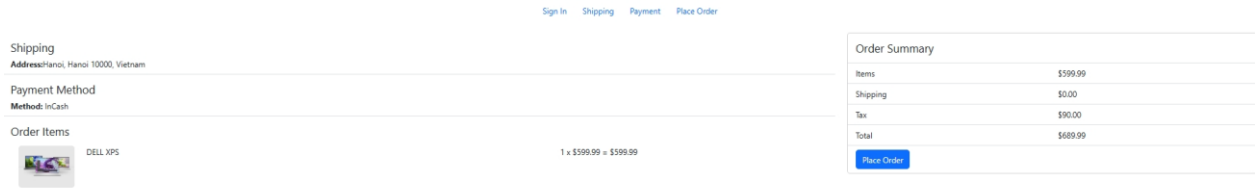
4.5.5 Giao diện profile



Hình 22: Giao diện thông tin cá nhân người dùng

Hình 4.5.7 mô tả giao diện theo dõi đơn hàng, người dùng có thể xem trạng thái cơ bản của đơn hàng cũng như xem thông tin chi tiết của đơn hàng bằng cách nhấn vào Details. Người dùng cũng có thể cập nhật thông tin tên, địa chỉ email, mật khẩu hay paypal client id.

4.5.6 Giao diện mua hàng



Hình 23: Giao diện mua hàng

Hình 4.5.8 mô tả giao diện giỏ hàng, tại đây người dùng có thể xem những mặt hàng mình đã thêm. Ngoài ra người dùng có thể thay đổi số lượng của từng loại sản phẩm cũng như xóa bớt sản phẩm không còn thích nữa. Người dùng có thể bấm vào nút Place Order để tiến hành đặt hàng.

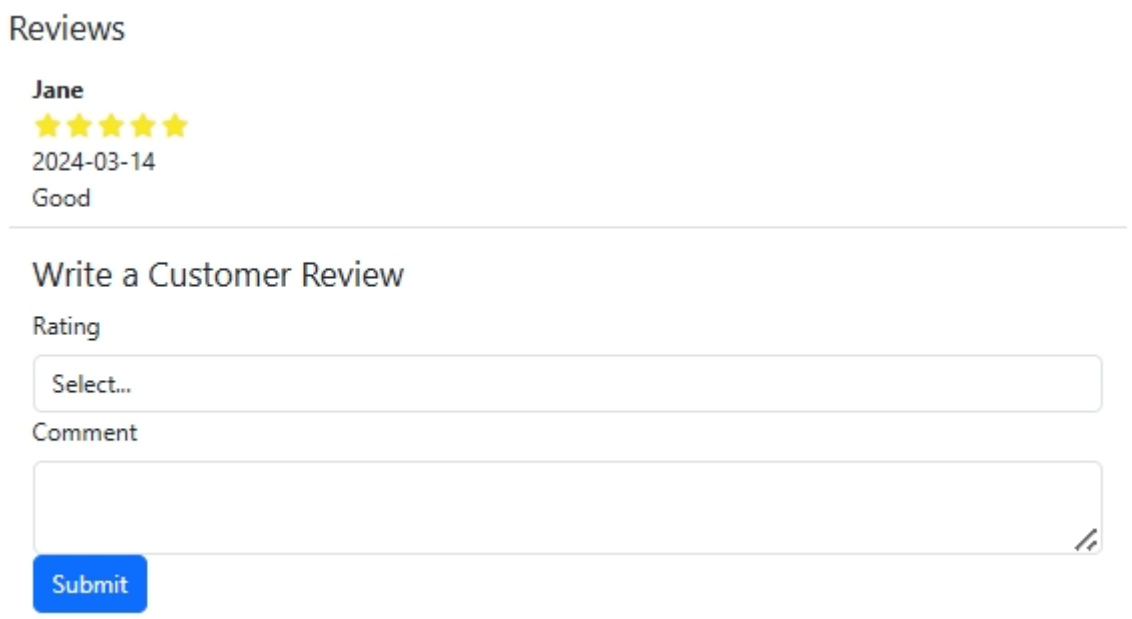
4.5.7 Giao diện đặt hàng



Hình 24: Giao diện đặt hàng

Hình 4.5.9 mô tả giao diện xác nhận đặt hàng. Người dùng có thể điền địa chỉ giao hàng và chọn phương thức thanh toán.

4.5.8 Giao diện đánh giá sản phẩm



Hình 25: Giao diện đánh giá sản phẩm

Hình 4.5.10 mô tả giao diện người dùng có thể đánh giá sản phẩm bằng cách chọn rating và viết comment. Sau đó người dùng cần chọn submit để gửi đánh giá.

4.5.9 Giao diện quản lý người dùng

Users

ID	NAME	EMAIL	ADMIN	
67400eae1cd3a40078beb36a	John Street	john@example.com	✗	
67400eae1cd3a40078beb369	Admin User	admin@example.com	✓	
67400eae1cd3a40078beb36b	Jane Street	jane@example.com	✗	
674014e6f25431b7dc4b4ee5	a	a@gmail.com	✗	
6740157e7628f3c9dca1abd3	a1	a1@gmail.com	✗	
674015ff878fc4b7a8a1a118	a12	a12@gmail.com	✗	
674016e482a5c9d965f6ce29	b	b@gmail.com	✗	
674016f082a5c9d965f6ce33	b1	b1@gmail.com	✗	
674017779d519e6304505382	b12	b12@gmail.com	✗	
674019ff4dc31508bdea4a62	f	f@gmail.com	✗	

Hình 26: Giao diện quản lý người dùng

Hình 4.5.11 mô tả giao diện quản lí người dùng của admin. Tại đây, admin có thể xóa người dùng, hoặc thay đổi quyền của người dùng bằng cách chọn chức năng edit.

4.5.10 Giao diện quản lý sản phẩm

Admin Product List + Add Product

ID	NAME	PRICE	CATEGORY	BRAND	ACTIONS
67400eae1cd3a40078beb371	Sony Playstation 4 Pro White Version	$399.99	Gaming	Sony	
67400eae1cd3a40078beb37b	Sample Product 2	$20	Sample Category	Sample Brand	
67400eae1cd3a40078beb37c	Sample Product 3	$30	Sample Category	Sample Brand	
67400eae1cd3a40078beb381	Sample Product 8	$80	Sample Category	Sample Brand	
67400eae1cd3a40078beb384	Sample Product 11	$110	Sample Category	Sample Brand	
67400eae1cd3a40078beb389	Sample Product 16	$160	Sample Category	Sample Brand	
67400eae1cd3a40078beb38d	Sample Product 20	$200	Sample Category	Sample Brand	
67400eae1cd3a40078beb392	Sample Product 25	$250	Sample Category	Sample Brand	
67400eae1cd3a40078beb39b	Sample Product 34	$340	Sample Category	Sample Brand	
67400eae1cd3a40078beb39d	Sample Product 36	$360	Sample Category	Sample Brand	
67400eae1cd3a40078beb3a0	Sample Product 39	$390	Sample Category	Sample Brand	
67400eae1cd3a40078beb3a1	Sample Product 40	$400	Sample Category	Sample Brand	
67400eae1cd3a40078beb3a2	Sample Product 41	$410	Sample Category	Sample Brand	
67400eae1cd3a40078beb3ab	Sample Product 50	$500	Sample Category	Sample Brand	
67400eae1cd3a40078beb3ac	Sample Product 51	$510	Sample Category	Sample Brand	
67400eae1cd3a40078beb3af	Sample Product 54	$540	Sample Category	Sample Brand	
67400eae1cd3a40078beb3b8	Sample Product 63	$630	Sample Category	Sample Brand	
67400eae1cd3a40078beb3c3	Sample Product 74	$740	Sample Category	Sample Brand	
67400eae1cd3a40078beb3c9	Sample Product 80	$800	Sample Category	Sample Brand	
67400eae1cd3a40078beb3d4	Sample Product 91	$910	Sample Category	Sample Brand	
67400eae1cd3a40078beb3d7	Sample Product 94	$940	Sample Category	Sample Brand	
67400eae1cd3a40078beb3d9	Sample Product 96	$960	Sample Category	Sample Brand	
67400eae1cd3a40078beb3db	Sample Product 98	$980	Sample Category	Sample Brand	

Hình 27: Giao diện quản lý sản phẩm

Hình 4.5.12 mô tả giao diện quản lí sản phẩm của admin. Admin có thể thêm sản phẩm bằng cách nhấn vào nút Add Product, xóa sản phẩm bằng cách nhấn vào Delete và thay đổi thông tin sản phẩm bằng cách nhấn vào Edit.

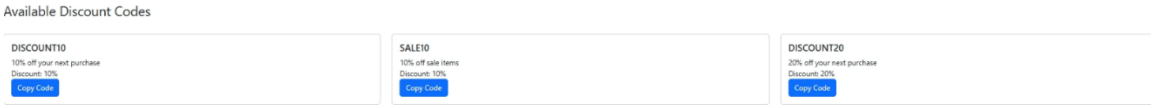
4.5.11 Giao diện quản lý đơn hàng



Hình 28: Giao diện thanh toán thành công

Hình 4.5.13 mô tả giao diện quản lí đơn hàng của admin. Tại đây admin có thể thay đổi trạng thái delivered của đơn hàng bằng cách nhấn vào details.

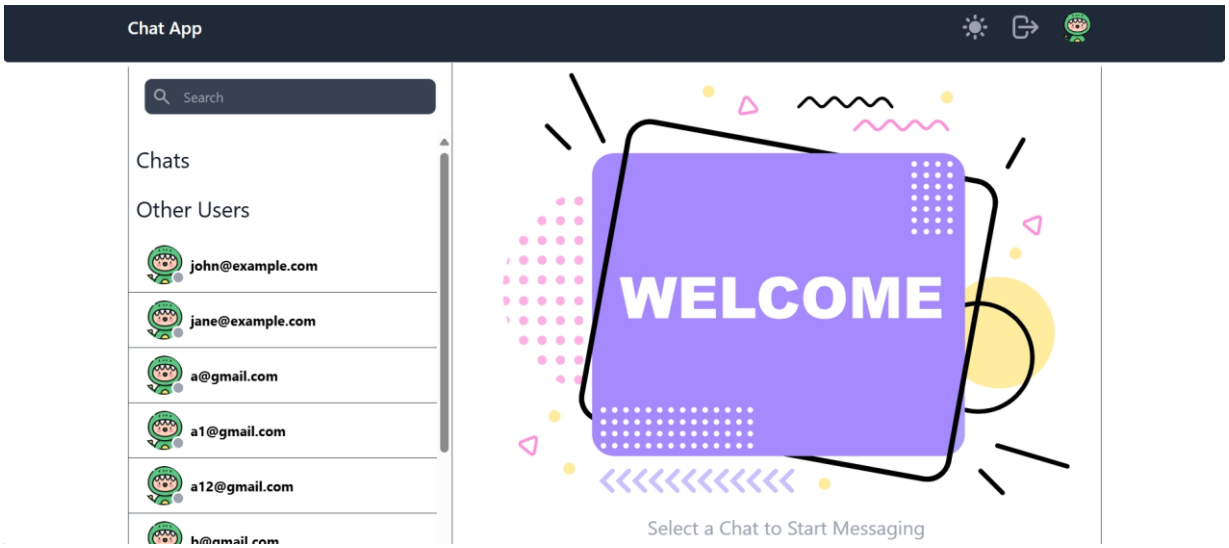
4.5.12. Giảm giá



Hình 29: Giao diện giảm giá

Hình 4.5.14 mô tả giao diện quản lý thẻ giảm giá. Người dùng có thể sử dụng mã giảm giá để giảm giá sản phẩm khi thanh toán.

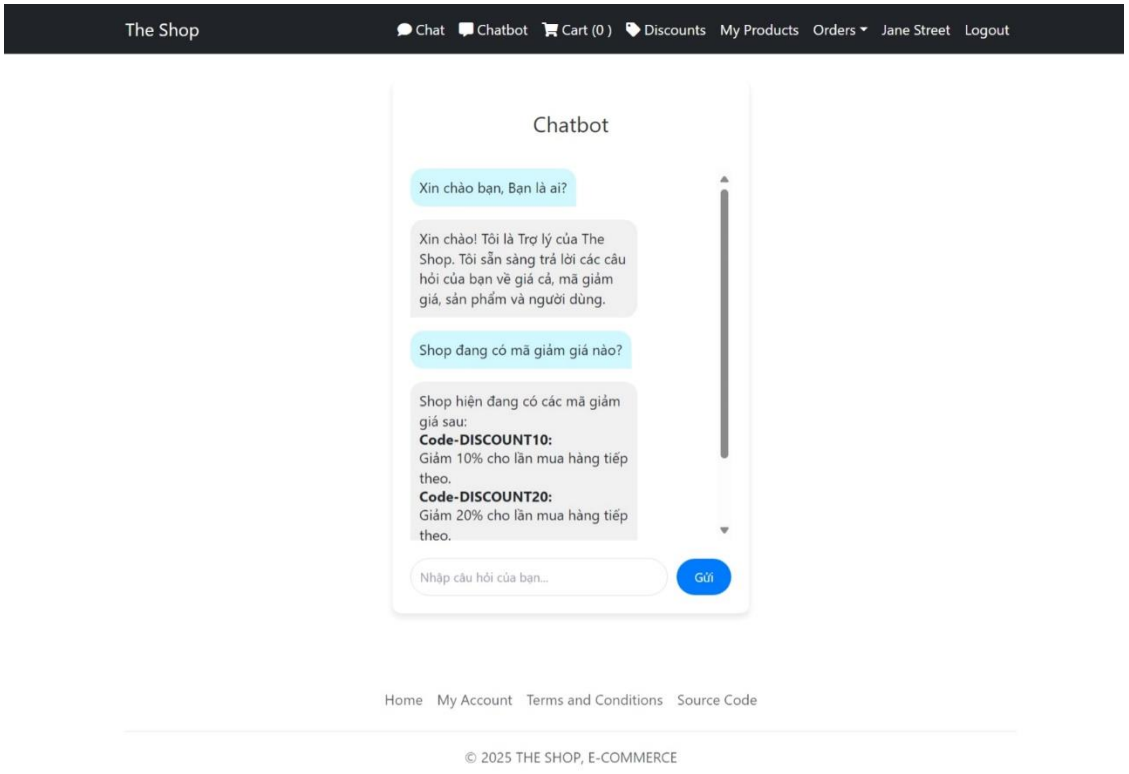
4.5.13. Nhắn tin



Hình 30: Giao diện nhắn tin

Hình 4.5.15 mô tả giao diện nhắn tin. Người dùng có thể tìm kiếm tên email người dùng, nhắn tin và hiện thông báo tin nhắn mới. Ứng dụng nhắn tin sử dụng Socket.io để hiện tin nhắn tức thời, giúp người dùng thuận tiện cập nhật thông tin từ người bán.

4.5.14. Chatbot



Hình 31: Giao diện Chatbot

Hình 4.5.16 mô tả giao diện nhắn tin với chatbot. Người dùng có thể tra cứu thông tin mã giảm giá, sản phẩm và người dùng với dữ liệu sẵn được cập nhật lúc khởi tạo dữ liệu cửa hàng với ứng dụng trí tuệ nhân tạo xử lý mô hình ngôn ngữ lớn.

Chương 5: Kết luận

5.1. Kết quả đạt được

Trong quá trình thực hiện khóa luận, tôi đã xây dựng và phát triển thành công một ứng dụng web thương mại điện tử hướng đến mục tiêu tối ưu hóa trải nghiệm người dùng và nâng cao hiệu quả quản lý sản phẩm trên nền tảng số. Hệ thống được thiết kế với giao diện thân thiện, khả năng phản hồi nhanh và kiến trúc linh hoạt, đảm bảo đáp ứng tốt các nhu cầu trong giao dịch trực tuyến. Bên cạnh đó, các tính năng cốt lõi của hệ thống đã được hiện thực hóa đầy đủ, hoạt động ổn định và phù hợp với yêu cầu thực tiễn.

Hệ thống cung cấp đầy đủ các chức năng xác thực người dùng như đăng ký, đăng nhập và đăng xuất. Người dùng có thể dễ dàng tạo tài khoản mới để sử dụng dịch vụ, đăng nhập vào hệ thống để thực hiện các thao tác mua sắm, và thoát khỏi tài khoản một cách an toàn khi không sử dụng. Bên cạnh đó, chức năng quản lý tài khoản cho phép người dùng cập nhật thông tin cá nhân, đổi mật khẩu, và theo dõi lịch sử đơn hàng. Đối với người bán và quản trị viên, hệ thống hỗ trợ phân quyền rõ ràng nhằm đảm bảo tính bảo mật và hiệu quả trong quản lý.

Chức năng hiển thị sản phẩm được phát triển với giao diện trực quan, hiển thị đầy đủ tên sản phẩm, hình ảnh, mô tả ngắn, giá bán và đánh giá từ người dùng. Người dùng có thể lọc sản phẩm theo danh mục, nhãn hiệu, mức giá hoặc mức đánh giá, giúp việc tìm kiếm sản phẩm trở nên nhanh chóng và chính xác hơn. Hệ thống tìm kiếm thông minh hỗ trợ truy xuất theo từ khóa hoặc tên sản phẩm. Khi người dùng nhấp vào một sản phẩm, một trang chi tiết sẽ được hiển thị với hình ảnh rõ nét, mô tả đầy đủ, thông tin nhà sản xuất và các đánh giá từ khách hàng trước đó.

Trong quá trình mua sắm, người dùng có thể thêm sản phẩm vào giỏ hàng, cập nhật số lượng, hoặc xóa sản phẩm. Hệ thống giỏ hàng được thiết kế đồng bộ, hỗ trợ người dùng theo dõi đơn hàng một cách thuận tiện. Các bước thanh toán được tối ưu hóa, hỗ trợ nhiều phương thức phổ biến như thẻ tín dụng, ví điện tử, chuyển khoản ngân hàng hoặc thanh toán khi nhận hàng, giúp nâng cao tính linh hoạt và tiện lợi cho người dùng. Các đơn hàng sau khi được đặt sẽ được lưu trữ và quản lý rõ ràng trong hệ thống, đảm bảo quy trình xử lý đơn hàng diễn ra nhanh chóng, chính xác và an toàn.

Hệ thống còn hỗ trợ chức năng đánh giá và nhận xét sản phẩm. Sau khi hoàn tất đơn hàng, người dùng có thể để lại nhận xét và đánh giá sản phẩm đã mua. Các đánh giá này sẽ được hiển thị công khai, góp phần tăng tính tin cậy cho các sản phẩm được bày bán trên nền tảng. Ngoài ra, website cũng tích hợp chức năng quản lý nội dung và tin tức nhằm cập nhật các thông tin hữu ích như hướng dẫn sử dụng sản phẩm, chương trình khuyến mãi và các bài viết liên quan đến thói quen tiêu dùng.

Đối với người bán và quản trị viên, hệ thống cung cấp đầy đủ các chức năng quản lý sản phẩm và đơn hàng. Người bán có thể thêm mới, chỉnh sửa hoặc xóa sản phẩm của mình.

Quản trị viên có quyền truy cập và kiểm soát toàn bộ sản phẩm trong hệ thống, đảm bảo tính công bằng, rõ ràng và tuân thủ chính sách. Tương tự, các đơn hàng được phân quyền theo vai trò: người mua có thể theo dõi đơn mua, người bán theo dõi đơn bán, và quản trị viên có quyền xem và cập nhật trạng thái giao hàng cho toàn bộ đơn hàng trên hệ thống.

Một điểm nổi bật của hệ thống là tính năng **nhắn tin trực tiếp** giữa người mua và người bán. Chức năng này cho phép hai bên trao đổi trực tiếp trong quá trình mua bán, giúp làm rõ thông tin sản phẩm, thống nhất thời gian giao hàng hoặc xử lý các yêu cầu phát sinh. Tin nhắn được lưu trữ trên cơ sở dữ liệu và hiển thị dưới dạng hội thoại trực quan. Tính năng thông báo được tích hợp giúp người dùng không bỏ lỡ các tin nhắn quan trọng, từ đó nâng cao tính tương tác và hiệu quả giao dịch.

Ngoài ra, hệ thống còn tích hợp một **chatbot hỗ trợ thông minh** sử dụng công nghệ trí tuệ nhân tạo (AI). Chatbot có khả năng hiểu ngôn ngữ tự nhiên và phản hồi linh hoạt, giúp người dùng tra cứu sản phẩm, kiểm tra đơn hàng, hướng dẫn mua hàng hoặc giải đáp thắc mắc mà không cần chờ nhân viên hỗ trợ. Chatbot có thể hoạt động song song với bộ phận hỗ trợ thủ công và tự động chuyển tiếp các yêu cầu phức tạp cho nhân viên khi cần thiết. Việc ứng dụng chatbot không chỉ tiết kiệm chi phí vận hành mà còn góp phần nâng cao trải nghiệm người dùng và mức độ chuyên nghiệp của nền tảng.

Tổng kết lại, hệ thống đã đáp ứng tốt các tiêu chí về hiệu suất, khả năng mở rộng, tính bảo mật và độ thân thiện với người dùng. Kết quả thử nghiệm thực tế cho thấy các chức năng như giỏ hàng, thanh toán, quản lý đơn hàng, quản lý sản phẩm và các tiện ích mở rộng đều hoạt động ổn định, không xảy ra lỗi nghiêm trọng trong quá trình sử dụng. Việc tích hợp các công nghệ hiện đại như hệ thống trò chuyện trực tiếp và chatbot AI đã góp phần nâng cao trải nghiệm người dùng, đồng thời thể hiện tiềm năng ứng dụng thực tiễn của đề tài trong bối cảnh phát triển thương mại điện tử hiện nay.

5.2. Định hướng phát triển

Trong khóa luận này, tôi đã đưa ra các hướng và phương pháp phát triển tiếp theo cho ứng dụng web thương mại điện tử, nhằm nâng cao hiệu quả vận hành cũng như mở rộng quy mô hệ thống trong tương lai. Một định hướng quan trọng là nâng cao khả năng tích hợp hệ thống với các nền tảng thương mại điện tử khác và các dịch vụ bên thứ ba như hệ thống vận chuyển, cổng thanh toán trực tuyến và dịch vụ chăm sóc khách hàng tự động. Việc kết nối này sẽ giúp xây dựng một hệ sinh thái thương mại điện tử hoàn chỉnh, tạo ra trải nghiệm mượt mà và liền mạch cho người dùng từ lúc chọn mua đến khi nhận hàng.

Ngoài ra, khóa luận cũng đưa ra phương pháp phát triển theo hướng ứng dụng các công nghệ hiện đại như trí tuệ nhân tạo (AI) và học máy (Machine Learning) để cá nhân hóa trải nghiệm người dùng. Cụ thể, hệ thống có thể áp dụng các thuật toán gợi ý sản phẩm dựa trên hành vi mua sắm, lịch sử tìm kiếm và sở thích cá nhân, từ đó nâng cao mức độ

hài lòng và gia tăng tỉ lệ chuyển đổi. Việc triển khai các chức năng quản lý kho hàng thông minh, tự động hóa quy trình xử lý đơn hàng và tối ưu tuyến giao hàng cũng được xem là những bước phát triển tất yếu giúp hệ thống vận hành hiệu quả và tiết kiệm chi phí.

Cuối cùng, tôi đã đưa ra các hướng phát triển ứng dụng theo hướng mở rộng ra thị trường quốc tế, bao gồm việc hỗ trợ đa ngôn ngữ, đa tiền tệ và khả năng cấu hình linh hoạt theo từng khu vực địa lý. Điều này giúp nền tảng tiếp cận được nhiều đối tượng người dùng trên toàn cầu, đặc biệt là các doanh nghiệp muốn khai thác thị trường nước ngoài. Bên cạnh đó, hệ thống sẽ tiếp tục được nâng cao các tính năng bảo mật, nhằm đảm bảo an toàn tuyệt đối cho dữ liệu cá nhân và tài chính của người dùng, góp phần xây dựng một môi trường thương mại điện tử đáng tin cậy và bền vững.

Tài liệu tham khảo

[1] C. J. Ihrig, Full Stack JavaScript Development with MEAN, O'Reilly Media, 2017.

[2] E. P. Alex Banks, Learning React, O'Reilly Media, 2018.

[3] M. Casciaro, Node.js Design Patterns, Packt Publishing, 2014.

[4] S. A. Thomas, Node.js 8 The Complete Guide, Independently published, 2018.

[5] E. Hahn, Express in Action, Manning Publications, 2018.

[6] A. Mead, Learning Node.js Development, Packt Publishing, 2016.

[7] K. Banker, MongoDB in Action, Manning Publications, 2011.

[8] K. Chodorow, MongoDB: The Definitive Guide, O'Reilly Media, 2013.

[9] A. &. P. E. Banks, Learning React, O'Reilly Media, 2018.

[10] S. Stefanov, React Up and Running, O'Reilly Media, 2016.

[11] M. Bertoli, React Design Patterns and Best Practices, Packt Publishing, 2018.

[12] B. M., Learning Redux, Independently published, 2019.

[13] M. T., Redux Essentials, Independently published, 2019.

[14] J. W., Redux for Beginners, Independently published, 2018.